

JavaScript Programmierung

Eine ~~kompatibles~~ ~~buntes~~ unterhaltsame ~~Script~~ Einführung

Hong-Phuc Bui

Sommer 2025

Inhaltsverzeichnis

Richtig Lesen	1
1. JavaScript kennenlernen	2
1.1. Aufbau eines JavaScript-Programms	2
1.2. Lexikalische Struktur	7
1.3. Eingabe und Ausgabe	9
1.3.1. Eingabe	9
1.3.2. Ausgabe	9
1.4. Datentypen	10
1.4.1. Begriffserklärung	10
1.4.2. String	16
1.4.3. Number	17
1.4.4. Boolean	25
1.4.5. Sondern Werten	27
1.5. Typ-Konvertierung	29
1.5.1. Der Operator typeof	29
1.5.2. Der Operator Plus (+)	29
1.5.3. Die Vergleich-Operatoren	30
1.5.4. Identität (===) und Gleichheit (==)	30
2. Ablaufsteuerung	34
2.1. Der if -Befehl	34
2.1.1. Syntax	34
2.1.2. Verschachtlung von else if -Befehlen	36
2.1.3. Dangling else	37
2.2. Die while -Schleife	40
2.2.1. Syntax	40
2.2.2. Präfix- und Postfix- Operatoren	42
2.3. Die for -Schleife	44
2.4. Gültigkeitsbereich eines Bezeichners	46
2.5. Verschachtlung	48
2.6. Beispielprogramme	50
2.7. Weitere Konstruktionen zur Ablaufsteuerung	53
2.7.1. Der bedingte (ternäre) Operator	53
2.7.2. Der Befehl do-while	54
2.7.3. Der Befehl break	54
2.7.4. Der Befehl continue	55
2.7.5. Der Befehl switch	55
2.8. Ausnahme-Behandlung	55

3. Modulare Programmierung	58
3.1. Funktion	58
3.1.1. Definition einer Funktion	58
3.1.2. Name einer Funktion	60
3.1.3. Hoisting	62
3.1.4. Callstack	62
3.1.5. Scope einer Variable in Funktion	64
3.1.6. Mathematische Funktionen	64
3.2. Bibliothek	69
3.2.1. Aufbau eines JavaScript-Moduls	69
3.2.2. API	71
3.3. DOM	72
3.3.1. Struktur eines DOM	72
3.3.2. Operationen auf DOM	74
3.4. Date	75
A. Ganzen Zahlen	77
B. Git Kompakt-Kurs	80
B.1. Installation	80
B.1.1. Für Linux	80
B.1.2. Für MacOS	80
B.1.3. Für Windows	80
B.2. Grundbegriffe	81
B.3. Benutzung von Git	82
B.4. Die Kommunikation zwischen Repositories	84
B.5. Bonus: vimdiff	85
C. Node und NPM	86
C.1. NPM Package Management	86
C.1.1. Installation der notwendige Software	86
C.1.2. Beispiel: Aufbau eines NPM-Projekt	88

Abbildungsverzeichnis

1.1.	Hello World Programm in JavaScript	2
1.2.	Ausgabe des Texts „Hello World“ in Chrome	2
1.3.	Die generelle Struktur eines JavaScript-Programm mit Modul terminal	4
1.4.	Ausgabe von „Hello World“ auf dem Browser	4
1.5.	Tokens in das Programm „Hello World“	7
1.6.	Zusammenhang von Identifier, Wert und Referenz	13
1.7.	Aufbau eines Ausdruck	13
1.8.	Aufbau einer Gleitkommazahl in JavaScript	19
1.9.	Zuordnung einer Zahl $r \in [0, 1)$ zu einer ganzen Zahl in $[\min, \max]$	24
1.10.	Würfel	33
2.1.	Aufbau eines if -Befehl	35
2.2.	Farbspektrum	38
2.3.	Aufbau einer while -Schleife	40
2.4.	Aufbau einer for -Schleife	45
2.5.	Lexikalische Scope von Variable	46
2.6.	Ausgabe des Programm 2.5	48
2.7.	Zugriff-Bit in Linux-Dateisystem	54
2.8.	Symmetry 131 [4]	57
3.1.	Definition einer Funktion	59
3.2.	Illustration von Callstack des Programmes 3.1	63
3.3.	Newton-Verfahren zur Berechnung von $c = \sqrt{4.657}$; $f(x) = x^2 - 4.657$	67
3.4.	Baum-Darstellung der Datei plain-calendar.html	73
3.5.	Drawing Hands [3]	76
C.1.	Erfolgreiche Installation von NPM , Beispielausgabe im Windows	86
C.2.	Erfolglose Installation von NPM , Beispielausgabe im Windows	87
C.3.	Die Installation von node-gyp ist erfolgreich abgeschlossen	87

Richtig Lesen

Unterschiedliche Literatur wird unterschiedlich gelesen. Beim Lesen von diesem Skript sollen Sie sich an einem Schreibtisch setzen. Ein für Programmierung eingerichteter Rechner soll direkt griffbereit auf dem Schreibtisch stehen. Sie brauchen auch einen Stift und Schmierpapier, um Ideen zu skizzieren. Lesen Sie dieses Skript nicht, wie Sie einen Roman lesen würden.

Dieses Skript ist so konzipiert, dass Sie sich –als Lernenden– den Lernstoff selbst beibringen. Der Lerninhalt des Skriptes wird nicht vom Dozenten in der Vorlesung frontal vorgetragen, sondern von Lernenden aktiv selbst vor der Vorlesung erarbeitet. In der Vorlesung können Sie auch die Fragen, die während der Einarbeitung des Lerninhaltes entstehen, dem Dozenten stellen.

Es gibt unterschiedlichen Sorten von Übungen. Die Fingerübungen sind kleine Übungen, die im ganzen Skript verteilt sind. Diese Übungen sind so konzipiert, damit der Lernende den eigenen Wissensstand selbst überprüfen kann. Sie können aber auch so konzipiert sein, dass der Lernende durch das Lösen der Fingerübungen neue Kenntnisse gewinnen, die die Voraussetzung für den nächsten Lerninhalt sind. Deshalb sollen Sie die Fingerübungen sofort lösen, wenn Sie sie lesen.

Übungen in den Übungsblättern sind in der Regel anspruchsvoller als Fingerübungen und nehmen mehr Zeit in Anspruch. Diese Übungen sollen vor den jeweiligen Veranstaltungen von den Lernenden vorbereitet werden.

Sie brauchen für diese Vorlesung folgende Entwicklung-Software:

- WebStrom: Eine IDE für Web-Entwicklung. Studenten der HTW Saar können sich mit HTW-E-Mail registrieren, um Professional Version zu nutzen. Link zum Download:
<https://www.jetbrains.com/webstorm/>
- Git: Programm um Source Code zu verwalten. Link zum Download:
 - Linux: Lesen Sie die Dokumentation Ihrer Distribution, um ein Programm zu installieren.
 - Windows: <https://git-scm.com/download/win>
 - MacOS: (Selbstversorgend)
- Ein oder mehrere geeignete Browser: Firefox, Chrome, Chromium, Safari sind mit brauchbaren Debugger ausgestattet.

Fingerübung 0.1: Installation von Entwicklung-Software

- Installieren Sie die oben genannten Software auf Ihrem Rechner, bevor Sie weiter lesen.
- Wie können Sie testen, ob Ihre Installation erfolgreich abgeschlossen ist und ob die installierte Software richtig funktioniert?
- Benutzen Sie Git, um die Repository
<https://bayes.htwsaar.de/gitblit/summary/?r=javascript-vorlesung/example-projects-2026.git>
auf Ihrem Rechner zu klonen.

1. JavaScript kennenlernen

Softwareentwicklung ist ein Prozess, in dem man die Realität im Programm-Code abbildet, um ein (reales) Problem zu lösen. Naturgemäß kann Programm-Code nicht alle Aspekte und Details der Realität reflektieren. Ein geeignetes abstraktes Modell der Realität zu erstellen, um das bevorstehende Problem zu lösen, ist ein der wichtigsten Schritte in Softwareentwicklungsprozess. Somit ist die Abstraktionsvermögen ein der entscheidenden Kompetenzen einer/eines Softwareentwickler*in. Nun gibt es eine gute Nachricht: Diese Kompetenz ist einfacher zu erwerben, als z.B. pädagogisches Lehrbuch zu schreiben. Deshalb haben Student*in*en die leichter Arbeit als Pädagoge*in, mindestens im Informatik-Bereich.

1.1. Aufbau eines JavaScript-Programms

JavaScript ist eine interpretierte Sprache. Sie läuft in einer Laufzeit-Umgebung, meistens ein Browser. In einer Browser-Umgebung werden `js`-Dateien zusammen mit einer `html`-Datei in einem Browser geladen. Die Abbildung 1.1 zeigt eine minimalistische `html`-Datei (Links, `helloworld.html`) mit einer verknüpften `js`-Datei (Rechts, `helloworld.js`).

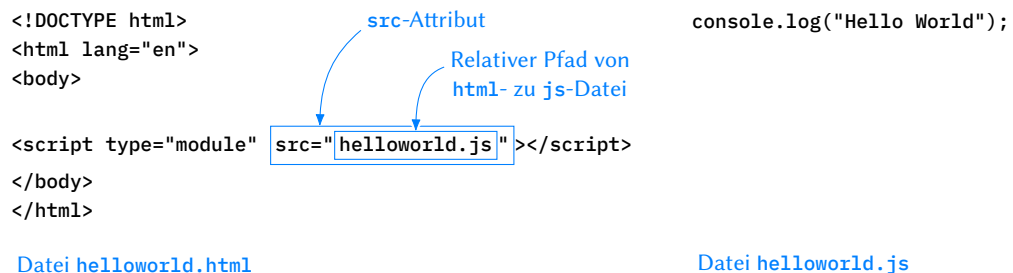


Abbildung 1.1.: Hello World Programm in JavaScript

Wird die Datei `helloworld.html` in einem Browser geöffnet, so wird der Text „Hello World“ auf der Browser-Console angezeigt. Die Darstellung des Texts kann von Browser zu Browser unterschiedlich sein. Die Abbildung 1.2 zeigt die Ausgabe des Texts in der Browser-Console von Chrome.

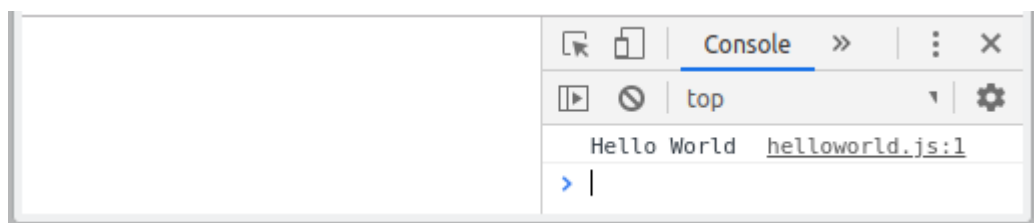


Abbildung 1.2.: Ausgabe des Texts „Hello World“ in Chrome

Der Wert des `src`-Attributs muss ein Pfad sein, der (von der `html`-Datei aus) in der `js`-Datei ausgelöst wird. Der englische Begriff ist *resolve*. Der Pfad kann relativ (zur `html`-Datei) oder absolut sein.

Pfad
Auslösung
resolve

Fingerübung 1.1: Hello World

Machen Sie sich mit Ihr bevorzugte Browser vertraut:

- Erstellen Sie das obigen JavaScript-Programm „Hello World“, indem Sie beide Dateien in einem gleichen Order speichern, z.B `helloworld/`. Die Order-Struktur wird am Ende der Aufgabestellung abgebildet.
- Öffnen Sie die Datei `helloworld.html` in Ihrem Browser.
- Finden Sie in Ihrem Browser, wie Sie die Debug-Console öffnen.

```
/helloworld/  
├─ helloworld.html  
└─ helloworld.js
```

Globale Gültigkeitsbereich Der einzige Befehl in der Datei `helloworld.js` nennt man ein Befehle in der globale Gültigkeitsbereich (*Scope*). Befehle, die nicht ein einer Funktion geschrieben sind, gehören zur globalen Gültigkeitsbereich. Diese Befehle werden unmittelbar vom Browser ausgeführt, wenn die `js`-Datei in einem Browser geladen wird.

Scope

Ausgabe auf dem Browser Daten in einem JavaScript-Programm können auf dem Browser dargestellt werden. Für den ersten Kapitel der Vorlesung werden wir das Objekt `terminal` aus dem Modul `dfhi.js` benutzen, um Daten wie Texten, Zahlen, Objekten direkt auf den Browser anstatt in der Browser-Console zu zeigen. Die generelle Struktur eines JavaScript-Programm, das das Modul `dfhi.js` verwendet, wird in der Abbildung 1.3 gezeigt. Öffnet man die Datei `helloworld.html` als Website von einem HTTP-Server, kann man den Text „Hello World“ wie in Abbildung 1.4 sehen. Zu beachten ist das Attribut `type` vom Tag `src`. Dessen Inhalt muss exakt `module` sein, damit man das Modul `dfhi.js` benutzen kann.

An diese Stelle gehen wir noch nicht in Detail, wir benutzen die Funktion `terminal.print` bzw. die Funktion `terminal.print1` um Daten auf dem Browser darzustellen.

Die notwendigen Module und weitere Beispielen findet man in der Git-Repository, unter dem Verzeichnis `/textbased-programms`.

Datei helloworld.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>plain-html</title>
  <link rel="stylesheet" href="dom-output.css" />
  <link rel="stylesheet" href="dfhi.css" />
</head>

<body>
  <code>main</code>'s arguments:
  <input type="text" id="argv" name="argv" />
  <button name="run" id="run" >Rerun</button>

  <pre id="text-output"></pre>
  <script type="module" src="helloworld.js">
</script>
</body>
</html>

```

type-Attribut muss
module sein

src-Attribut zeigt auf
eine js-Datei

Datei helloworld.js

```

import {terminal} from './dfhi.js'

terminal.print("Hello World");

```

Pfad zeigt auf die
importierte Datei

terminal ist nach dem import
ein global verfügbares Objekt

Abbildung 1.3.: Die generelle Struktur eines JavaScript-Programm mit Modul terminal

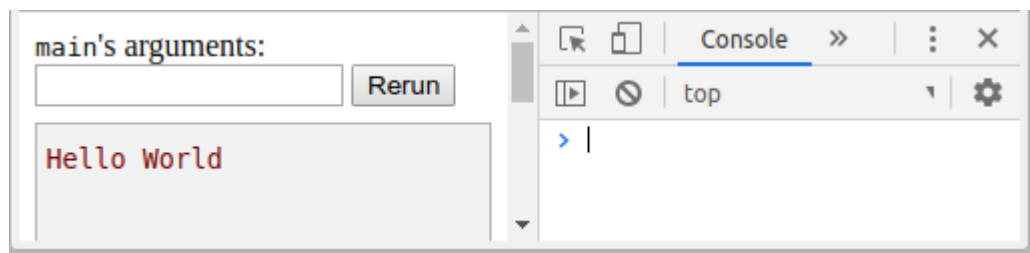


Abbildung 1.4.: Ausgabe von „Hello World“ auf dem Browser

Fingerübung 1.2: WebStorm

1. Öffnen Sie den Ordner **textbased-programms** als ein Projekt im WebStorm. (File → Open File or Project → .../example-projects-2026/textbased-programms → Ok)
2. Öffnen Sie die Datei **helloworld-terminal.html** in dem Editor-Teil von WebStorm, indem Sie die in dem Project-Reiter gezeigte Datei anklicken.
3. Öffnen Sie die Datei **helloworld-terminal.html** in einem Browser *vom WebStorm aus*, indem Sie die Taste-Kombination **Strg** + **⌘** + **F10** drücken. Alternative können Sie den Befehl Run 'helloworld-terminal.html' über den Kontext-Menü ausführen.
4. Was wird in der Adress-Zeile vom Browser angezeigt? Was ist der Unterschied wenn Sie die Datei **helloworld-terminal.html** direkt von dem „Datei Explorer“ in dem Browser öffnen?

Die Funktion `window.main` Programme, die keine Eingabe von Benutzer verarbeiten, sind weniger interessant. Die Funktion `window.main` bietet eine Möglichkeit an, ein JavaScript Programm mehrfach mit unterschiedlichen Eingabe der Benutzer auszuführen.

Schreibt man den JavaScript Code nicht in der globalen Gültigkeitsbereich sondern im Rumpf der Funktion `window.main`, wird diese Code bei dem Laden der Website im Browser nicht unmittelbar ausgeführt. Diese Code wird nur ausgeführt, wenn der Benutzer auf dem Knopf „Rerun“ anklickt. Das Programm 1.1 zeigt eine Anwendung von `window.main`.

Programm 1.1.: Nutzung von `window.main` (hello-name)

```
import {terminal} from "./dfhi.js";

window.main = (...argv) => {
  const name = argv[0];
  terminal.print(`Hallo ${name}!`);
  //${name} replaces name by value of variable name.
}
```

Der Benutzer kann den Namen in der Eingabe-Field eingeben und auf „Rerun“ anklicken um einen Text auf dem Browser anzeigen zu lassen.

Die Funktion `window.main` erwartet einen Array von String als Argument. Der Eingabetext des Benutzers wird in einem Array von String aufgeteilt indem der Text in Teilstring zerlegt wird, wobei die Leerzeichen als Trennzeichen verwendet wird, um zu bestimmen, wo die Aufteilung erfolgen soll. Die Eingabe des Benutzer ist immer ein String, gibt der Benutzer nichts ein, wird ein Leerstring als Eingabe verwendet. Der Array `argv` hat in dieser Situation ein Element als Leerastring (`argv[0]` ist ""). Möchte man die Eingabe anders als String weiter verarbeiten, muss man sie zuerst in erwartete Datentyp konvertieren. Das Programm 1.2 berechnet die Summe von zwei Zahlen, die in Eingabe-Feld eingegeben werden.

Programm 1.2.: Summe von zwei Zahlen rechnen

```
import {terminal} from "./dfhi.js";

window.main = (...argv) => {
  const n1 = Number(argv[0]);
  const n2 = Number(argv[1]);
  const s = n1 + n2;
  terminal.print(s);
};
```

Ausführung eines JavaScript-Programms Da die Datei `helloworld.html` nur als Entry-Point (Einstiegspunkt) eines JavaScript-Programm dient, und deren Struktur sich in den ersten Kapitel nicht viel ändert, werden wir generell nur über den Inhalt der `js`-Datei diskutieren, wenn wir von einem JavaScript-Programm sprechen. Es wird dennoch implizit zu jeder `js`-Datei eine gleichnamige `html`-Datei mit dem entsprechenden Wert des Attributes `src` erstellt.

Fingerübung 1.3: Pfad-Konzept

Gegeben ist eine Ordner-Struktur wie folgt:

```
/
├─ helloworld.html
└─ js
   └─ helloworld.js
```

Recherchieren Sie den Begriff „Path“, „relative Path“, „absolute Path“, „resolve“ und beantworten Sie folgenden Fragen:

1. Wie muss der Wert des `src`-Attribut sein, damit der Pfad von der Datei `helloworld.html` zu der Datei `helloworld.js` ausgelöst wird? Geben Sie den Werten als relative Pfad und absolute Pfad an.
2. Was ist ein *File path separator*-Zeichen? Welches Zeichen wird in Windows, Linux, Unix, MacOS Betriebssysteme verwendet, welches Zeichen wird in World Wide Web verwendet?
3. Welches *File path separator* Zeichen müssen Sie als Wert des `src`-Attribut als *File path separator* benutzen?

Fingerübung 1.4: Ordner Struktur eines Projekt

1. Öffnen Sie das Projekt `textbased-programms` aus der Repository, die Sie in der Finger Übung 0.1 in Ihrem Rechner geklont haben, in WebStrom.
2. Öffnen Sie die Datei `helloworld.html` im Editor-Bereich von WebStrom, indem Sie den Dateiname doppelt anklicken; Starten Sie einen Browser *von WebStrom aus*, sodass der Browser die Datei `helloworld.html` des Projektes öffnet (`(Strg) + (⬆) + (F10)`).
3. Recherchieren Sie die Begriffe „URL“, „Protocol“, „host“, „port“, in dem Kontext von URL und beantworten Sie folgenden Fragen:
 - a) Wie sieht der URL aus, den WebStrom verwendet, um die Datei `helloworld.html` zu öffnen?
 - b) In diesem URL, wie lauten der Host-, Port- und Path-Teilen?
4. Was passiert wenn die Datei `helloworld.html` direkt vom Programm File Explorer aus in einem Browser geöffnet werden? Beschreiben Sie die Ausgabe vom Browser-Console.
5. Ändern Sie das JavaScript Programm `helloworld-terminal/helloworld.js` so, dass der Text „Hallo Welt“ auf dem Browser dargestellt wird.

Fingerübung 1.5: Benutzer-Eingabe via `window.main(...argv)`

- Erstellen Sie ein Programm „Hallo mich“ aus eine `html`-Datei `hallomich.html` und eine `js`-Datei `hallomich.js`. Das Programm soll den Text „Hallo ...“ auf dem Browser ausgeben, wobei statt „...“ soll dort Ihr Name stehen, zum Beispiel: „Hallo Peter“ (Sieh

das Programm 1.1).

- Überprüfen Sie ob Ihr Programm funktioniert wie Sie es erwarten.
- Sie können den Inhalt des HTML-Tag `title` auf „Hallo mich“ setzen.
- Kopieren Sie den Order `helloworld-terminal` in einem neuen Ordner `my-js-project`.

1.2. Lexikalische Struktur

Ein JavaScript Programm besteht aus eine Reihe von JavaScript-Token. Token sind Code-Segmenten, die einen lexikalische Regel erfüllt. Die Reihenfolge der Token in einer Programmieren-Sprache sind fest definiert. Diese Definition nennt man in Informatik die Syntax der Sprache. Die Abbildung 1.5 zeigt die Tokens im Programm „Hello World“. Detail über jeweiligen Token werden in den nächsten Kapitel besprochen.

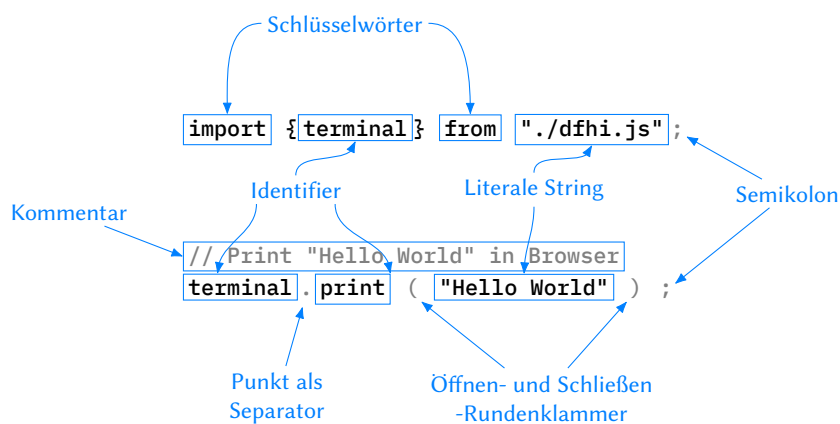


Abbildung 1.5.: Tokens in das Programm „Hello World“

Groß- und Kleinschreibung JavaScript unterscheidet Großbuchstaben von Kleinschreibung. Während der Code

```
terminal.print("Hello World");
```

einen Text auf dem Browser ausgibt, verursacht der Code

```
Terminal.print("Hello World");
```

einen Fehler, der im Browser-Console dargestellt wird. Die Markup-Sprache HTML unterscheidet jedoch nicht von Groß- und Kleinschreibung. Sie können sowohl

```
<script src="helloworld.js">
```

als auch

```
<Script Src="helloworld.js">
```

schreiben. Konventionell werden alle HTML-Tags klein geschrieben. In diesem Kurs werden die HTML Tags nur klein geschrieben.

Unterscheidet eine Sprache die Großbuchstaben von Kleinbuchstaben, so nennt man die Sprache „case-sensitive“. Anderenfalls nennt man die Sprache „case-insensitive“.

Fingerübung 1.6: Groß- und Kleinbuchstaben

Funktioniert das Programm „Hello World“ noch, wenn statt

```
<Script Src="helloworld.js"></Script>
```

wird

```
<Script Src="HelloWorld.js"></script>
```

geschrieben? Begründen Sie Ihre Antworten.

Whitespace Whitespace sind Leerzeichen, Tabulator und Newline zwischen Token. JavaScript ignoriert die Whitespace. So können Sie Ihr SourceCode ordentlich formatieren. JavaScript ist etwa empfindlich mit Zeilenumbrüche, wie wir in Zusammenhang mit Semikolon im nächsten Kapitel sehen.

Semikolon In JavaScript werden einfache Befehlen mit Semikolon beendet. Der Semikolon ist ein Bestandteil der Grammatik vom JavaScript Sprache. Der Lexer vom Browser kann jedoch den Semikolon automatisch hinzufügen, sodass der Programmierer Semikolon weglassen kann. Das Programm „Hello World“ kann auch wie folgt geschrieben werden:

```
terminal.print("Hello World")
```

und hat die gleiche Bedeutung wie die Version mit Semikolon. Dennoch

```
return  
true;
```

hat die gleiche Bedeutung wie

```
return;  
true; // ← this line of code is never executed.
```

Was Sie wahrscheinlich erwarten, ist

```
return true;
```

Wenn eine Codezeile so umgebrochen wird, dass die Zeile vor dem Umbruch eine vollständige JavaScript Anweisung erscheint, wird ein Semikolon vor dem Umbruch hinzugefügt.

Das Weglassen von Semikolon ist deshalb ein schlechter Programmierstil. Sie sollen Semikolon ar immer anwenden.

Kommentar Sämtliche Texte zwischen // und einem Zeilenumbruch werden als (einzeiliges) Kommentar behandelt und von JavaScript ignoriert.

Auch sämtliche Texte zwischen /* und */ wird als (mehrzeiliges) Kommentar behandelt. Mehrzeilige Kommentare können nicht in einander verschachtelt werden.

1.3. Eingabe und Ausgabe

Sowohl das Programm 1.1 als auch das Programm 1.2 arbeiten nach dem Prinzip Eingabe-Verarbeitung-Ausgabe. Das heißt, sie akzeptieren die Eingabe vom Benutzer, hier über den Input-Feld, verarbeiten die Eingabe, und geben das Ergebnis der Verarbeitung aus. Das Ergebnis einer Ausführung der Programm hängt deshalb von der Eingabe ab. Unterschiedliche Eingaben führen in der Regel zu unterschiedliche Ergebnisse. Dieses Prinzip gilt nicht nur in JavaScript sondern auch in andere Programmierung-Sprachen.

1.3.1. Eingabe

In meisten Programm in diesem Skript kann der Benutzer Text in der Input-Feld mit dem `id="argv"`-Attribut eingeben. Die Eingabe ist im Prinzip einen String. Sie wird in einem Array von Strings zerlegt und als Argument der JavaScript Funktion `window.main` weiter geleitet. Leerzeichen in der Eingabe dienen als Separator bei der Zerlegung. Zum Beispiel: gibt der Benutzer den Text `12.3 5.7` in dem Feld ein, wird die Eingabe in einem Array von zwei Elementen `"12.3"` und `"5.7"` vom Typ String zerlegt. Diese beide Werten kann man in der Funktion mit `argv[0]` und `argv[1]` zugreifen, etwa

```
window.main = (...argv) => {
  const first = argv[0]; // Type String!
  const second = argv[1]; // Type String!
}
```

Möchte man die Eingabe weiter verarbeiten, muss man sie zuerst in geeignete Datentype wie im Abschnitt 1.4 und 1.5.4 konvertieren. Es ist wichtig, dass erst die Korrektheit der Eingabe überprüft wird, dann kann die Eingabe weiter verarbeitet werden.

Die Eingabe von Benutzer als einen einzigen String entgegen zu nehmen und dann in Array zu zerlegen, ist nicht gängig in JavaScript. Sie wird dennoch in diesem Skript verwendet, um die Einfachheit des Codes zu gewähren. Die gängige Methode, wie man die Eingabe von Benutzer in JavaScript entgegen nimmt, ist mehrere Input-Felde für mehrere Eingabe anzuwenden.

1.3.2. Ausgabe

Wir benutzen bisher das Objekt `terminal` um Objekten auf dem Browser zu präsentieren. Man nennt ein Programm, das Zeichen für Zeichen, Zeilen für Zeilen auf dem Bildschirm ausgibt, ein „Zeichen-“ oder „Terminal-“ orientiertes Programm. JavaScript wurde ursprünglich für Browser entwickelt. Die primäre Ausgabe von JavaScript ist deshalb das Document Object Model, abgekürzt als DOM, also mehr graphisch orientiertes Ausgabe als Zeichen-orientiert. Das Objekt `terminal` imitiert ein Pseudo-Zeichen-orientierte Ausgabe-Medium, sodass man in diesem Kurs die Konzept der Programmiersprache JavaScript schnell und einfach demonstrieren kann, zu mindestens in den ersten Kapitel. Neben `terminal.print` bzw. `terminal.printl` verfügt das Objekt `terminal` noch die Methode `terminal.prnth` um HTML-Segment auf dem Browser auszugeben. Diese Method wird nicht oft im Kurs verwendet, dennoch stellt er eine Möglichkeit da, die Ausgabe übersichtlich zu gestalten. Man kann zum Beispiel einen Text in Farbe und formatiert ausgeben:

```
const name = argv[0];
const color = 'blue';
const greeting = `

```

1.4. Datentypen

Um die Daten effizient zu verarbeiten, werden sie in unterschiedlichen Typen kategorisiert. Ein Datentyp ist eine Menge von Werten. In JavaScript können fast jede Operatoren auf Werten von jeden Datentypen angewendet werden. Oft wird eine Menge von Operatoren, die sinnvoll auf Werten von einem Datentyp anwenden lassen, mit diesem Datentyp kombiniert. Deshalb bezeichnet man in JavaScript implizit ein Datentypen als die Kombination einer Menge von Werten und der Menge der Operatoren, die sinnvoll auf der Werten anwenden lassen.

1.4.1. Begriffserklärung

Im folgt werden wir Begriffen anhand des folgenden Code-Fragmentes erklären.

```
const a = 12;
const b = 34;
const sum = a + b;
```

Der obige Code-Fragment erstellt drei Werten der Typ „Zahl“ (`number`) aus zwei Integer-Literalen `12` und `34` und einen Ausdruck (`a + b`) und verknüpft die Variablen `a`, `b`, und `sum` an diesem Werten. Das Ergebnis ist, die Variable `sum` ist ein Referenz auf dem Wert der Typ `number` mit dem Wert `46`.

Werten und Objekten

Werten wie Zahlen (3,14) und Texten („Hello World“) werden in JavaScript mit primitiven Datentypen dargestellt und verarbeitet. Die JavaScript-Sprache bietet primitiven Datentypen um Zahlen, Text, logische Ausdrücken (Tabelle 1.1) zu verarbeiten. Außerdem definiert JavaScript zwei trivialen Datentypen `null` und `undefined`, die jeweils nur einen Wert definieren.

Typ	Menge	Gängige Operationen	Beispiel
<code>number</code>	Ganzen Zahlen und Dezimalzahlen	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>%</code> <code>**</code>	<code>12</code> <code>34.5</code> <code>0xA</code>
<code>boolean</code>	Wahrheit-Werten	<code>&&</code> <code> </code> <code>!</code>	<code>true</code> <code>false</code>
<code>string</code>	Texten	<code>+</code>	<code>"Hello World"</code> <code>"12.34"</code>

Tabelle 1.1.: Primitive Datentypen in JavaScript

Streng genommen ist String kein primitive Datentyp

Ein weitere Datentyp in JavaScript ist Objekt. Ein Objekt in JavaScript stellt eine Sammlung von Werten dar. Die Werten können sowohl primitive Datentypen als auch Objekt sein. JavaScript-Objekte können eine ungeordnete Sammlung von Werten oder eine geordnete Sammlung nummerierter Werten dargestellt. Im zweiten Fall bezeichnet man das Objekt als *Array*. Objekte und Arrays sind zwar JavaScript-Objekt Datentypen, wir betrachten Objekte und Arrays dennoch in dem Skript unterschiedlich. Diese beide Datentypen werden in eignen Kapitel gewidmet.

Die JavaScript Sprache behandelt primitive Werten und zusammengesetzten Objekt unterschiedlich.

Literal

Ein Literal ist einen JavaScript Code, die einen Datenwert darstellt. Zum Beispiel die Tokens `12` und `34` in der obigen Code-Listing sind Integer Literal. Weitere Beispielen von Literal sind:

```
3.14          // a number
"Hello World" // a string
```

JavaScript unterstützt Objekt- und Array-Literal:

```
[2, 3, 5, 7, 11]          // Array-literal
{a: 49.247, l: 6.912, t: 25 } // Object-literal
```

Aus der Sicht des JavaScript-Interpreter bestehen Array Literal und Objekt Literal aus mehrere lexikalische Token.

Identifizier

Ein Identifier ist ein Token, der einen Name präsentiert. In JavaScript dienen Identifier als Variable-, Funktion-, und (lexikalische) Klasse-Name. Ein Identifier fängt mit einem Buchstaben, das Unterstrich (_) oder das Zeichen Dollar (\$) an. Die weitere Zeichen können Buchstaben, Ziffer, Unterstrich, oder Dollar sein. Da JavaScript in der Lage ist, Source Code in UTF-8 zu verarbeiten, kann man auch UTF-8 Zeichen (z.B die Buchstaben „ä“, „ö“, „ü“) in Identifier verwenden. Das ist dennoch ein sehr schlechte Idee: Der Source Code lässt sich schwierig portieren. Im Code-Listing am Anfang der Kapitel sind die Tokens `a`, `b` und `sum` (Variable)-Identifier.

Schlüsselwörter dürfen nicht als Identifier verwendet werden. Sie sind in folgenden Abschnitten aufgelistet.

Literale Werten Die Konstanten `true` und `false` sowie die besondere Werten `null` und `undefined` können nicht als Identifier verwendet werden. Der Wert `Infinity` kann man nicht überschreiben, dennoch ist es ratsam, den Token `Infinity` nicht als eignen Identifier zu benutzen.

Reservierte Schlüsselwörter Diese Schlüsselwörter sind Bestandteil vom Standart ECMAScript 2015.

<code>await</code>	<code>break</code>	<code>case</code>	<code>catch</code>	<code>class</code>
<code>const</code>	<code>continue</code>	<code>debugger</code>	<code>default</code>	<code>delete</code>
<code>do</code>	<code>else</code>	<code>export</code>	<code>extends</code>	<code>finally</code>
<code>for</code>	<code>function</code>	<code>if</code>	<code>import</code>	<code>in</code>
<code>instanceof</code>	<code>new</code>	<code>return</code>	<code>super</code>	<code>switch</code>
<code>this</code>	<code>throw</code>	<code>try</code>	<code>typeof</code>	<code>var</code>
<code>void</code>	<code>while</code>	<code>with</code>	<code>yield</code>	

Das reservierte Wort `let` wird bereits von vielen Browser implementiert. In dieser Vorlesung wird das Wort `let` als Schlüsselwort verwendet.

Zukünftig reservierte Schlüsselwörter Folgenden Schlüsselwörter haben kein besondere Funktionalität, dennoch sind sie für zukünftige Version der JavaScript reserviert. Man soll sie nicht als Identifier verwenden.

<code>enum</code>		
<code>implements</code>	<code>package</code>	<code>public</code>
<code>interface</code>	<code>private</code>	<code>static</code>
<code>protected</code>		

Fingerübung 1.7: Lexikalische Syntax von Identifier

- Nennen Sie drei Beispielen für korrekte Identifier!
- Warum sind folgenden Wörter kein gültige Identifier?
 - `length of river`
 - `1.name`
 - `letter#1`

Variable und Konstante

Eine Variable ist ein Identifier, der mit einem Wert verbunden ist. Diese Verbindung nennt man Referenz einer Variable. Man bezeichnet, dass eine Variable auf einen Wert referenziert, oder der Wert an einer Variable zugewiesen ist.

Eine Konstante ist ein Identifier, der auf einen Wert referenziert. Die Referenz einer Konstante kann nicht wie bei der Variable geändert werden. Eine Konstante in der Programmierung ist nicht notwendig eine physikalische oder mathematische Konstante.

Deklaration Eine Variable wird mit `let` deklariert, eine Konstante wird mit `const` deklariert. Konventionell schreibt man eine Konstante in Großbuchstaben. Das Beispiel am Anfang des Kapitel zeigt JavaScript Code, der drei Variablen deklariert und initialisiert. Die Variable `a` wird mit dem Wert `12` initialisiert. Die Variable `b` wird mit dem Wert `34` deklariert. Die Variable `sum` wird mit dem Summe von `a` und `b` (also die Summe von `12` und `34`, das ist `46`) initialisiert.

Weitere Beispielen: der unter stehende Code deklariert die Variable `length` und die Konstante `PI`; initialisiert sie mit jeweiligen Werten `13` und `3.14159`.

```
const length = 13;      // length ist zwar keine natürliche Konstant,  
                        // möchte man sie jedoch nicht neu zuweisen, deklariert man sie mit  
const PI = 3.14159;
```

Ein nicht deklariertes Identifier wird als Global Variable implementiert. Es ist ein schlechter Programmierstil, Variable ohne Deklaration zu verwenden.

Wenn man gleichzeitig mehrere Variablen deklarieren, kann man die Kurzform von `let` benutzen. Die Variable `a` und `b` können wie folgt deklariert und initialisiert werden:

```
const a = 12,  
      b = 34;
```

Die Einrückung vor der Variable `b` ist zwar nicht notwendig, dennoch gestaltet die Format vom Code übersichtlicher und leserlich.

Der Zusammenhang von Variable bzw. Konstante und Werten kann man wie in dem unten Diagramm 1.6 vorstellen. Die `const`-Deklaration unterbindet die nur Änderung der Referenz eines Identifier, jedoch nicht die Änderung der Wert selbst.

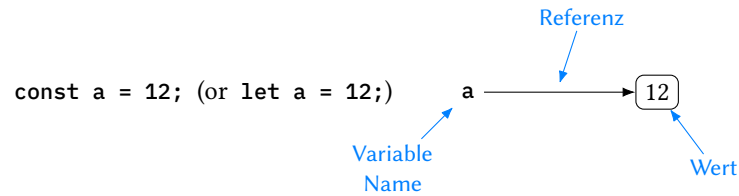


Abbildung 1.6.: Zusammenhang von Identifier, Wert und Referenz

Der Wert einer Variable bzw. einer Konstante kann man mit `terminal.print` auf dem Browser ausgeben, oder mit `console.log` in dem Browser-Console ausgeben.

Konvention der Namegebung Variable und Konstanten sind mit sinnvollen Name zu benennen. Der Name einer Variable besteht aus ein kleines Buchstabe, gefolgt von kleine Buchstaben, Großbuchstaben und Ziffer. Großbuchstaben markieren die Wörter von Variable-Namen, die aus mehrere Wörter bestehen. Wir benutzen kurzen Namen, z.B. `a`, `b`, `r` für Variable in kleinen Gültigkeitsbereich und aussagekräftige Namen, z.B. `textColor`, `perimeter` für Variable in große Gültigkeitsbereich. Konstanten-Namen bestehen aus ein Großbuchstabe, gefolgt von Ziffer, Unterstrich. Unterstrich-Zeichen trennen die Wörter in der Name. Wir können zum Beispiel die Name `GRAVITATION` oder `ELECTRIC_PERMITTIVITY` für Konstanten verwenden.

Man kann eine Variable mit `var` deklarieren. Das Gültigkeitsbereich dieser Variable sind dann anders als von der Variable, die mit `let` deklariert wurden. Das Schlüsselwort `var` soll man wegen seines außergewöhnliches Verhalten in neuen Code *nicht* weiter verwenden, stattdessen verwendet man `let` um Variable zu deklarieren. Weitere Detail über das Schlüsselwort `var` und `let` wird in spätere Kapitel gewidmet.

Operator

Operatoren sind Token, die Operationen darstellen. Zum Beispiel: der Operator `+` stellt die Addition dar; die Operatoren `&&`, `||`, `!` stellen die Boolesche Operationen dar.

Ausdruck

Ein Ausdruck ist einen JavaScript-Code, der Literal, Variable, und Operatoren kombiniert, sodass JavaScript ihn evaluiert und Werten erstellt. Die Operanden des Operators können auch wiederum Ausdrücken, eventuell in Klammer, sein. Zum Beispiel: `a + b` ist ein Ausdruck, JavaScript evaluiert es als Summe von Variablen `a` und `b`; Der Ausdruck `2 * (a + b)` ist ein Produkt von 2 und eine Summe von `a` und `b`. Der Zusammenhang von Operator und Operand in einem Ausdruck wird im Diagramm 1.7 illustriert.

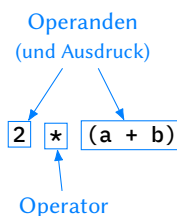


Abbildung 1.7.: Aufbau eines Ausdruck

Operator Priorität Ein Ausdruck ist eine Kette von Operationen. JavaScript definiert die Reihenfolge der Ausführung der Operationen in einem Ausdruck. Zum Beispiel:

- In einem arithmetischen Ausdruck werden Multiplikation und Division vor Addition und Subtraktion ausgeführt.
- Wenn die arithmetischen Operationen gleich Priorität haben, werden sie von Link nach Recht ausgeführt.
- Klammer (und) werden verwendet, um die Priorität der Operationen anzupassen.

Weitere Regel zu Operator Priorität finden Sie in [5, S. 64–65]. Die alte Auflage (2007) beschreibt noch alte JavaScript Version, die noch kein Operator `**` implementiert.

Link- und Recht-Assoziativität Es ist interessant bei Operationen, ob deren Operatoren Link-assoziativ oder Recht-assoziativ sind, besonders wenn die Operationen nicht kommutativ sind. Ein Operator ist Recht-assoziativ, wenn dessen rechten Operand vor dem linken Operand evaluiert wird. Ein Operator ist Link-assoziativ, wenn dessen linken Operand vor dem rechten Operand evaluiert wird.

Wir nehmen den Operator `**` als Beispiel. Der Ausdruck `a ** b` berechnet die b -Potenz von a (a^b):

```
const a = 2, b = 3;
const p = a**b;
terminal.print(p);
// → 8
```

Der Operator `**` ist Recht-assoziativ. Der Ausdruck `a**b**c` ist gleich wie `a**(b**c)`:

```
const a = 2, b = 3, c = 2;
const p = a**b**c; // i.e. a**(b**c)
terminal.print(p);
// → 512
```

Ein weiteres Beispiel ist der Operator `/`. Er ist Link-assoziativ:

```
const d = 60/12/4; // (60/12) / 4
terminal.print(d)
// → 1.25
```

Weitere Regel zu der Operator-Priorität finden Sie auch in [5, S. 64–65].

Zuweisung (Assignment)

Ein Zuweisung-Befehl ist ein JavaScript Code, der den Wert, das aus der rechten Seite des Operators `=` erstellt wird, mit der Variable von der linken Seite des Operators `=` verbindet. Die Zuweisung kann nicht auf eine Konstante angewendet werden.

In JavaScript kann man gleichzeitig die Zuweisung an mehreren Variablen mit Hilfe von *Destructuring Assignment* Konstruktion durchführen:

Destructu-
ring
Assignment

```

let a = 12,
    b = 13;
let x = 0,
    y = 0;
// multiple assignment
[x,y] = [a,b]
terminal.print(x, ' ', y)
// -> 12 13

```

Eine gängige Anwendung von Destructuring Assignment ist den *Swap* der Werten von zwei Variablen. [Swap](#)

```

// in above context
[a, b] = [b, a]; // swap a and b
terminal.print(a, ' ', b)
// -> 13 12

```

Die formale Beschreibung von Destructuring Assignment kann in [2, Abschnitt 12.15.5] entgegengenommen werden.

Fingerübung 1.8: Variablen und Konstanten

- Deklarieren Sie eine Variable `language` mit dem initialisierten Wert „JavaScript“ und eine Konstante `VERSION` mit dem initialisierten Wert „ES6“.
- Warum soll man bei der Deklaration von Variable immer mit einem Wert die Variable auch initialisieren?
- Kann man eine Konstante deklarieren, ohne sie zu initialisieren?
- Geben Sie die Werten der Variable `language` und der Konstante auf dem Browser aus.

Fingerübung 1.9: Fläche eines Kreis

Gegeben ist eine Variable `radius` wie folgt:

```

const radius = 5.4;
const area = /*... Your Code here */

```

- Formulieren Sie einen Ausdruck, der den Fläche eines Kreises mit dem Radius `radius` berechnet.
- Weisen Sie den Wert des Ausdrucks zu der Variable `area` zu.
- Geben Sie den Wert der Variable `area` auf dem Browser aus.
- Überprüfen Sie die Priorität der arithmetischen Operationen: Multiplikation und Division haben höherer Priorität als Addition und Multiplikation.

Die Formel der Fläche eines Kreises lautet:

$$A = \pi \cdot r^2$$

1.4.2. String

String in JavaScript sind Kette von Zeichen. JavaScript-Spezifikation schreibt nicht vor, in welcher Zeichencodierung die Zeichen gespeichert werden. In der Regel verwendet man Unicode als Zeichencodierung in einer JavaScript-Datei. Die Zeichencodierung kann man in jedem Code-Editor einstellen. JavaScript bietet drei Typen von Trennzeichen *Delimiter* um String aufzuzeichnen. Delimiter

Single Quote und Double Quote

Ein String Literal in JavaScript wird in einem Single-Quote-Paar (') oder in einem Double-Quote-Paar (") umrahmt. Die String

```
'Hello World'
```

und

```
"Hello World"
```

sind identisch. Der Inhalt eines Single-Quote-String kann Double-Quote Zeichen enthalten, der Inhalt eines Double-Quote-String kann wiederum Single-Quote erhalten.

Ein Single- oder Double-Quote String Literal darf keinen Newline-Zeichen enthalten.

Fingerübung 1.10: String Quote Zeichen

Überlegen Sie sich ein paar String Literale mit Single Quote bzw. Double Quote als Inhalt.

Back Quote

In JavaScript kann man ein String mit Back-Quote Paar (`) umrahmen. Es ist streng genommen kein Literal mehr. Man nennt diese Schreibweise String-Interpolation. In einem Back-Quote-String kann man JavaScript-Ausdruck in

```
`Text ${expression} Text`
```

integrieren. Der Wert des Ausdrucks wird zur Laufzeit evaluiert und ersetzt *expression* in String. Das Programm 1.1 zeigt die Anwendung der Back-Quote String. Wir betrachten den String im Programm genauer.

```
const name = argv[0];
`Hello ${name}!`; // use of ${expr}
```

Die Variable *name* wird zur Laufzeit evaluiert. Gibt der Benutzer einen beliebigen Name ein, zum Beispiel *Anton*, wird der String *Hello Anton* zusammen gesetzt. Der Teil *expression* kann ein beliebiger gültiger Ausdruck sein. Der Abschnitt 1.4.3 (Seite 19) erklärt die Template genauer.

Der Inhalt von Back-Quote-String in JavaScript kann mehrzeilig sein. Will man langwierigen Text, z.B. HTML-Code, oder Lizenztext, in JavaScript Code integrieren, kann man den Text in Back-Quote-String erfassen:

```
const html = `
<span class="name">Name: ${name}</span><br>
<span class="matrikel">Matrikel-Nummer: ${matrikel}</span>
`;
```

String Verkettung

Der Operator `+` verkettet die Teil-String in einem zusammengesetzten String. Diese Operation heißt *Concatenation*. Der Operator `+` konvertiert die Operanden wenn mindestens ein Operand ein String ist. Zum Beispiel:

Concate-
nation

```
const name = "Peter",
      surname = "Smith",
      age = 19;
const fullname = name + " " + surname + " (" + age + ")";
terminal.printl(fullname);
// → Peter Smith (19)
```

Man hat hier die weitere Bedeutung des Operator `+` kennengelernt. Wie im Programm 1.2 gezeigt wurde, führt der Operator eine Addition aus, wenn die Operanden Zahlen sind.

Eine bessere Alternative zu den obigen Code ist, den Back-Quote-String zu verwenden. Der folgenden Code hat den gleichen Effekt wie den oben Code:

```
const fullname = `${name} ${surname} (${age})`;
```

Unter bestimmter Bedingung konkateniert der Operator `+` auch andere Datentypen als String in einem String. Weitere Diskussion über das Verhalten von Operator `+` findet man im Abschnitt 1.5.2 und im Kapitel ?? Im nächsten Abschnitt wird gezeigt, wie man Nummer in String-Repräsentation auf unterschiedlichen Basis konvertiert.

Länge eines String

Die Eigenschaft `string.length` gibt die Länge eines String zurück. Man schreibt den Name der Eigenschaft eines Objekt hinter das Objekt, getrennt von einem Punkt:

```
const rule = "veni, vidi, vici";
const ruleLength = rule.length; // ruleLength holds the value 16
```

Fingerübung 1.11: Unicode String

Schreiben Sie ein Programm, das einen Name als `argv[0]` entgegen nimmt und die Länge der Name ausgibt. Testen Sie Ihr Programn mit unterschiedlichen Namen. Funktioniert Ihr Programm auch richtig, wenn der Name Nicht-ASCII Buchstaben enthält?

1.4.3. Number

JavaScript unterscheidet nicht explizit zwischen ganzen Zahlen (*Integer*) und Gleitkommazahlen. In JavaScript werden alle Zahlen in 64-Bit Gleitkommazahl nach IEEE 754 dargestellt (S. [2, Abschnitt 4.3.20]). Also die Zahlen zwischen $\pm 1.7976931348623157 \cdot 10^{308}$ und $\pm 5 \cdot 10^{-324}$

können dargestellt werden. Die Datentyp `number` ist die Menge der alle positiven Zahlenwerten, negativen Zahlenwerten, den speziellen Wert „Not-a-Number“ (Literal: `NaN`), positiven Unendlich (`+Infinity`) und negativen Unendlich (`-Infinity`) (S. [2, Abschnitt 4.3.21]). Die speziellen Werten werden im Abschnitt 1.4.5 detaillierter diskutiert. Die Tabelle 1.2 fasst die Operatoren der Datentyp `number` zusammen.

Wert		Ganze Zahlen oder Gleitkommazahlen			
Literalen		12	-34	0	4294967296 34.5
		Hexadezimal: 0xAB Oktal: 0o07 Binär: 0B11			
Operation Operator	Vorzeichen	Addition	Subtraktion	Multiplikation	
	- +	+	-	*	
	Division	Modulo	Exponent		
	/	%	**		
	Bit-Shift	Bitwise and bzw. or	Bitwise-complement	Bitwise exclusive or	
	<< >> >>>	&	~	^	

Tabelle 1.2.: Operatoren auf Datentyp `number`

Operationen auf dem Datentyp `number` ändern die Operanden nicht, sondern erstellen neuen Werten als Ergebnis der Operationen. Als Beispiel betrachten wir die folgenden Code:

```
const x = 12,
      y = 3;
const c = x << y;
terminal.printl(x) // → 12
terminal.printl(y) // → 3
terminal.printl(c) // → 96
```

Nachdem der Code ausgeführt wird, haben die Variablen `x` und `y` die ursprüngliche Werten. Das Ergebnis der Operation Left Shift (`<<`) wird in der Variable `c` gespeichert. Man nennt es „die Operation gibt einen Wert zurück“.

Numerische Literal

Direkt auftretenden Zahlen in JavaScript Source Code werden als numerische Literal genannt. JavaScript unterstützt unterschiedlich Formaten von numerischen Literal. Aus technische Sicht besteht der Ausdruck `-12` aus zwei Tokens: der unäre Operator Minus (`-`) und der numerische Literal (`12`). Die formale Beschreibung der lexikalischen Regel von numerischen Literal kann man aus [2, Abschnitt 11.8.3] entgegennehmen.

Dezimalen ganzzahligen Literal In JavaScript wird ein dezimales ganzzahliges Literal (*decimal Integer*) als Folge von dezimalen Ziffer geschrieben, zum Beispiel:

```
0
3
12345
```

decimal Integer

Hexadezimal-Integer Literal, Oktal-Integer Literal, Binär-Integer Literal Neben Integer Literal zur Basis 10 erkennt JavaScript auch numerische Literal zur Basis 16 (Hexadezimal-Integer Literal), zur Basis 8 (oktal-Integer Literal) und zur Basis 2 (Binär-Integer Literal). Ein Hexadezimal-Literal

fängt mit `0X` oder `0x` an, gefolgt von Ziffer zwischen 0 und 9 oder Buchstaben zwischen „A“ bzw. „a“ (entspricht den Wert 10) und „F“ bzw. „f“ (entspricht den Wert 15). Hier sind einige Beispielen:

```
0xFF // 15*16 + 15 = 255 (base 10)
0xF3 // 15*16 + 3 = 243
```

Ein Oktal-Integer fängt mit `00` bzw. `0o` an, gefolgt von Ziffer zwischen 0 und 7. Ein Binär-Integer fängt mit `0B` bzw. `0b` an, gefolgt von Ziffer 0 oder 1. Wir betrachten einige Beispielen:

```
0o12 // Octal-Integer 1*8 + 2 = 10
0b111 // Binary-Integer 1*(2**2) + 1*2 + 1 = 7
```

Gleitkommazahl-Literal Eine Gleitkommazahl (*Floating-Point Number*) wird als ein ganzzahliger Zahlteil dargestellt, gefolgt von einem Dezimalpunkt (.) und dem Bruchteil der Zahl. Zum Beispiel: `0.3`, `3.14159`.

Floating-
Point
Number

Eine Gleitkommazahl kann auch in der Exponentialschreibweise dargestellt werden. In diesem Fall folgt das Exponent zur Basis Zehn-Faktor nach dem Bruchteil bzw. dem ganzzahligen Zahlteil. Das Exponent-Faktor besteht aus einem Zeichen `E` oder `e` gefolgt von einer ganzen Zahl. Zum Beispiel der SI-Faktor *mega* kann man in JavaScript als `1E6` codieren. Die Gravitation-Konstant wird in JavaScript als `6.67408E-11` oder `6.67408e-11`.

In der vereinfachten Form sieht sich die Syntax der Gleitkommazahl aus:

```
[integer][.fraction][(E | e)[(+ | -)]10-based-exponent]
```

Hier wird die abstrakte Beschreibung von Syntax eines Idioms, eine Sprache-Konzept oder Syntax eines Operators beschrieben:

- Ein Text im Format wie *integer* wird als Platzhalter interpretiert. Solche Text muss man in Quellcode durch entsprechenden Code ersetzen. Im dem obigen Beispiel ersetzt man *integer part* durch die ganze Zahl (6) in der Gravitation-Konstant.
- Beschreibungen in `[...]` sind Optionen. Es bedeutet, dieser Teil kann weggelassen werden. In der Zahl `1E3` wird der Teil `[.fraction part]` weggelassen.
- Ein Text in `(opt1 | opt2 | ... optn)` heißt „wählen eine und nur eine von `opt1` bis zur `optn` aus“. Die Annotation `(E | e)` bedeutet: man kann nur entweder für `E` oder für `e` entscheiden.
- Texten wie `E` oder `+` sind literal JavaScript Code, die man in Quellcode schreiben kann.
- Um die Zweideutigkeit zu vermeiden werden die Zeichen `( ) [ ] |` in der Beschreibung von Syntax farbig gedruckt, während sie in JavaScript Code in Schwarz gedruckt werden.

Die Abbildung 1.8 zeigt den Aufbau der Gravitationskonstante ($G = 6,67408 \cdot 10^{-11} \frac{\text{m}^3}{\text{kg}^2 \text{s}^2}$).

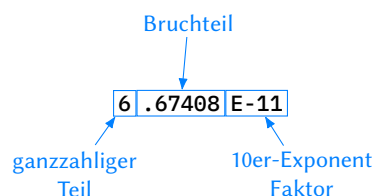


Abbildung 1.8.: Aufbau einer Gleitkommazahl in JavaScript

Fingerübung 1.12: Gültige Gleitkommazahlen

Geben Sie Beispielen für folgende literalen Gleitkommazahlen:

1. Eine Gleitkommazahl ohne den *integer*, die den Wert $\frac{1}{3}$ annäht.
2. Eine Gleitkommazahl ohne den *fraction*, die den Wert $\frac{1}{10}$ beträgt.

Fingerübung 1.13: Lexikalische Beschreibung von Integer

Beschreiben Sie die Syntax einer ganzen Zahl in JavaScript mit der Schreibweise wie die Beschreibung der Syntax von Gleitkommazahl.

Arithmetischen Operatoren

Die arithmetischen Operationen in JavaScript folgen den arithmetischen Regel in Mathematik. Sie sind relative intuitiv zu benutzen. Das Programm 1.3 veranschaulicht die arithmetischen Operationen. Das Programm zeigt auch die Nutzung von der eingebauten Funktion `Number` für die Konvertierung eines Strings in Zahl. Auf einer Seite wird das Ergebnis bei der Division von ganzen Zahlen in JavaScript in Gleitkommazahl konvertiert, wenn der Zähler kein Mehrfach von Nenner ist. Auf der anderen Seite werden die Operanden der Rest-Division zuerst in 32-Bit ganzen Zahlen konvertiert, bevor die Operation aus geführt wird.

Programm 1.3.: Arithmetische Operationen (arithmetic-op)

```
import {terminal} from "./dfhi.js";

window.main = (...argv) => {
  const x = Number(argv[0]);
  const y = Number(argv[1]);
  const s = x + y,
        d = x - y,
        p = x * y,
        q = x / y,
        r = x % y;
  terminal.printl(`Summe: ${x} + ${y} = ${s}`);
  terminal.printl(`Differenz: ${x} - ${y} = ${d}`);
  terminal.printl(`Produkt:  ${x} * ${y} = ${p}`);
  terminal.printl(`Quote:   ${x} / ${y} = ${q}`);
  terminal.printl(`Rest:    ${x} % ${y} = ${r}`);
};
```

Die arithmetischen Operatoren sind bereits eingebaut in JavaScript. Die Funktion `Number()` konvertiert einen String in eine Zahl. Repräsentiert der String keine Zahl, so wird der besondere Wert `NaN` zurückgegeben.

Operatoren, die ein Operand akzeptieren, werden als *Unär*-Operatoren genannt. Operatoren, die zwei Operanden akzeptieren, werden als *Binär*-Operator genannt. Die Vorzeichen Operatoren

Unär-
Operator
Binär-
Operator

sind Unär-Operatoren, die anderen arithmetischen Operatoren sind binär-Operatoren.

Operatoren, die vor deren Operanden stehen, nennt man *Prefix*-Operator. Operatoren, die zwischen deren Operanden stehen, nennt man *Infix*-Operator. Die Vorzeichen-Operatoren sind Prefix-Operatoren, die anderen arithmetischen Operatoren sind Infix-Operatoren. Prefix
Infix

Die Vorzeichen-Operatoren können beliebig oft mit einander so kombinieren, dass der entstehende Ausdruck unleserlich ist:

```
const a = -(1+2)*--3;
terminal.print(a);
```

Solche Ausdrücken soll man vermeiden.

Bei der Arbeit mit Zahlen muss man auf die Genauigkeit der Berechnung achten. JavaScript kann zum Beispiel den Bruch $\frac{1}{3}$ nicht genau darstellen. Das Ergebnis von `1 / 3` ist etwa `0.3333333333` –je nach der Implementierung, des JavaScript-Interpreter– also einen approximierten Wert zum Wert $\frac{1}{3}$.

Das Objekt `Number`

Wir haben die Funktion `Number(argument)` im Program 1.3 bereits verwendet, um einen String in eine Zahl zu konvertieren. Das Objekt `Number` bietet weitere nützliche Funktionen:

`Number.parseInt(obj[, radix=10])` Diese Funktion konvertiert das Argument `obj` in eine ganze Zahl. Wird das Argument `radix` angegeben, wird das Argument im Zahlssystem mit dem `radix`-Basis konvertiert. Ist das Argument `radix` abwesend, gilt die Konvertierung als 10er-Basis. Diese Funktion ist ziemlich liberal. Sie bewertet die Zeichen des Argumentes bis sie ein Zeichen trifft, das kein Ziffer im Zahlssystem `radix` ist. Wir betrachten das folgende Beispiel: Der Ausdruck

```
Number.parseInt("funny")
```

gibt `NaN` zurück, denn `f` repräsentiert kein gültige Ziffer im Zehner-Zahlssystem. Der Aufruf

```
Number.parseInt("funny", 16)
```

gibt jedoch die ganze Zahl `15` zurück, denn `f` ist ein gültige Ziffer in Hexadezimal-System. Das Zeichen `u` ist kein gültige Ziffer, so werden das Zeichen `u` und die weitere Zeichen ignoriert.

`Number.parseFloat(obj)` Diese Funktion konvertiert das Argument `obj` in eine Zahl. Sie verhält sich bezüglich der Verwertung von Zeichen ähnlich der Funktion `Number.parseInt`. Die Zeichen müssen allerdings in Zehner-Zahlssystem sein. Das Zeichen `.` wird als Dezimalpunkt verwendet.

Der Aufruf

```
Number.parseFloat("12.5cm")
```

gibt `12.5` zurück.

`Number.isInteger(number)` und `Number.isSafeInteger(number)` Diese Funktionen überprüfen, ob die Zahl `number` eine ganze Zahl bzw. eine ganze Zahl, die JavaScript fehlerfrei darstellen kann. Beide Funktionen gibt einen Wert der Typ `boolean` (Abschnitt 1.4.4) zurück.

Weitere Funktionen des Objekts `Number` kann man aus [11] oder [2, Abschnitt 20.0] nachlesen. Funktionen zur Verarbeitung von sondern Werten werden im Abschnitt 1.4.5 diskutiert.

Das Objekt Math

Das Objekt **Math** definiert gängigen mathematische Konstanten wie die Kreiszahl π (**Math.PI**), die eulersche Zahl e (**Math.E**) und gängigen Funktionen wie logarithmische, trigonometrische, exponentielle Funktionen. Sie werden in [2, Abschnitt 20.2.1] spezifiziert. Ein Leser-freundliches Dokument ist die Sammlung von MDN Web Docs [10].

In Folge sind einige Funktionen vom Objekt **Math**.

Math.sqrt Das Ergebnis von einem Berechnung kann auch das besondere Wert **Infinity** (wenn das Ergebnis zu groß ist) oder **NaN** (wenn das Ergebnis undefiniert ist) sein. Die Funktion **Math.sqrt(n)** akzeptiert ein Argument **n** der Typ **number**. Ist das Argument negativ, gibt die Funktion **NaN** zurück.

Das Programm 1.4 berechnet die Lösungen einer quadratischen Gleichung der Form

$$ax^2 + bx + c = 0$$

für $a, b, c \in \mathbb{R}, a \neq 0$. Die Lösung der Gleichung kann auch **NaN** sein. Es bedeutet, das die Gleichung keine Lösung hat.

Programm 1.4.: Quadratische Gleichung (quadratic-equation)

```
import {terminal} from "./dfhi.js";

window.main = (...argv) => {
  const a = Number(argv[0]),
        b = Number(argv[1]),
        c = Number(argv[2]);
  const delta = b*b - 4*a*c;
  const d = Math.sqrt(delta);
  const x1 = (-b + d) / (2*a),
        x2 = (-b - d) / (2*a);

  terminal.printl(`Equation: ${a}x^2 + (${b})x + (${c}) = 0`);
  terminal.printl(`first solution: ${x1}`);
  terminal.printl(`second solution: ${x2}`);
};
```

Dieses Programm nimmt drei Argumenten als `argv[0]` (a), `argv[1]` (b) und `argv[2]` (c) an, und gibt die Lösungen der Gleichung $ax^2 + bx + cx = 0$ für $a, b, c \in \mathbb{R}, a \neq 0$ aus. Das Programm verwendet die Formel

$$\Delta = b^2 - 4ac$$

$$x_{1,2} = \frac{-b \pm \sqrt{\Delta}}{2a}$$

Der Benutzer gibt den Wert der Parameter a, b und c getrennt durch Leerzeichen in der Eingabefeld ein. Zum Beispiel zum Berechnen der Lösung der Gleichung $x^2 - x - 1$ gibt der Benutzer `1 -1 -1` in der Eingabefeld ein. Die Gleichung $x^2 + x + 1 = 0$ hat keine Lösung, die Ergebnisse sind deshalb `NaN`.

Fingerübung 1.14: Euklidischer Abstand zwischen zwei Punkten

Schreiben Sie ein Programm, das die Koordinate eines Punktes $P(x, y)$ als `argv[0]` (x -Komponente) und `argv[1]` (y -Komponente) annimmt und den euklidischen Abstand vom Punkt P zum Punkt $(0, 0)$ ausgibt. Benutzen Sie die Formel

$$d = \sqrt{x^2 + y^2}$$

zum Berechnen des euklidischen Abstand.

Math.random und Math.floor Funktionen können beliebig in einander kombiniert werden. Die Funktion `Math.random()` erstellt eine Zufallszahl im Intervall $[0, 1)$. Das heißt die Zufallszahl kann 0 sein, dennoch ist immer kleiner als 1.

Die Funktion `Math.floor(expression)` schneidet den Bruchteil vom *expression* ab, wenn *expression* nicht negativ ist.

Das Programm 1.5 simuliert den Wurf eines Würfels (Abb. 1.10 am Ende des Kapitel, Seite 33). Das Programm verwendet `Math.random` als Argument der Funktion `Math.floor` um von einer zufälligen Gleitkommazahl eine ganze Zahl zu generieren.

Programm 1.5.: Würfel-Simulation (dice-simulation)

```
import {terminal} from "./dfhi.js";

window.main = (...argv) => {
  const max = 6, min = 1;
  const face = Math.floor(Math.random() * (max + 1 - min) + min);
  terminal.printl(face);
};
```

Das Programm simuliert einen (fairen) Würfel. Jedes Mal wenn der Benutzer auf „Rerun“ anklickt, wird eine ganzen Zahl von eins bis sechs auf dem Browser angezeigt. Den Ausdruck zur Generierung von Zufallszahl kann man in diesem Fall auch abkürzen.

Die Abbildung 1.9 visualisiert das mathematische Konzept hinter dem Ausdruck

`Math.floor(Math.random() * (max + 1 - min) + min)`

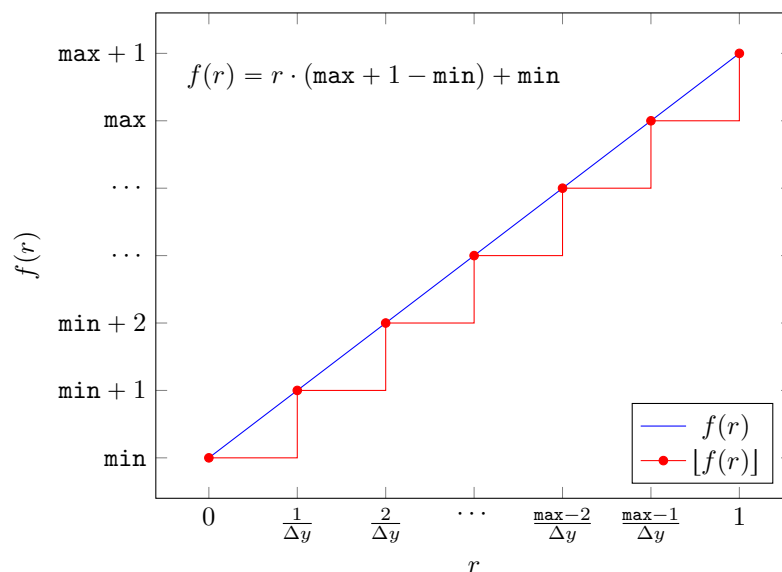


Abbildung 1.9.: Zuordnung einer Zahl $r \in [0, 1]$ zu einer ganzen Zahl in $[\min, \max]$

Der obige Ausdruck ordnet eine zufällige Gleitkommazahl r im Intervall $[0, 1]$ zu einer ganzen Zahl $z \in \mathbb{Z}$, $\min \leq z \leq \max$ zu. Dabei wird eine lineare Funktion $f(r) : [0, 1] \rightarrow [\min, \max + 1]$ konstruiert.

$$\begin{aligned}
 f(r) &= r \frac{\Delta f(r)}{\Delta r} + f(0) \\
 &= r \frac{(\max + 1) - \min}{1 - 0} + \min \\
 &= r \cdot (\max + 1 - \min) + \min
 \end{aligned}$$

Die Zahl z ist die Abrundung von $f(r)$ (mathematische Schreibweise: $z = \lfloor f(r) \rfloor$) für $r \in [0, 1)$ ist eine ganze Zahl in der Intervall $[\min, \max]$.

$$\begin{array}{ll}
 r \in [0, 1/\Delta y) \Rightarrow f(r) \in [\min, \min + 1) & \Rightarrow z = \lfloor f(r) \rfloor = \min \\
 r \in [1/\Delta y, 2/\Delta y) \Rightarrow f(r) \in [\min + 1, \min + 2) & \Rightarrow z = \lfloor f(r) \rfloor = \min + 1 \\
 r \in [2/\Delta y, 3/\Delta y) \Rightarrow f(r) \in [\min + 2, \min + 3) & \Rightarrow z = \lfloor f(r) \rfloor = \min + 2 \\
 \dots & \dots \\
 r \in [\max - 2/\Delta y, \max - 1/\Delta y) \Rightarrow f(r) \in [\max - 1, \max) & \Rightarrow z = \lfloor f(r) \rfloor = \max - 1 \\
 r \in [\max - 1/\Delta y, 1) \Rightarrow f(r) \in [\max, \max + 1) & \Rightarrow z = \lfloor f(r) \rfloor = \max
 \end{array}$$

Fingerübung 1.15: Math.floor

1. Wie verhält sich die Funktion `Math.floor(expression)` mit einem negativem Argument *expression*?
2. Überprüfen Sie, ob der Ausdruck `Math.floor(Math.random() * (max + 1 - min) + min)` auch zufällige Zahlen im Intervall $[\min, \max]$ generiert, wenn $\min < 0$ und $\max > 0$ sind!
3. Kann man die Funktion `Math.floor` durch die Funktion `Math.round` ersetzen? Eine Online-Dokumentation der Funktion `Math.round` finden Sie in [10].

1.4.4. Boolean

Der Datentyp `boolean` repräsentiert die Wahrheit-Werten. Der Datentyp hat nur zwei Werten: `true` und `false`.

Wert	Wahr oder Falsch		
Literalen	true	false	
Operation	Und	Oder	Nicht
Operator	&&		!

Tabelle 1.3.: Operatoren auf Datentyp `boolean`

Die Operationen „Und“, „Oder“ und „Nicht“ werden wie folgt definiert:

- `a && b` ist `true` wenn sowohl `a` `true` als auch `b` `true` sind.
- `a || b` ist `true` wenn mindestens ein von `a` und `b` `true` ist.

- `!` `a` ist `true` wenn `a` `false` ist.

Die Operatoren `&&` und `||` sind infix binäre Operatoren. Der Operator `!` ist ein prefix unärer Operator. Da es exakt zwei Werten –`true` und `false`– gibt, die eine boolesche Variable belegen kann, kann man all mögliche Ergebnisse von booleschen Operatoren abzählen, wie in der Tabelle 1.4 zeigt. Solche Tabelle wird als Wahrheitstabelle genannt.

a	b	a && b	a b	a	! a
true	true	true	true	true	false
true	false	false	true	false	true
false	true	false	true		
false	false	false	false		

Tabelle 1.4.: Wahrheitstabelle von den Operatoren `&&`, `||` und `!`

Wir können die Operatoren mit Klammer kombinieren, um beliebige komplexe Ausdrücke zu formulieren. Manche Ausdrücke sind äquivalent. Zum Beispiel laut De Morgansche Gesetze sind die Ausdrücke `a && b` und `!(!a || !b)` äquivalent.

Weitere Operationen auf boolesche Werten sind *Implikation* und *Exklusiv-Oder*, welche wie folgt definiert werden:

Implikation
Exklusiv-Oder

- Implikation $a \Rightarrow b$ ist äquivalent zu `!a || b`
- Exklusiv Oder $a \oplus b$ ist äquivalent zu `(a && !b) || (!a && b)`

JavaScript hat kein entsprechenden Operatoren für Implikation und Exklusiv-Oder.

Fingerübung 1.16: Wahrheitstabelle von $\Rightarrow$ und $\oplus$

Stellen Sie die Wahrheitstabelle für die Operationen Implikation ($\Rightarrow$) und Exklusiv-Oder ($\oplus$) auf.

Die booleschen Ausdrücke werden verwendet um das Verhalten eines Programm zu formulieren. In der Regel wird eine Bedingung in Form von boolesche Ausdrücke formuliert. Weiterhin wird die Ausführung eines Code-Abschnitt in Abhängigkeit von der Bedingung formuliert. So kann man kontrollieren, ob und –viel wichtiger– wie oft ein Code-Abschnitt in einem Programm ausgeführt wird. Wir diskutieren näher über dieses Thema im Kapitel 2.

Vergleich von numerische Werte

Die Vergleich-Operatoren sind Mittel in der JavaScript um Bedingung zu formulieren. Diese Operatoren vergleichen zwei Werten, in der Regel sind sie numerische Werten, und gibt einen Wert der Typ `boolean` zurück. Die Tabelle 1.5 zeigt die verfügbare Vergleich-Operatoren in JavaScript.

Typische Formulierungen mit Vergleich-Operatoren sind zum Beispiel:

- Ist die Diskriminante Δ nicht negative? `(b*b - 4*a*c) >= 0`
- Ist die Zahl gerade? `(z % 2) === 0`
- Ist die Würfel-Zahl gültig (liegt in der Intervall $[1, 6]$)? `(face > 0) && (face <= 6)`

Operator	numerische Bedeutung	true	false
<code>==</code>	Gleich	<code>2 == 1+1</code>	<code>2 == 1+3</code>
<code>!=</code>	Nicht gleich	<code>2 != 3*2</code>	<code>2 != 1+1</code>
<code>===</code>	Identisch	<code>2 === 2</code>	<code>2 === 3</code>
<code>!==</code>	Nicht identisch	<code>2 !== 3</code>	<code>2 !== 2</code>
<code><</code>	Kleiner als	<code>a+2 < a+3</code>	<code>a-2 < a-2</code>
<code><=</code>	Kleiner als oder gleich	<code>3 <= 3</code>	<code>5 <= 4</code>
<code>></code>	Größer als	<code>4 > 3</code>	<code>4 > 4</code>
<code>>=</code>	Größer als oder gleich	<code>4 >= 4</code>	<code>4 >= 5</code>

Tabelle 1.5.: Vergleich-Operatoren mit numerische Beispielen

Die Vergleich-Operatoren haben niedrigere Priorität als die arithmetischen Operatoren, und höhere Priorität als die booleschen Operatoren. Die obigen Ausdrücke können deshalb ohne Klammern geschrieben werden, etwa `b**b - 4*a*c >= 0` oder `face > 0 && face <= 6`. Dennoch ist die Schreibweise mit Klammer eine bessere Programmierstil. Die Priorität der Operatoren kann man aus [5, S. 65] nachlesen.

Der Operator `==` und `===` bzw. deren negative Korrespondent `!=` und `!==` haben gleiche Verhalten wenn deren beide Operanden vom Typ `number` sind. Dennoch haben sie unterschiedliche Verhalten wenn die Operanden nicht vom Typ `number` sind. Man benutzt immer bevorzugt den Operator `===` bzw. den Operator `!==` gegenüber `==` bzw. `!=`. Die Grund wird im Abschnitt 1.5.4 und tiefer im Kapitel ?? diskutiert.

Fingerübung 1.17: Boolesche Ausdrücke

- Die Variablen `a` und `b` haben den Typ `number`. Vereinfachen Sie den folgenden Ausdruck: `(!(a < b) && !(a > b))`.
- Gegeben sind die Variable `a`, `b` und `c` der Type `number`. Test der Ausdruck `a < b < c`, ob `b` in der Intervall `(a, c)`?
- Was ist der Unterschied zwischen `a & b` und `a && b`?

1.4.5. Sondern Werten

Es gibt einige besondere Werten in JavaScript, die besondere Aufmerksamkeit bei dem Umgehen mit ihnen fordern. In diesem Abschnitt diskutieren wir über solche Werten.

Der Wert `undefined` und der Wert `null`

Um die Abwesenheit eines Wertes darzustellen, stellt JavaScript zwei besondere Werten zur Verfügung: `undefined` und `null`. Der Wert `undefined` ist der initialisierte Wert einer Variable, wenn sie deklariert, jedoch nicht explizit initialisiert wird. Zum Beispiel Der Befehl `let x;` deklariert die Variable `x` aber nicht initialisiert sie, deshalb die Wert von `x` ist `undefined`. Der Ausdruck

```
typeof identifier name === 'undefined'
```

überprüft ob ein Identifier deklariert und initialisiert ist. Sowohl nicht-deklarierte Identifier als auch deklariert aber nicht initialisierte Variable verursachen, dass der Ausdruck als `true` evaluiert

wird.

Der Ausdruck

```
identifizier name === undefined
```

erzeugt einen **ReferenceError** wenn der Identifier nicht deklariert ist; und als **true** evaluiert wird wenn der Identifier deklariert aber nicht initialisiert ist.

Während der Wert **undefined** die Abwesenheit eines primitiven Wert repräsentiert, repräsentiert der Wert **null** die Abwesenheit eines Objektes. JavaScript initialisiert niemals impliziert eine Variable mit **null**. In den meisten Situationen soll man **null**, besonderes als Rückgabewert einer Funktion, vermeiden.

Mathematische spezielle Werten

NaN Es gibt mathematische Operationen, die nicht definiert für bestimmte Zahlen sind, zum Beispiel gerade Wurzel aus einer negativen Zahl ($\sqrt{-2}$, **Math.sqrt(-2)**) oder Logarithmus einer negativen Zahl ($\ln -1$, **Math.log(-1)**), sind nicht definiert. Der Wert **NaN** („not a Number“) repräsentiert solche Werten. Er ist deshalb von Typ **number**.¹

Um ein Zahl zu überprüfen, ob sie eine mathematisch definierte Zahl ist, verwendet man die Funktion **Number.isNaN(number)** oder je nach Situation auch deren Negation.

```
const x = -1;
const l = Math.log(x);
Number.isNaN(l) // → true because l is not a number
Number.isNaN(x) // → false
```

Wenn ein Operand der arithmetischen Operatoren ein **NaN** ist, werden die Ausdrücke als **NaN** ausgewertet.

```
// in the context above
const s = l + 2;
Number.isNaN(s) // → true
```

Der Vergleich von **NaN** mit andere beliebige Werten ergibt sich als **false**. Zum Beispiel:

```
NaN === 0 // → false
NaN === NaN // → false
```

Infinity Wenn das Ergebnis einer Operation außerhalb der darstellbare Bereiche von Zahlen in JavaScript ist, wird der Wert **Infinity** oder **-Infinity** zurückgegeben. Um eine Zahl zu testen, ob sie eine **infinity** ist, kann man die Funktion **Number.isFinite(number)** verwenden.

```
const inf = 100 / 0, // inf is Infinity
      ninf = -100 / 0; // ninf is -Infinity
Number.isFinite(inf); // → false
Number.isFinite(ninf); // → false
```

Die arithmetischen Operatoren folgen den mathematischen Regel wenn sie die Werten **Infinity** oder **-Infinity** als Operanden haben.

¹Wenn man den Ausdruck in englisch läse, klänge es wie ein Paradox: *Not a Number is a number*.


```
// in the context above
inf + 12    // → Infinity
inf * 0     // → NaN
inf / 0     // → Infinity
inf + ninf  // → NaN
```

Fingerübung 1.18: Eigenschaften von NaN

Beantworten Sie folgenden Fragen:

- Welche Funktionen in [10] kann NaN zurückgeben? Finden Sie drei Funktionen. Geben Sie für jede Funktion ein Beispiel!
- Evaluieren Sie den Ausdruck `Number.isNaN(Math.log(0))`!

1.5. Typ-Konvertierung

Da JavaScript eine Variable nicht mit einem Typ fest verbindet, muss man immer den Typ einer Variable im Auge behalten. Die Operatoren in JavaScript können die Typen deren Operanden implizit konvertieren. In meisten Fälle ist es besser, die Operanden in passenden Typ explizit zu konvertieren als auf den Regel der Konvertierung der Operatoren zu überlassen. Explizit Typ-Konvertierung drückt die Absicht des Autors aus, und deshalb führt zu ein leserlicher Code.

1.5.1. Der Operator `typeof`

Um der Typ eines Ausdruck fest zu stellen kann man den Operator `typeof` benutzen. Die Operation `typeof expression` gibt den Typ vom Ausdruck `expression` als String zurück.

```
const name = "Peter";
typeof name;           // → 'string'
const age = 2019 - 2000;
typeof age;            // → 'number'
typeof undefined       // → 'undefined'
typeof null            // → 'object'
typeof NaN             // → 'number'
```

1.5.2. Der Operator Plus (+)

Das Verhalten des Operators `+` ist komplex. Die Semantik des Operator `+` ist definiert in [2, Abschnitt 12.8.3]. Vereinfacht gesagt, konvertiert der binäre Operator `+` die Operanden in String und führt eine Konkatenation aus, wenn mindestens ein Operand nicht der Typ `number` ist. Folgenden Beispielen veranschaulichen das Verhalten.

- Der Ausdruck `1 + 2` gibt `3` als `number` zurück. Der Ausdruck `'1' + '2'` gibt `'12'` als `string` zurück.
- Die Ausdrücke `1 + '2'` und `'1' + 2` geben `'12'` als `string` zurück.

Der unäre Operator `+` konvertiert seinen Operand in Zahl; erfolgt das Konvertierung, wird der Wert des Operand als Zahl zurückgegeben, sonst der spezielle Wert `NaN`. Wir betrachten einige Beispielen:

- Der Ausdruck `+'3'` und `+3` gibt die Zahl `3` zurück.
- Der Ausdruck `+'1' + '2'` gibt den String `'12'` zurück während der Ausdruck `+( '1'+ '2' )` die Zahl `12` zurück.
- Der Ausdruck `+'funny'` gibt den speziellen Wert `NaN` zurück.

Der Korrespondent von Unär-Operator `+`, der Unär-Operator `-`, verhält sich ähnlich. Erfolgt der Konvertierung wird die Negation der numerischen Wert zurückgegeben, sonst der Wert `NaN`.

Fingerübung 1.19: Bonus Übung – String Konkatenation

Evaluieren Sie den Ausdruck `"b"+ "a"+ +"a"+ "a"`.

1.5.3. Die Vergleich-Operatoren

Die Operatoren in der Tabelle 1.5 konvertieren die Operanden in numerische Werten wenn mindestens ein deren Operand der Typ `number` hat, jedoch nicht wenn beider Operanden nicht numerisch sind. Wir betrachten einigen Beispielen:

- Der Ausdruck `3 < 22` gibt offensichtlich `true` zurück.
- Der Ausdruck `'3' < '22'` gibt `false` zurück, denn beide Operanden den Typ `string` haben. Die beide Operanden werden lexikalisch verglichen. Da das Zeichen `'3'` nach das Zeichen `'2'` in der ASCII-Tabelle zugeordnet, gibt der Ausdruck `false` zurück.
- Der Ausdruck `'3' < 22` so wie `3 < '22'` gibt `true` zurück, denn mindestens ein Operand in den jeweiligen Ausdrücke den Typ `number` hat.

Wenn man zwei Werten vergleichen will, muss man die Typen jeweiligen Werten genau wissen. Es ist in der Praxis sinnvoll, die Typen von Operanden explizit zu konvertieren, wenn es nicht klar ist, zu welchem Typ eine Variable gehört. (Es ist der Fall wenn man zum Beispiel Legacy Funktionen, oder schlechte Bibliotheken verwenden muss, die nicht ausreichend dokumentiert sind.) Zum Beispiel:

```
const r = ...           // Type of r is unknown
Number(r) < 0           // both operands are Number
```

1.5.4. Identität (===) und Gleichheit (==)

Wir diskutieren über die Operatoren `==` und `===` in diesem Abschnitt im Kontext der primitive Datentypen und String. In Kapitel ?? diskutieren wir über das Thema im Kontext der zusammengesetzten Objekten.

Das folgende Abschnitt ist einen Auszug aus dem Buch *JavaScript: Das umfassende Referenzwerk*, S. 72 von Flanagan [5]. Fußnoten sind mein Kommentar. Der Autor des Buch benutzt den Begriff „Umwandlung“ statt „Konvertieren“.

Mit den folgenden Regeln kann man ermitteln, ob zwei Werte laut dem Operator `===` identisch sind:

- Wenn beiden Werten unterschiedliche Typen haben, sind sie nicht identisch.

- Wenn beide Werte Zahlen sind und denselben Wert haben, sind sie identisch, sofern nicht einer oder beide **NaN**-Werte sind. In diesem Fall sind sie nicht identisch. Der Wert **NaN** ist nie mit einem anderen Wert und auch nicht mit sich selbst identisch. Um zu prüfen, ob ein Wert **NaN** ist, verwenden Sie die Funktion **isNaN()**.²
- Wenn beide Werte Strings sind und genau dieselben Zeichen an genau denselben Positionen enthalten, sind sie identisch. Wenn die Strings sich in Länge oder Inhalt unterscheiden, sind sie nicht identisch. Beachten Sie, dass der Unicode-Standard in einigen Fällen für einen String mehrere Schreibweisen zulässt. Aus Effizienzgründen werden Strings in JavaScript jedoch streng Zeichen für Zeichen verglichen, und der String-Vergleich geht davon aus, dass alle Strings vor dem Vergleich in eine „normalisierte Form“ umgewandelt wurden. Eine weitere Möglichkeit, Strings zu vergleichen, finden Sie im Referenzteil dieses Buches.³
- Wenn beide Werte der Boolesche Wert **true** oder wenn beide der Boolesche Wert **false** sind, dann sind sie identisch.
- Wenn beide Werte auf dasselbe Objekt oder Array oder dieselbe Funktion verweisen, sind sie identisch. Wenn sie auf verschiedene Objekte (oder Arrays oder Funktionen) verweisen, sind sie selbst dann nicht identisch, wenn beide Objekte identische Eigenschaften oder beide Arrays identische Elemente haben.
- Wenn beide Werte **null** oder beide Werte **undefined** sind, sind sie identisch.

Mit den folgenden Regeln kann man ermitteln, ob zwei Werten laut dem Operator **==** übereinstimmt sind:

- Prüfen Sie die beiden Werten auf Identität, falls sie denselben Typ haben. Falls die Werten identisch sind, sind sie gleich; wenn sie nicht identisch sind, sind sie nicht gleich.
- Die beiden Werten können auch gleich sein, wenn sie nicht denselben Typ haben. Mit den folgenden Regeln und Typumwandlungen können Sie auf Gleichheit prüfen.
 - Wenn der eine Wert **null** und der andere **undefined** ist, sind sie gleich.
 - Wenn der eine Wert eine Zahl und der andere ein String ist, wandeln Sie den String in eine Zahl um, und versuchen dann, den Vergleich mit dem umgewandelten Wert vorzunehmen.
 - Wenn einer der Werte **true** ist, wandeln Sie ihn in 1 um, und versuchen Sie, den Vergleich noch einmal vorzunehmen. Wenn einer der Werte **false** ist, wandeln Sie ihn in 0 um, und versuchen Sie dann den Vergleich noch einmal vorzunehmen.
 - Wenn der eine Wert ein Objekt und der andere eine Zahl oder ein String ist, wandeln Sie das Objekt in einen Wert eines primitiven Datentyps um, und nehmen Sie den Vergleich erneut vor. Für die Umwandlung des Objekts in einen einfachen Wert verwenden Sie entweder seine Methode

²In neuen Browsern, die den Standard ECMAScript 2015 und neuer unterstützen, sollen Sie statt der globalen Funktion **isNaN()** die Funktion **Number.isNaN()** benutzen.

³„Dieses Buch“ bezieht sich auf [5]. Sie können die Referenzen z.B. auch aus dem [13] nachlesen.

`toString()` oder seine Methode `valueOf()`. Die integrierten Klassen des Sprachkerns von JavaScript versucht erst die Umwandlung mit `valueOf()` und dann erst die mit `toString()`. Die einzige Ausnahme ist die Klasse `Date`, die Umwandlung mit `toString()` vornimmt. Objekte, die nicht zum Sprachkern von JavaScript gehören, können sich selbst auf eine Weise, die von der jeweiligen Implementierung definiert wird, in einfache Werte umwandeln.

- Alle anderen Kombinationen von Werten sind nicht gleich.

Kurz gefasst, für primitive Typen gilt: Der Operator „Identisch“ `===` vergleicht zwei primitive Werten unter der Berücksichtigung deren Datentypen. Übereinstimmen die Datentypen der Operanden nicht, so gibt den Operator `===` immer `false` zurück. Der Operator „Gleich“ `==` vergleicht zwei Werte, ob sie gleich sind, ohne deren Datentyp zu berücksichtigen. Der Ausdruck `3 == '3'` gibt deshalb `true` zurück während `3 === '3'` `false` zurück gibt.

Die Negation der Operatoren `==` und `===` folgen den obigen Regeln um die Übereinstimmung der Operanden festzustellen. Der Operator `!=` gibt `false` zurück wenn beide Operanden gleich sind; sonst `true`. Der Operator `!==` gibt `false` zurück, wenn beide Operanden miteinander identisch sind; sonst `true`.

Es ist zu empfehlen, dass man immer `===` statt `==` und deren Negation Korrespondenten verwendet (Sieh auch [1]).

Fingerübung 1.20: Vergleich-Operator

Überprüfen Sie das Ergebnis folgenden Ausdrücke; füllen Sie die Lücken aus:

Ausdruck	Erwartet	Evaluiert	Ausdruck	Erwartet	Evaluiert
<code>1 == ("true" && 0)</code>	_____	_____	<code>1 == "true"</code>	_____	_____
<code>2 == ("true" + 1)</code>	_____	_____	<code>1 == ("true" + 0)</code>	_____	_____

Sind die Ergebnisse vorhersehbar?

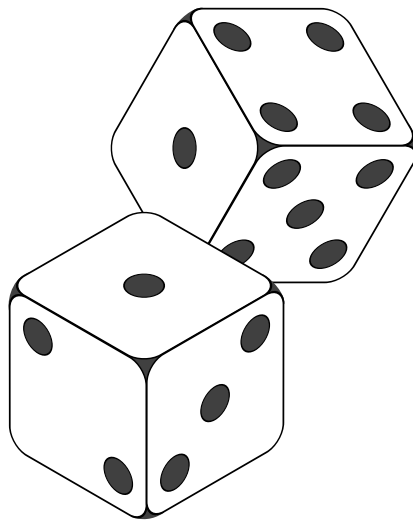


Abbildung 1.10.: Würfel

2. Ablaufsteuerung

Bisher werden alle Befehle sequentiell nacheinander ausgeführt. Es gibt dennoch Situationen, bei denen nur bestimmte Befehle ausgeführt werden, oder bestimmte Befehle werden mehrfach ausgeführt. Wir nennen Konstruktionen, die steuern, ob und wie oft ein Befehl ausgeführt wird, Ablaufsteuerung. Der englische Begriff für Ablaufsteuerung ist *control flow*.

control
flow

2.1. Der if-Befehl

Der **if**-Befehl definiert eine Sequenz von Befehlen, die nur ausgeführt werden, wenn eine Bedingung erfüllt ist.

2.1.1. Syntax

Die einfachste Form des **if**-Befehls hat die folgende Syntax:

```
if (expression) {  
    statements  
}
```

In dieser Konstruktion repräsentiert *expression* einen JavaScript-Ausdruck. Ist der Ausdruck kein Typ **boolean**, wird er in **boolean** automatisch konvertiert. Der Teil **if** (*expression*) wird in manchen Lehrbüchern als **if**-Header oder **if**-Klausel genannt. *statements* repräsentiert einen Befehl-Block. Besteht der Befehl-Block nur aus einem Befehl, kann man die geschweiften Klammer weglassen. Den Teil { *statements* } nennt man **if**-Body.

Als erstes Beispiel schreiben wir einen Befehl, der den Wert der Variable *x* mit seinem Betrag überschreibt. Mathematisch gesehen ist es eine Zuweisung des Wertes $|x|$ an die Variable *x*.

```
if (x < 0) x = -x;
```

Da der **if**-Body nur aus dem einzigen Befehl **x = -x;** besteht, werden die geschweiften Klammern weggelassen. Der obige Code ist deshalb semantisch exakt wie:

```
if (x < 0) {  
    x = -x;  
}
```

Die geschweiften Klammer und die Trennung von **if**-Header und **if**-Body in zwei Zeilen führen zu leserlicher Code, somit ist die zweite Variante zu bevorzugen. Ist der **if**-Body nur ein einfacher Befehl, kann man den ternären Operator verwenden. Er wird im Abschnitt 2.7.1 (S. 53) beschrieben.

Als zweites Beispiel betrachten wir den Code in Abbildung 2.1. Der Code ist eine verbesserte Version vom Programm 1.4. Er berechnet nur die Lösung der Gleichung $ax^2 + bx + c = 0$, wenn die Variable **delta** nicht negativ ist.

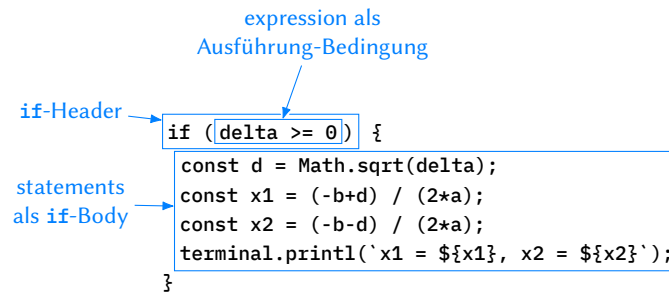


Abbildung 2.1.: Aufbau eines if-Befehl

Folgende Werten werden automatisch in einem `if`-Klausel als **false** evaluiert. Man nennt sie *falsy*-Werten:

falsy-Wert

literale **false**: `if (false)`

besondere Werten: `if (undefined)`
`if( null )`

„Null“-Zahlen: `if (0)`
`if (0n)`
`if (NaN)`

Ein leerer String: `if ( '')`
`if ( "")`
`if (``)`

Der if-else Konstrukt Es gibt Situationen in der Programmierung, in denen will man entweder einen Befehl-Block oder den anderen Befehl-Block ausführen lassen, je nach ob eine Bedingung erfüllt ist. JavaScript hat den Konstrukt `if-else` Befehl um dieses Konzept auszudrücken. Man nennt den angehängten `else` den `else`-Klausel. Der Syntax von `if`-Befehl mit `else`-Klausel ist wie folgt:

```
if (expression) {
  statements true
} else {
  statements false
}
```

Der Befehl-Block *statement true* wird nur ausgeführt, wenn die Bedingung *expression* als **true** evaluiert wird. Sonst wird der Befehl-Block *statement false* ausgeführt.

Der Code

```
if (a > b) {
  max = a;
} else {
  max = b;
}
```

weist den größeren Wert von `a` und `b` in der Variable `max` zu. Wir erweitern den Code in Abbildung 2.1 mit einem `else`-Klausel um eine Meldung anzeigen zu lassen wenn die Gleichung keine reelle Lösung hat:

```

if (delta >= 0) {
  const d = Math.sqrt(delta);
  const x1 = (-b+d) / (2*a);
  const x2 = (-b-d) / (2*a);
  terminal.printl(`x1 = ${x1}, x2 = ${x2}`);
} else {
  terminal.printl("Die Gleichung hat keine reelle Lösung.");
}

```

Das Programm 2.1 ist ein weiteres Beispiel für **if-else**-Konstruktion in JavaScript. Das Programm simuliert den Wurf einer nicht gezinkten Münze. Die Idee ist simpel: aufgeteilt. Die Funktion **Math.random** generiert eine Pseudo-zufällige Zahl im Intervall $[0; 1)$. Der Intervall wird in zwei Teil $[0; 0,5)$ und $[0,5; 1)$. Fällt die generierte Zahl in den ersten Teil, gilt der Wurf als ein Kopf-Wurf; fällt sie in den zweiten Teil, gilt er als ein Zahl-Wurf.

Programm 2.1.: Münzwurf (flipping-a-coin)

```

import {terminal} from "./dfhi.js";

window.main = (...argv) => {
  if (Math.random() < 0.5) {
    terminal.printl("Kopf");
  } else {
    terminal.printl("Zahl");
  }
};

```

Das Program simuliert den Wurf einer Münze. Jedes Mal wenn der Benutzer auf „Rerun“ anklickt, wird entweder das Wort „Kopf“ oder das Wort „Zahl“ ausgegeben.

2.1.2. Verschachtlung von else if-Befehlen

Der Befehl-Block *statements false* kann auch ein **if**-Befehl (ggf. mit einem **else**-Klausel) sein. Der Syntax lautet:

```

if (expression 1) {
  statements 1
} else {
  if (expression 2) {
    statements 2
  } else {
    ...
    if (expression n) {
      statement n
    } else {
      statements m
    }
  }
}

```


Die Bedingungen werden von oben nach unten ausgewertet: wird eine Bedingung *expression* k für $k = 1 \dots n$ als **true** evaluiert, wird der Befehl-Block *statement* k ausgeführt und werden die nächsten Bedingungen sowie die entsprechenden Befehl-Blöcke übersprungen. Wird keine von der Bedingung als **true** evaluiert und existiert der **else**-Klausel, wird der Befehl-Block *statements* m ausgeführt. Besteht der Befehl-Block *statement* m ausschließlich aus einem einzigen Befehl, kann man die geschweiften Klammer der **else**-Klausel weglassen.

Als Beispiel betrachten wir das Programm 2.2. Das Programm konvertiert die Punktzahl einer Klausur in der textuelle Bewertung.

Programm 2.2.: Punktzahl zur Bewertung (point-to-text)

```
import {terminal} from "./dfhi.js";
window.main = (...argv) => {
  const points = Number.parseFloat(argv[0]);
  if (points < 0) {
    terminal.printl("Keine gültige Punktzahl");
  } else if (points < 40) {
    terminal.printl("Nicht ausreichend");
  } else if (points < 50) {
    terminal.printl("Ausreichend");
  } else if (points < 70) {
    terminal.printl("Befriedigend");
  } else if (points < 90) {
    terminal.printl("Gut");
  } else if (points <= 100) {
    terminal.printl("Sehr gut");
  } else {
    terminal.printl("Keine gültige Punktzahl");
  }
};
```

Das Programm akzeptiert eine Punktzahl zwischen 0 und 100 und konvertiert sie nach ASPO Regel in der Bewertung Text von der bewertung „sehr gut“ bis zur der Bewertung „nicht ausreichend“.

2.1.3. Dangling else

Betrachten wir die folgende Konstruktion:

```
if (expression 1) if (expression 2) statements true else statements false
```

Hier ist es nicht eindeutig, zu welchem **if**-Befehl der **else**-Klausel gehört. Die Spezifikation [2, Abschnitt 13.6] ordnet den **else**-Klausel zu dem letzten **if**. Im allgemeine gilt, jeder **else**-Klausel, deren Zuordnung zu einem **if**-Befehl nicht eindeutig ist, wird zu dem nächsten **if**-Befehl zugeordnet. Es ist zu empfehlen, die geschweiften Klammer –zugunsten der Lesbarkeit– zu setzen:

```

if (expression 1) {
    if (expression 2) {
        statements true
    } else {
        statements false
    }
}

```

Fingerübung 2.1: Quadratische Gleichung

Erweitern Sie das Programm 1.4 so, dass das Programm drei Fälle von Δ verarbeiten kann:

- $\Delta < 0$ Die Gleichung hat keine reelle Lösungen.
- $\Delta = 0$ Die Gleichung hat eine Doppellösung.
- $\Delta > 0$ Die Gleichung hat zwei unterschiedliche Lösungen.

Achten Sie darauf, dass bei der Fall $\Delta = 0$ können Sie nur testen, ob $|\Delta - \text{Numeric.MIN_VALUE}| \geq 0$ gilt. Somit müssen Sie diese Bedingung als Erstes oder Letztes testen. Das Dokument von `Numeric.MIN_VALUE` finden Sie in [12].

Fingerübung 2.2: Farbspektrum

Das sichtbare Licht ist elektromagnetisches Spektrum mit Wellenlänge etwa von 400 nm bis 700 nm. Die Farben der Wellenlänge sind wie in der Abbildung 2.2 aufgeteilt. Schreiben Sie ein Programm, dass die Wellenlänge vom Benutzer entgegennimmt und den Name der Farbe ausgeben.

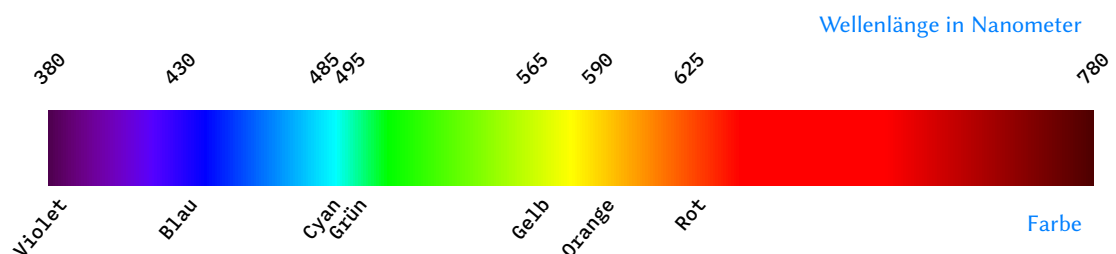


Abbildung 2.2.: Farbspektrum

Fingerübung 2.3: Schaltjahr

Schreiben Sie ein Programm, die eine Jahreszahl entgegennimmt und berechnet ob das Jahr ein Schaltjahr nach Gregorianischer Kalender ist. Die Definition von Schaltjahr finden Sie im Wikipedia Artikel „Schaltjahr“ [18]. Einige Schaltjahren sind 2000, 2020, 2024. Die Jahre 1700, 1800, 1900 sind keine Schaltjahren.

Einige Konstruktionen für gängige Testen in der Programmierung

- Weise den Absoluten Wert von x an x zu

```
if (x < 0) {x = -x;}
```

- Weise den größeren Wert zu a und kleineren Wert an b zu

```
if (a < b) {
    [a, b] = [b, a] ;
}
```

- Überprüfen von Null-Division

- Hier muss man sicher sein, dass b eine ganze Zahl repräsentiert

```
if (b === 0) {
    terminal.print("Null-Division!")
} else {
    result = x / b
}
```

- Bessere alternative in JavaScript wenn man numerische Berechnung hat

```
if (Math.abs(b) <= Numeric.MIN_VALUE) {
    terminal.print("b ist approximiert gleich Null");
} else {
    result = x / b;
}
```

- Überprüfen ob eine Zahl einen gültigen Monat darstellt

```
if (m >= 1 && m <= 12) {
    terminal.print(`${m} ist ein gültige Monat`);
} else {
    terminal.print(`${m} ist eine ungültige Monat Zahl`);
}
```

2.2. Die while-Schleife

Es kommt in der Programmierung vor, dass ein Block von Befehlen wiederholend ausgeführt werden muss, solange eine Bedingung zutrifft. Die einfachste Konstruktion in JavaScript für diese Konzept ist der `while`-Befehl.

2.2.1. Syntax

Die Syntax vom `while`-Befehl lautet:

```
while (expression) {
    statements
}
```

So wie der `if`-Befehl kann man die geschweiften weglassen, wenn der Befehl-Block nur aus einem Befehl besteht.

Der Aufbau vom `while`-Befehl ist gleich wie vom `if`-Befehl –abgesehen davon, dass statt `if` `while` geschrieben wird– dennoch die Semantik vom `while`-Befehl völlig vom `if`-Befehl unterschied. Der Befehl-Block *statements* wird solange wiederholt, bis die Bedingung *expression* zutrifft. Der Befehl-Block nennt man *while*-Body, der Ausdruck *expression* nennt man manchmal die Fortsetzung-Bedingung. Die *expression* wird genau wie die *expression* im `if`-Befehl ausgewertet.

Eine Ausführung des Befehl-Block *statements* ist eine Iteration. In der Regel hängt der Wert des Ausdrucks *expression* von einer oder mehreren Variablen *vor* der Schleife. Diese Variable wird *in* der Schleife geändert, sodass der Wert des Ausdrucks *expression* nach jeder Iteration eventuell geändert wird.

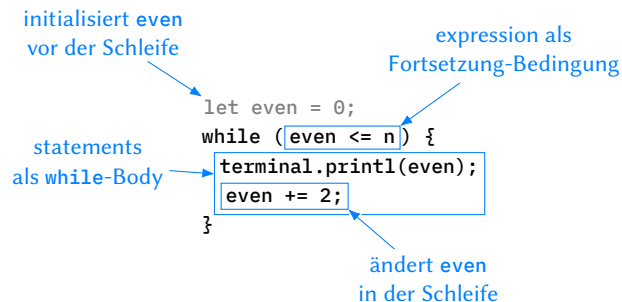


Abbildung 2.3.: Aufbau einer while-Schleife

Die Abbildung 2.3 zeigt den Aufbau eines `while`-Befehls. Der Befehl gibt alle geraden Zahlen von 0 bis n auf dem Browser aus. Die Fortsetzung-Bedingung `even <= n` ist mit der Variable `even`, die vor der Schleife deklariert ist, verknüpft, sodass sie nach jeder Iteration mit anderem Wert von `even` evaluiert wird. Der Befehl `even += 2;` ist die Abkürzung von `even = even + 2;`. Der Befehl bedeutet: addiere 2 auf den aktuellen Wert der Variable `even` und weise das Ergebnis der Addition auf der Variable `even` zu. Die Variable `even` nennt man deshalb manchmal auch *akkumulierte Variable*.

Trace Tabelle Das Programm 2.3 benutzt eine `while`-Schleife, um die ersten n Potenz von 2 zu berechnen und auf dem Browser auszugeben. Die Variable `i` wird lediglich als Zähler verwendet, um die Anzahl der Iterationen zu steuern. Sie wird deshalb als *Index-Variable* bezeichnet. Die

akku-
mulierte
Variable

Index-
Variable

while-Schleife besteht aus drei Befehlen: der erste gibt den *i*-te Exponent aus, der zweite berechnet den nächsten Exponent, der letzte aktualisiert die Variable *i*.

Programm 2.3.: Potenz von 2 [16, S. 57] (**power-of-2**)

```
import {terminal} from "./dfhi.js";

window.main = (...argv) => {
  const n = Number.parseInt(argv[0], 10);
  let power = 1;
  let i = 0;
  while (i <= n) {
    terminal.printl({i, power});
    power *= 2;
    i += 1;
  }
};
```

Das Programm akzeptiert eine Zahl *n* und gibt eine Tabelle die ersten *n* Potenz von 2 aus.

Der Ausdruck `{i, power}` bildet ein literales Objekt mit zweif. Attributen *i* und *power*; die Werten der Attributen entsprechen die Werten der gleichnamigen Variablen. Der Aufbau eines literalen Objekt wird im Kapitel ?? ausführlich beschrieben. Die Funktion `terminal.printl` enthält dann ein Objekt statt primitive Daten. Die Ausgabe ist deshalb detaillierter. Ausgaben wie im Programm 2.3, die die Werten der Variablen in jeder Iteration einer Schleife ausgibt, nennt man ein *Trace* einer Schleife. Das Programm 2.3 gibt die Werten der Variablen bevor jeder Iteration, abhängig von der Situation kann man auch die Werten der Variablen auch nach jeder Iteration ausgeben. *Trace* ist ein einfache dennoch effiziente Methode um semantische Fehler im Programm (*Bug*) zu finden. Es ist in JavaScript gängig, dass man den *Trace* eines Programms oder einer Schleife zwecks Debuggen in die Browser-Console ausgeben. Man schreibt `console.log(...)` statt `terminal.printl(...)`.

Trace

Bug

Fingerübung 2.4: Größter gemeinsamer Teiler

Das Programm 2.4 berechnet den größten gemeinsamen Teiler von zwei natürliche Zahlen.

1. Fügen Sie den Befehl `console.log(...)` im Programm ein, um die Werten der Variablen *a*, *b* und *t* in jeder Iteration und nach der Schleife auf der Browser-Console auszugeben.
2. Schreiben Sie ein Programm **fraction-simplify**, um einen Bruch zu vereinfachen. Der Benutzer gibt einen Bruch ein, das Programm gibt den vereinfachte Bruch auf dem Browser. Benutzen Sie den euklidischer Algorithmus um den größten gemeinsamen Teiler von Nenner und Zähler zu berechnen.

Programm 2.4.: Euklidischer Algorithmus (euclidean-algorithm)

```
import {terminal} from "./dfhi.js";

window.main = (...argv) => {
  const a = Number.parseInt(argv[0], 10);
  const b = Number.parseInt(argv[1], 10);
  let t = b;
  while (b !== 0) {
    t = b;
    b = a % b;
    a = t;
  }

  terminal.print(a);
};
```

2.2.2. Präfix- und Postfix- Operatoren

Die Schreibweise `power *= 2` ist die Abkürzung von `power = power * 2`. Eben so ist die Schreibweise `i += 1` die Abkürzung von `i = i + 1`. Die Zuweisung `i = i + 1`; hat noch zwei kompaktere Abkürzungen: *Präfix Inkrement* `++i`; und *Postfix Inkrement* `i++`; . Werden sie als eigenständig Befehl verwendet, haben die beide Befehlen gleiche Effekt. Die folgenden Schleife sind semantisch identisch:

<pre>let i = 0; while (i <= n) { terminal.printl({i, power}); power *= 2; ++i; }</pre>	<pre>let i = 0; while (i <= n) { terminal.printl({i, power}); power *= 2; i++; }</pre>
---	---

Präfix
Inkrement
Postfix
Inkrement

Werden sie dennoch als Ausdruck verwendet, haben sie unterschiedliche Effekten. Während die Präfix-Schreibweise das Inkrement ausführt, dann das Ergebnis zurück gibt, führt die Postfix-Schreibweise das Inkrement aus, weise das Ergebnis in der Variable, dennoch gibt die Postfix-Schreibweise das originale Wert zurück. Wir vergleichen die Trace-Tabelle von beide Schreibweisen bei `n = 3`:

Präfix

```
let i = -1;
while ( (++i) <= n ) {
    terminal.printl({i, power});
    power *= 2;
}
```

i	i+1	++i <= n	Ausgabe
-1	0	0 <= 3	{i: 0, power: 1}
0	1	1 <= 3	{i: 1, power: 2}
1	2	2 <= 3	{i: 2, power: 4}
2	3	3 <= 3	{i: 3, power: 8}
3	4	4 <= 3	(Anhalten)

Postfix

```
let i = -1;
while ( (i++) <= n ) {
    terminal.printl({i, power});
    power *= 2;
}
```

i	i+1	i++ <= n	Ausgabe
-1	0	-1 <= 3	{i: 0, power: 1}
0	1	0 <= 3	{i: 1, power: 2}
1	2	1 <= 3	{i: 2, power: 4}
2	3	2 <= 3	{i: 3, power: 8}
3	4	3 <= 3	{i: 4, power: 16}
4	5	4 <= 3	(Anhalten)

Wie wir in der Trace-Tabelle entgegennehmen können, der Operator ++ ändert seinen Operand. Man nennt es Nebeneffekt eines Operators. Neben Priorität ist Nebeneffekt ein Aspekt die man bei dem Aufbau eines Ausdrucks beachten muss. Der Gegenteil von der Operation Inkrement ist Dekrement (--). Es gibt sowohl Prefix- als auch Postfix- Schreibweise für den Operator --, die genau wie Inkrement funktionieren, abgesehen davon dass statt einer Addition (+1) eine Subtraktion (-1) ausgeführt wird.

Beispiel: Berechnen die größten Zweierpotenzen

Als komplizierteres Beispiel bauen wir eine Konstruktion, die die größten Zweierpotenzen 2^i berechnet, die kleiner als eine gegebene Zahl n ist. Zum Beispiel:

n	$\max(2^i)$	i
5	4	2
10	8	3
64	64	6
1000	512	9

Diese Berechnung ist durch eine while-Schleife einfach durchzuführen:

```
let power = 1;
while (2 * power <= n) {
    power = 2 * power;
}
terminal.printl(power);
```

Durch die Beobachtung können wir feststellen:

- Die Variable `power` ist immer eine Potenz von 2.
- `power` ist nie größer als n (wegen $2p \leq n \Leftrightarrow p \leq n/2 \Rightarrow p \leq n$)
- `power` wird nach jeder Iteration größer (verdoppelt), deshalb die Schleife anhalten muss.
- Nach der Schleife ist $2 \cdot \text{power}$ (der nächste Potenz von 2 nach der `power`) größer als n .

Anhand dieser Beobachtung wissen wir, dass das Programm die Berechnung ausführt, wie wir sie erwarten. Das Programm ist dennoch nicht sehr effizient. Die Multiplikation `2 * power` wird in jeder Iteration doppelt ausgeführt. Wir schreiben die Schleife um:

```
let power = 1;
const halfN = Math.floor(n/2);
while (power <= halfN) {
    power = 2 * power;
}
terminal.println(power);
```

Diese Schreibweise eliminiert die Multiplikation in der Bedingung. Die Konstant `halfN` wird nur ein mal berechnet. Wir haben für jede Iteration nur noch eine Vergleich-Operation und eine Multiplikation-Operation.

Fingerübung 2.5: Trace einer Schleife

Erfassen Sie die obigen Schleife in ein vollständig Programm:

- Fügen Sie in der Schleife-Body Befehlen hinzu, um die Werten der Variable so wie Anzahl der Iterationen vor jeder Iteration auszugeben. Benutzen Sie eine Variable um die Anzahl der Iterationen zu verfolgen.
- Erweitern Sie das Programm so, das es zusätzlich nach der Schleife noch `power * 2` ausgibt.
- Entspricht der Trace der obigen Beobachtung?

2.3. Die for-Schleife

Neben der `while`-Schleife bietet JavaScript weitere Konstruktionen um Programme mit Schleife zu schreiben. Ein davon ist die `for`-Schleife. Die Syntax der `for`-Schleife ermöglicht man eine kompakte und leserliche Schleife zu schreiben. Die abstrakte Syntax der `for`-Schleife ist wie folgt:

```
for ([initialize]; [expression]; [update]) {
    statements
}
```

Die Semantik der Schleife ist –bis auf die Gültigkeitsbereich der Variablen– äquivalent zu der `while`-Schleife mit folgenden Schema:

```
initialize
while (expression) {
    statements
    update
}
```

Der Teil `initialize` kann eine Zuweisung- oder eine Deklaration-Befehl sein. Wie die abstrakte Syntax angedeutet hat, sind alle Teilen der `for`-Header: `initialize`, `expression` und `update` optional.

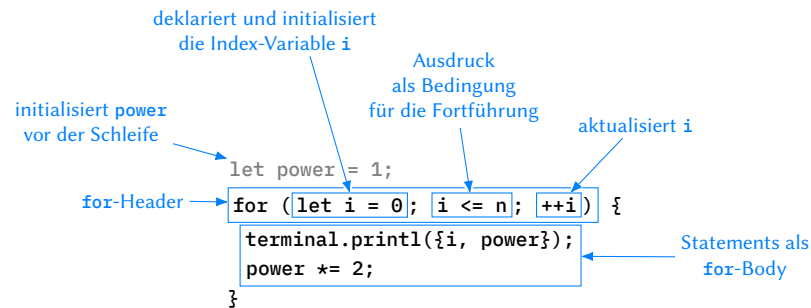


Abbildung 2.4.: Aufbau einer for-Schleife

Die Abbildung 2.4 zeigt den Aufbau einer konkreten **for**-Schleife, die fast wie die **while**-Schleife im Programm 2.3 macht. Wir erkennen in dem Programm wieder den Muster `++i`. In diese **for**-Schleife wird der Muster als ein eigenständiger Befehl verwendet, sodass es unerheblich ist, ob man die Präfix- (`++i`) oder die Postfix-Schreibe (`i++`) im *update*-Befehl benutzt.

Ob die **for**- oder **while**-Schleife benutzt wird, entscheidet in der Regel der Entwickler je nachdem, welche Aufgabestellung er hat. Oft wird **for**-Schleife verwendet, wenn die Anzahl der Iterationen vor dem Schleife-Anfang bestimmt werden kann, und **while**-Schleife wenn man die Fortsetzung-Bedingung ohne eine zusätzliche Index-Variable formulieren kann, oder wenn die Anzahl der Iterationen vor dem Schleife-Anfang nicht festgelegt werden kann. Unter „vor dem Schleife-Anfang“ versteht man die *Laufzeit* des Programms und nicht den Zeitpunkt beim Schreiben des Code. Die Variable `n` im Programm 2.3 ist zwar zur Zeitpunkt, bei dem der Code geschrieben ist, nicht bekannt, dennoch zur Laufzeit des Programm ist diese Variable bekannt, bevor die Schleife anfängt. Während die **for**-Schleife in der Abbildung 2.4 eleganter als die **while**-Schleife für das Programm 2.3 ist, sind die beide **while**-Schleife im Abschnitt 2.2.2 die elegantere Formulierung für das vorgestellte Problem. Interessieren wir uns dennoch auch noch, für welchen maximalen Exponent i gilt $p = 2^i < n$, dann ist eine **for**-Schleife ist eventuell leserlicher als eine **while**-Schleife:

```
let power = 1,
    i = 0;
const halfN = Math.floor(n/2);
for(i = 0; power <= halfN; ++i) {
    power = 2 * power;
}
```

Da wir uns für den größte Exponent, der in der Variable `i` gespeichert wird, *nach* der Schleife interessieren, muss die Variable `i` *außerhalb* der **for**-Schleife deklarierte werden. Wir diskutieren im nächsten Abschnitt über die Gültigkeitsbereich einer Variable.

Fingerübung 2.6

Erfassen Sie die obigen Schleife in einem Programm **max-power-for**. Das Programm soll den Wert der Variable `i` und `power` nach der Schleife ausgeben. Danach schreiben Sie ein neu Programm **max-power-while**, das das Gleich wie das Programm **max-power-for** macht, dennoch verwenden Sie die **while**-Schleife statt **for**-Schleife.

Fingerübung 2.7

Terminieren diese Schleife?

- Schleife 1

```
const LIMIT = 0.3;
for (let i = 0.1; i !== LIMIT; i += 0.1) {
  terminal.printl(i);
}
```

- Schleife 2

```
const LIMIT = 3;
for (let i = 1; i !== LIMIT; i++) {
  terminal.printl(i);
}
```

Erklären Sie das Ergebnis!

2.4. Gültigkeitsbereich eines Bezeichners

Die Gültigkeitsbereich eines Identifier (engl. *Scope*) (Variablen, Konstanten, Funktionen) ist der Teil eines Programm, der den Identifier bei Name referenzieren kann. Scope

Der Gültigkeitsbereich einer Variablen, die mit **let** deklariert ist, bzw. einer Konstant, die mit **const** deklariert ist, ist der Code-Block, in dem die Variable bzw. die Konstant deklariert sind. Code-Blöcke sind Code zwischen den geschweiften Klammer (**{ ... }**). Code in **for**-Header gehört zu den Code-Block der **for**-Body. Man sagt, diese Variable und Konstanten haben *lexikalische Scope*, oder auch *Block-Scope*. Denn ihren Scope lassen sich via lexikalische Regel bestimmen. lexika-
lische
Scope
Block-
Scope
Funktion-
Scope

Die Gültigkeitsbereich einer Variable, die mit **var** deklariert ist, ist die Funktion, in der sie deklariert ist, oder global, wenn sie nicht in einer Funktion deklariert ist. Man sagt, diese Variable hat eine *Funktion-Scope*. Wir werden mehr über Funktion-Scope im Kapitel 3.4 diskutieren.

Da lexikalische Scope einfache zu bestimmen als Funktion-Scope, soll man den lexikalischen Scope bevorzugt anwenden, indem man **let** benutzt.

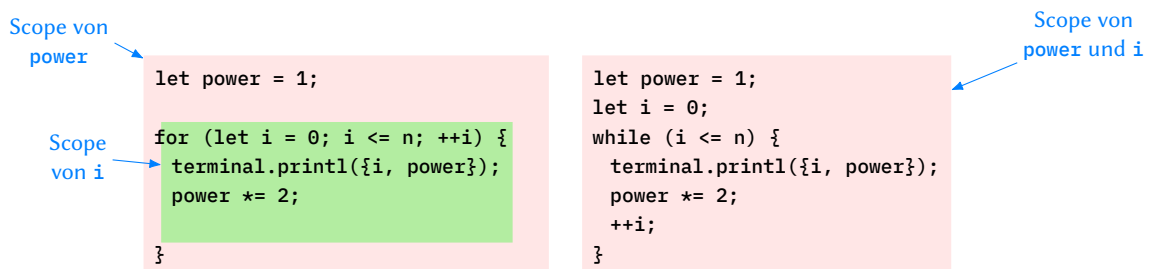


Abbildung 2.5.: Lexikalische Scope von Variable

Die Abbildung 2.5 zeigt die Scope der Variablen **power** und **i** in jeweiligen Schleifen. In der **for**-Schleife hat die Variable **i** den kleineren Scope (als Grün markierte Bereiche) als die Variable **power**. In der **while** Schleife haben die beide Variablen den gleichen Scope. Ersetzt man **let** durch **var** in der **for**-Schleife, so erweitert man den Scope der Variable **i** auf dem ganzen Bereich wie der

Variable **power**. Es ist eine gute Programmierstil, den Scope einer Variable so klein wie möglich zu halten.

Gängige Schleife-Konstruktionen Hier sind die gängige Schleife-Konstruktionen, die man in der Programmierung als Vorlag öfter verwendet. Sie können die Implementierung der Konstruktionen ergänzen. Es gibt nicht nur eine Implementierung.

- *Berechnen den größten Exponent von 2, der kleiner als n ist.* (Platz für Ihre Notizen)

- *Berechnen die endliche Summe $\sum_{i=1}^n i = 1 + 2 + \dots + n$* (Platz für Ihre Notizen)

- *Berechnen das endliche Produkt $\prod_{i=1}^n i = n! = 1 \cdot 2 \cdot \dots \cdot n$* (Platz für Ihre Notizen)

- *Gibt eine Tabelle von Funktionswerten (z.B $f(x) = \sin(x)$ für $x \in [-\pi.. \pi]$, Schrittweite 0.1) aus* (Platz für Ihre Notizen)

2.5. Verschachtlung

Wie bereits bei dem Abschnitt über den `if`-Befehl (Programm 2.2) erwähnt, ein oder mehr `if`-Befehl kann an der Stelle von einem *statements* verwendet werden. Diese Regel gilt auch für `for`- und `while`- so wie weitere Befehlen. Man kann ein oder mehrere von denen an der Stelle von *statements* in der abstrakten Syntax-Beschreibung verwenden um zusammengesetzten Befehl zu bauen.

Wir schreiben ein Programm, das eine natürliche Zahl n vom Benutzer erhält und ein Dreieck aus Sternzeichen (*) nach folgenden Schema ausgibt:

n	Ausgabe
1	*
2	* * *
3	* * * * * *
4	* * * * * * * * * *
...	...

Abbildung 2.6.: Ausgabe des Programm 2.5

Das Dreieck hat n Zeilen. Eine Zeile l des Dreieck besteht aus $n - l$ Leerzeichen und genau l Star-Zeichen, die von Leerzeichen getrennt wird.

Das Programm 2.5 verwendet das Objekt `terminal`, um das Dreieck Zeichen für Zeichen auszugeben. Die `line`-te Iteration der äußersten Schleife gibt eine Zeile des Dreieck aus. Die äußerste Schleife besteht wiederum aus zwei interne Schleife, die eine Zeile in zwei Teilen ausgibt.

Im Prinzip kann man auch das Dreieck zuerst als einen String aufbauen, dann kann man den ganzen String ausgeben. Der Vorteil dieses Verfahren ist, man kann den String mit CSS-Segment aufbauen, sodass man die Formatierung der Ausgabe, etwa Farbe, Schriftart, usw. steuern kann. Zum Beispiel möchte man einen Text in Blau ausgibt, kann man zu erst den Text in einem `<span>`-Tag mit entsprechenden CSS-Format Befehl aufbauen, dann den String via `terminal.printh` ausgeben:

```
const text = "Ein Text in Blau";
const style = "color:blue; font-weight: bold;";
const output = `${text}</span>`;
terminal.printh(output);
```

Die Methode `terminal.printh` erwartet ein Argument als gültige HTML-Segment. Sie fügt diese HTML Segment in der Terminal als DOM-Elementen ein.

Programm 2.5.: Programm zum Zeichnen eines Dreieck (triangle-star)

```
import {terminal} from "./dfhi.js";

window.main = (...argv) => {
  const n = Number.parseInt(argv[0], 10);
  const character = "*",
        separator = " ";
  for (let line = 1; line <= n; ++line) {
    for (let space = n-line; space > 0; --space) {
      terminal.print(separator);
    }
    for (let star = 1; star <= line; ++star) {
      terminal.print(character);
      if (star < line) {
        terminal.print(separator);
      }
    }
    terminal.println()
  }
};
```

Das Program verwendet `terminal.print` bzw. `terminal.println` um Zeichen auf dem Browser auszugeben.

Fingerübung 2.8

- Man kann den Tag `<table>` benutzen um die Sternzeichen zu rangieren. Schreiben Sie das Programm 2.5 um. Bauen Sie eine HTML-Tabelle auf, sodass jede Zeile des Dreieck ist eine Zeile der Tabelle, ein leeren Abstand entspricht einen leeren `<td>`. Geben Sie anschließend die aufgebaute Tabelle via `terminal.printh` aus.
- Erweitern Sie das Programm so, dass es eine Raute ausgibt, die aus $2n - 1$ Zeile besteht. Die Zeile von 1 bis n der Raute sind genau so wie vom oben beschriebenen Dreieck. Die Raute hat die Zeile n als horizontale symmetrische Achse.

Für $n = 3$ und $n = 4$ sieht die Raute so aus:

```

  *           *
 * *         * *
* * *       * * *
* *         * * * *
  *           * * *
              * *
                *
```

Programm 2.6.: Harmonische Reihe (harmonic)

```
import {terminal} from "./dfhi.js";

window.main = (...argv) => {
  const n = Number.parseInt(argv[0]);
  let h = 0;
  for(let i = 1; i <=n; ++i) {
    h += 1/i;
  }
  terminal.printl(`H_${n} = ${h}`);
};
```

Das Program berechnet approximiert die harmonische Reihe bis zum n -te Glied.

2.6. Beispielprogramme

In diesem Abschnitt untersuchen wir einige Programme, deren Paradigmen wir öfter wieder werden können. Die ausgewählte Themen in diesem Abschnitt stammt aus dem Abschnitt 1.3 vom [16]. Die Programme-Code wird für die Sprache JavaScript angepasst.

Endliche Summe sind Summe, man als $\sum_{n=0}^N t_n$ für $N \in \mathbf{N}$ formulieren kann. Endliche Summe lassen sich in Programmierung mit Schleife sehr gut abbilden. In diesem Abschnitt untersuchen wir einige interessante Endliche Summe.

Die harmonische Reihe

Die Reihe

$$H_n = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n}$$

nennt man die harmonische Reihe. Diese Reihe lässt sich sehr einfach in Computer approximiert berechnen. Das Programm 2.6 berechnet die Reihe für eine eingegebene n . Das Programm kann man als Basis verwendet um Summe von endlichen Summanden zu rechnen.

Fingerübung 2.9

Der approximiert Wert von e ist

$$e = \sum_{n=0}^{\infty} \frac{1}{n!} = 1 + \frac{1}{1!} + \frac{1}{2!} + \frac{1}{3!} + \cdots$$

für

$$\begin{aligned} 0! &= 1 \\ n! &= 1 \cdot 2 \cdot 3 \cdots n \quad \text{für } n > 0 \end{aligned}$$

Schreiben Sie ein Programm, das den Wert von e annähernd bis zum eingegeben n -Glieder berechnet. Vergleichen Sie das Ergebnis Ihres Programms mit dem Konstant `Math.E`.

Zahlen Konvertierung

Das Programm 2.7 konvertiert die eingegebene Zahl in Binär-Darstellung. Das Programm basiert auf die Zerlegung einer Zahl in eine Summe von Zweierpotenzen. Zum Beispiel $13 = 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 1101_2$. Um die Binär-Darstellung einer Zahl n zu berechnen, finden wir zu erst die größte Potenz von 2, die noch kleiner als oder gleich n ist. Wir suchen –von Link nach Recht als *absteigend*– die Potenz von 2, die die Summe bilden. Sie entsprechen den Ziffer **1** in der Darstellung. Die Potenz von 2, die die Summe nicht bilden, entsprechen den Ziffer **0**. Als Beispiel nehmen wir die Zahle $n = 13$.

1. 8 ist die größte Potenz von 2, die noch kleiner als oder gleich 13 ist.
2. Die Potenzen von 2 sind $2^3 = 8$, $2^2 = 4$, $2^1 = 2$, $2^0 = 1$. Von Links nach Recht: Ist ein Potenz p kleiner als oder gleich n , bildet sie als Summanden die Summe der 2er-Potenz von n . Wir untersuchen für den Differenz $n = n - p$ die weiter 2er-Potenzen. Der Prozess wird solange wiederholt, bis es kein Potenzen mehr gibt.

$$\begin{array}{llll}
 p = 2^3 = 8 & n = 13 > 8 & \Rightarrow 13 = 2^3 + \dots \\
 p = 2^2 = 4 & n = 13 - 8 = 5 > 4 & \Rightarrow 13 = 2^3 + 2^2 + \dots \\
 p = 2^1 = 2 & n = 13 - 8 - 4 = 1 < 2 & \Rightarrow 13 = 2^3 + 2^2 + 0 \cdot 2^1 + \dots \\
 p = 2^0 = 1 & n = 13 - 8 - 4 = 1 & \Rightarrow 13 = 2^3 + 2^2 + 0 \cdot 2^1 + 2^0
 \end{array}$$

3. Endergebnis: $n = 13 = 2^3 + 2^2 + 2^0 = 1101_2$

Ist das Basis größer als 2, besteht die Summe nicht nur aus Summand als einfache Potenzen, sondern auch Summand als mehrfache Potenzen. Zum Beispiel die Zahl $n = 190$ wird in Basis 8 als 276_8 dargestellt, denn $190 = 2 \cdot 8^2 + 7 \cdot 8^1 + 6 \cdot 8^0$. Wir können den obigen Zerlegung-Prozess verallgemeinern, indem wir nicht nur die Potenzen finden, sondern auch die Mehrfach von jeweiligen Potenz. Die Mehrfach ist durch die ganzzahlige Division $\lfloor \frac{n}{p} \rfloor$ zu bestimmen. Als Beispiel konvertieren wir die Zahle $n = 190$ in Basis 8.

1. $p = 64$ ist die größten Potenz von 8, die kleiner oder gleich $n = 190$
2. Die Potenzen von 8 sind $8^2 = 64$, $8^1 = 8$, $8^0 = 1$. Von Links nach Rechts: wir bestimmen den Faktor d für jede Potenzen $p = 8^3$. d ist der e -te Stelle in der Oktal Darstellung. Diese Prozess wird mit $n = n - dp$ solange wiederholt, solange $p > 0$ gilt.

$$\begin{array}{llll}
 p = 8^2 = 64 & n = 190 & d = \lfloor 190/64 \rfloor = 2 & 190 = 2 \cdot 8^2 + \dots \\
 p = 8^1 = 8 & n = 62 & d = \lfloor 62/8 \rfloor = 7 & 190 = 2 \cdot 8^2 + 7 \cdot 8^1 + \dots \\
 p = 8^0 = 1 & n = 6 & d = \lfloor 6/1 \rfloor = 6 & 190 = 2 \cdot 8^2 + 7 \cdot 8^1 + 6 \cdot 8^0
 \end{array}$$

3. Endergebnis: $190 = 276_8$

Programm 2.7.: Dezimale Zahl in Binär-Darstellung (binary)

```
import {terminal} from "./dfhi.js";

window.main = (...argv) => {
  let n = BigInt(argv[0]);
  let power = 1n;
  const LIMIT = n / 2n;
  // find the largest power of 2, which is less than or equals to n/2
  while(power <= LIMIT) {
    power *= 2n;
  }
  let convert = "";
  while(power > 0n) {
    if (n < power) {
      convert += "0";
    } else {
      convert += "1";
      n -= power;
    }
    power /= 2n;
  }
  terminal.printl(convert);
};
```

Das Programm verwendet den Datentype `BigInt` um die Ganzzahl-Division zu führen, ohne sich die Funktion `Math.trunc` zu bedienen.

Fingerübung 2.10

Stellen Sie die Zahl 13 in Basis 8 und Basis 3 dar. Benutzen Sie den oben beschriebenen Algorithmus.

Fingerübung 2.11

Schreiben Sie ein Programm, das zweit Integer als Parameter aus `argv` entgegennimmt. Das Programm konvertiert die erste Integer in Basis von der zweiten Integer und das Ergebnis auf dem Browser ausgibt. Die Basis ist eine Zahl zwischen 2 und einschließlich 16. Beispielen:

- Eingabe: 15 16 Ausgabe: F (F ist der 15.te Ziffer in Hexadezimal-System)
- Eingabe: 17 8 Ausgabe: 21 ($17 = 2 \cdot 8^1 + 1 \cdot 8^0$)
- Eingabe: 13 3 Ausgabe: 111 ($13 = 3^2 + 3^1 + 3^0$)

Portieren Sie zuerst das Programm 2.7 in ein Program `octal`, das eine Zahl in Oktal-Darstellung konvertiert.

2.7. Weitere Konstruktionen zur Ablaufsteuerung

Neben der `while`- so wie `for`-Schleife und dem Befehl `if`-Befehl bietet JavaScript noch weiteres Befehlen um Control Flow zu steuern. Sie werden seltener als `if`-, `while`-, und `for`-Befehlen verwendet. Hier werden sie kurz beschrieben.

2.7.1. Der bedingte (ternäre) Operator

Der ternäre Operator bildet keinen Befehl zum Steuern vom Control Flow aus. Er kann verwendet werden um kompakte Ausdruck der Form „wenn *X* gilt dann *a* sonst *b*“ zu formulieren. Die Syntax des ternären Operator lautet:

`test_expression ? true_expression : false_expression`

Der ternäre Operator bildet einen Ausdruck. Der Wert des Ausdruck ist *true expression* wenn der Ausdruck *test expression* als `true` evaluiert wird, sonst ist der Wert des Ausdruck *false expression*. Er wird oft in Kombination mit eine Zuweisungsbefehl verwendet. Das Idiom ist wie folgt:

`value = test_expression ? true_expression : false_expression;`

Das Programm 2.8 zeigt das Zugriffsrecht einer Datei in Linux/Unix System an. In einem Linux-/Unix System wird der Zugriffsrecht als eine Oktalziffer codiert (Abbildung 2.7). Das Programm konvertiert eine Oktalziffer in der übliche Ausgabe der Linux Befehl `ls`. Es verwendet den ternären Operator anstatt einen `if`-Befehl. Das Programm ist dann kompakter als wenn es mit `if`-Befehl implementiert wäre.

Programm 2.8.: Anzeige des Zugriffsrechtes (`print-octal-permission`)

```
import {terminal} from "./dfhi.js";

window.main = (...argv) => {
  const octal = parseInt(argv[0], 8);
  const readableMask = 0b100;
  const readable = (octal & readableMask) ? "r" : "-";
  const writableMask = 0b010;
  const writable = (octal & writableMask) ? "w" : "-";
  const executableMask = 0b001;
  const executable = (octal & executableMask) ? "x" : "-";
  terminal.printl(`${readable}${writable}${executable}`); // $
};
```

Mögliche Eingabe sind zum Beispiel 7, oder 6. Die Zahl 7 entspricht das Zugriffsrecht `rxw`, die Zahl 6 entspricht das Zugriffsrecht `rw-`.

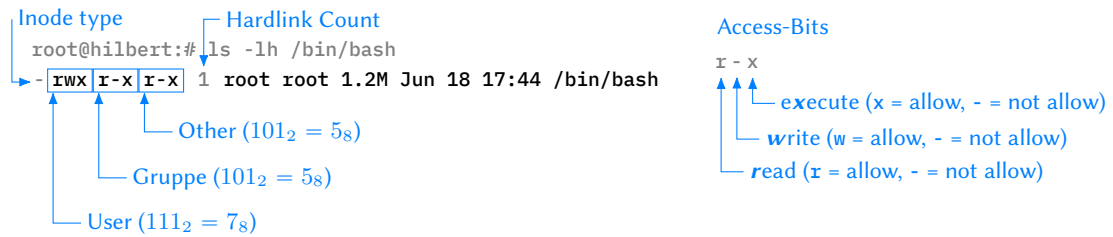


Abbildung 2.7.: Zugriff-Bit in Linux-Dateisystem

2.7.2. Der Befehl do-while

Der Befehl **do-while** hat die Syntax

```
do {
    statements
} while (condition);
```

Er führt den Befehl-Block *statements* mindestens einmal durch. Er kann deshalb als **while**-Schleife wie folgt umformuliert werden:

```
statements
while (condition) {
    statements
}
```

2.7.3. Der Befehl break

Es ist manchmal erwünscht, eine Schleife –evtl. bei Auftritt einer Bedingung– sofort zu verlassen. In solche Situation kann man den Befehl **break** benutzen. Der folgenden Code-Segment testet ob eine natürliche Zahle *n* größer als 1 eine Primzahl ist:

```
for (let factor = 2n; factor <= n/factor; ++factor) {
    if (n % factor === 0n) {
        break;
    }
}
if (factor > n/factor) {
    terminal.println(`${n} is a prime`);
} else {
    terminal.println(`${n} is not a prime`);
}
```

Wir benutzen den Literal `2n` statt `2` um eine Zahl der Typ *BigInt* (`[7]`) statt *number* zu erzeugen. Der Operation `n/factor` führt eine ganzzahlige Division, statt Gleitkommazahl-Division aus, wenn deren Operanden *BigInt* sind. Der Datentyp *BigInt* wird im Moment nur in Chrome ab Version 67 und Firefox ab Version 68 unterstützt. Edge und Internet Explorer kennen die Schreibweise `2n` (noch) nicht.

Hier ist es sinnvoll, dass wir `var` statt `let` benutzen, um die Variable `factor` zu deklarieren. Denn wir benutzen die Variable `factor` auch nach der Schleife.

2.7.4. Der Befehl `continue`

Der Befehl `continue` lässt den Control Flow die Schleife die aktuellen Iteration verlassen und die nächsten Iteration starten. Deshalb darf der Befehl `continue` nur in einer Schleife verwendet werden. In einer `for`-Schleife führt der Control Flow den `update`-Befehl aus, bevor er die nächste Iteration startet.

2.7.5. Der Befehl `switch`

Wenn man einen Wert mit Elementen einer Menge vergleichen muss und je nachdem, welche Element in der Menge zu dem Wert passt, kann man den Befehl `switch` benutzen.

Die Syntax vom `switch`-Befehl lautet:

```
switch (value) {
  case value 1 : {
    statement 1
  } break;
  case value 2 : {
    statement 2
  } break;
  [...]
  [default: {
    statement n
  }]
}
```

Als Beispiel betrachten wir einen Code-Segment, der eine Zahle von 0 bis 7 in Tage-Name konvertiert:

```
switch (day) {
  case 0: terminal.println("Sunday");    break;
  case 1: terminal.println("Monday");    break;
  case 2: terminal.println("Tuesday");   break;
  case 3: terminal.println("Wednesday"); break;
  case 4: terminal.println("Thursday");  break;
  case 5: terminal.println("Friday");    break;
  case 6: terminal.println("Saturday");  break;
  default: terminal.println("Not a day");
}
```

2.8. Ausnahme-Behandlung

Unter Ausnahme-Handlung versteht man den Mechanismen einer Programmier-Sprache, um unerwartete Situation zu verarbeiten. Manchmal nennt man Ausnahme-Handlung auch „Fehlerbehandlung“. Die Sprache JavaScript bietet die Konstruktion `try-catch` für Ausnahme-Handlung, sowie den Befehl `throw` um einen Fehler zu erzeugen. Zum Beispiel:

```
try {
  const monat = Number.parseInt(argv[0], 10);
  const isMonatANumber = !( Number.isNaN(argv[0]) || Number.isNaN(monat) );
  if (!isMonatANumber) {
```

```
        throw new Error("Der eingegebene Wert ist keine Zahl");
    } else if (monat < 1 || monat > 12) {
        throw new RangeError("Die eingegebene Zahl repräsentiert keinen Monat.");
    }
    terminal.print(`Der eingegebene Monat ist ${monat}`);
} catch (e) {
    let error = `<span class="error">${e}</span>`;
    terminal.printh(error);
}
```

Der obigen Code überprüft, ob die Eingabe einen Monat repräsentiert ist oder nicht. Die Überprüfung passiert in zwei Schritten: zuerst wird geprüft, ob die Eingabe eine Ganzzahl repräsentiert, dann wird geprüft ob die Ganzzahl im Bereich von 1 bis 12 ist. Der letzte Befehl im `try`-Block wird nur ausgeführt, wenn die Eingabe eine Monat-Zahl repräsentiert.

Im Gegensatz zu reinen Objekt-orientierten Sprachen wie zum Beispiel Java werden die Befehle `try-catch` so wie `throw` sehr sparsam verwendet. Deshalb ist es umso wichtiger, dass man diese Befehle nur anwenden, um Fehler –also Ausnahmesituation– zu behandeln, und nicht als eine Alternative um Programmablauf zu steuern. Es ist auch nicht gängig in JavaScript, eine tiefe Vererbung-Hierarchie von `Error` zu konzipieren.

Fingerübung 2.12

Recherchieren Sie drei Built-In Errors in JavaScript.



Abbildung 2.8.: Symmetry 131 [4]

3. Modulare Programmierung

Die Funktion ist das erste Mittel der Sprache JavaScript, um ein Programm in kleine Einheiten zu zerlegen. Dieses Konzept ist wichtig, denn auf einer Seite bietet es uns die Möglichkeit, komplexen, vielschichtigen Problemen in klar getrennte Teil-Problemen zu lösen; auf der anderen Seite können wir Funktionen in Modulen gruppieren und wieder verwenden, um weiter ähnlichen Probleme zu lösen. Man nennt dieses Konzept Modular Programmierung.

In diesem Kapitel diskutieren wir auch darüber, wie wir Code von dritten Party in eigene Programm verwenden können. Code von dritten Party nennt man Abhängigkeit. „Abhängigkeit“ ist ein abstrakter Begriff. JavaScript als Sprache bietet keine Möglichkeit, die Abhängigkeit in konkreten Einheit zu erfassen. Wir benutzen deshalb das Tool **NPM** um abhängigen Paketen zu verwalten. Wir unterscheiden dennoch zwischen abstrakten Konzepten wie Abhängigkeit, Modularisierung und die konkreten Umsetzung von dieser Konzepten, denn das Konzept bleibt, die technische Umsetzung von Konzepten ändert sich mit der Zeit. Unsere Ziel ist, durch die verwendeten Tools die Konzepten zu verstehen und in andere Technologie zu übertragen.

3.1. Funktion

Wir haben bisher vordefinierte Funktionen benutzen, zum Beispiel die Funktion `Math.sqrt()` um Wurzel einer Zahl zu ziehen. Wir können Funktionen auch selbst definieren.

3.1.1. Definition einer Funktion

Die allgemeine Syntax der Definition einer Funktion lautet:

```
const name = ([param1, param2, ..., paramk][,restParam]) => {  
    statements  
}
```

Die Funktion `toDegree` wie folgt konvertiert einen Winkel aus Radiant in Grad. Die Funktion `hypotenuse` berechnet die Länge der Hypotenuse eines rechtwinkliges Dreiecks aus zwei andere Kanten.

```
const toDegree = (angle) => {  
    return angle * 180/Math.PI;  
}  
  
const hypotenuse = (a, b) => {  
    return Math.sqrt(a**2 + b**2);  
}
```

In JavaScript ist eine Funktion auch ein Objekt. Man definiert eine Funktion, hier mit der Syntax von *Arrow-Function*, dann weist man die Arrow Function an einer Variable zu. Arrow-Functions binden den `this`-Bezeichner in lexikalische Scope. Somit sind sie weniger fehleranfällig.

Arrow-
Function

Funktionen, die mit dem Schlüsselwort **function** deklariert sind, binden den **this**-Bezeichner beim Aufruf. Sie werden im Kapitel ?? beschrieben. In diesem Kapitel definieren wir nur Funktionen mit der Syntax von Arrow-Function.

Funktionen haben eine Liste von Parameter. Die Parameter-Liste wird in () geschrieben und kann leer sein. Die Funktion **toDegree** hat eine Parameter-Liste aus einem Parameter. Die Funktion **hypotenuse** hat eine Parameter-Liste aus zwei Parametern. Der Funktions-Rumpf oder Funktions-Body besteht aus in geschweiften Klammern erfassten Befehlen, die beim Aufruf der Funktion ausgeführt werden. Die Abbildung 3.1 zeigt den Aufbau einer Funktion.

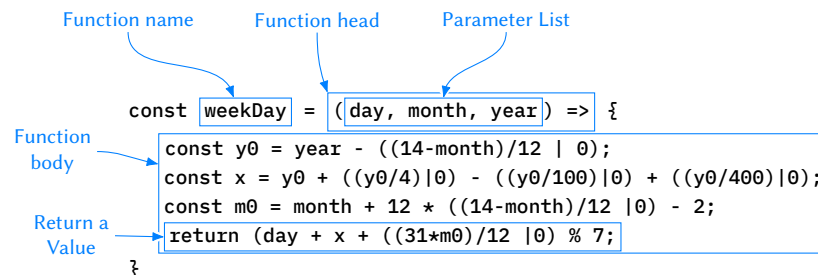


Abbildung 3.1.: Definition einer Funktion

Funktion-Inhalt

Der Inhalt einer Funktion, manchmal wird auch als Funktions-Body oder Rumpf der Funktion genannt, kann beliebige Befehle enthalten. Eine Deklaration einer Funktion ist auch als Befehl im Funktions-Body erlaubt. Der Inhalt der Funktion **weekday** besteht aus vier Befehlen, die sequentiell nacheinander ausgeführt werden. Die Inhalte der Funktion **toDegree** sowie der Funktion **hypotenuse** bestehen jeweils aus nur einem Befehl. Sie werden dennoch in einem Paar von geschweiften Klammern erfasst.

Rückgabewert einer Funktion

Der Befehl **return [expr];** beendet die Ausführung der Funktion an der Stelle des Befehls **return** und gibt den Wert der Ausdruck **expr** zurück, sodass man auch diesen Wert an eine Variable zuweisen kann:

```

const radian = Math.PI / 4;
const degree = toDegree(radian);

```

Man schreibt in JavaScript immer den Ausdruck **expr** direkt nach dem Schlüsselwort **return**, ohne einen Zeilenumbruch zwischen **return** und **expr**. Ist der Ausdruck **expr** abwesend, gibt der Befehl **undefined** zurück.

Hat eine Funktion keinen **return**-Befehl, gibt sie nach dem letzten ausgeführten Befehl **undefined** zurück.

Argumenten einer Funktion

Die Argumente einer Funktion werden evaluiert, bevor sie an die Funktion weitergegeben wird. Das obige Beispiel kann auch wie folgt umgeschrieben werden:

```

const degree = toDegree(Math.PI/4);

```

Default-Argument Ein Parameter kann mit einem Default-Argument festgelegt werden. Das Default-Argument wird verwendet, wenn diese Parameter bei dem Aufruf der Funktion nicht gesetzt wird. Die Funktion `format` gibt einen Text in eine Farbe aus.

```
const format = (text, color='red') => {
  return `

```

Das Verhalten der Funktion wird in den nächsten Code-Auszug veranschaulicht:

```
const text = "hello world";
const blueText = format(text, "blue");
// → <span style="color:blue;">hello world</span>
const stdText = format(text);
// → <span style="color:red;">hello world</span>
```

Default-Argument kann auch den Wert eines Argument übernehmen. Als Beispiel betrachten wir die Funktion `emphasize`. Sie wird wie folgt definiert:

```
const emphasize = (text, open='i', close=open) => {
  return open === close ?
    `<${open}>${text}</${close}>` :
    `${open}${text}${close}` ;
}
```

Man kann die Funktion mit unterschiedlichen Belegungen der Parameter aufrufen:

```
const littleImportantTask = emphasize("Vorlesung besuchen");
// → <i>Vorlesung besuchen</i>

const moreImportantTask = emphasize("Hausaufgabe machen", "b");
// → <b>Vorlesung besuchen</b>

const opening = `<span class="attention">`;
const closing = `</span>`;
const mostImportantTask = emphasize("JavaScript verstehen", opening, closing);
// → <span class="attention">JavaScript verstehen</span>
```

3.1.2. Name einer Funktion

In JavaScript muss eine Funktion nicht zwingend einen Name haben. Die Syntax der Deklaration fordert zwar, dass einen Name für die deklarierte Funktion angegeben wird, dennoch fordern andere Methoden zum Erstellen von Funktionen keinen Angabe von Funktionsname.

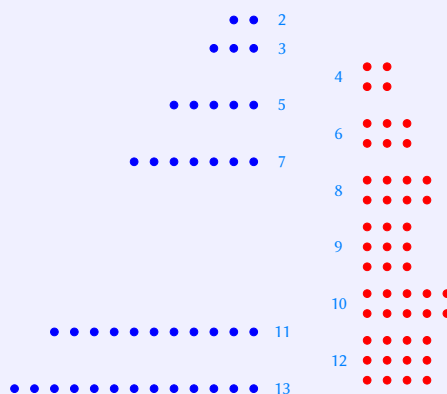
Programm 3.1.: Primzahl Test (prime-bigint-function)

```

import {terminal} from "./dfhi.js";
// 1
window.main = (...argv) => { // 2.1
  const n = BigInt(argv[0]);
  const primeTest = isPrime(n); // 3
  // 2.2
  if (primeTest) {
    terminal.printl(`${n} ist eine Primzahl`); // 4
  } else {
    terminal.printl(`${n} ist keine Primzahl`);
  }
  // 2.3
};
// 1
const isPrime = (n) => { // 3'
  // convert n to BigInt, sothat it can work with different datatypes
  const p = BigInt(n);
  if (p < 2n) {
    return false;
  }
  for(let i = 2n; i <= p/i; ++i) {
    if (p % i === 0n) {
      return false;
    }
  }
  return true;
}

```

Das Programm verwendet den Datentyp `BigInt`. Es kann mit Zahlen wie 22815088913 oder 3314192745739 umgehen, aber kann nicht mit 6161791591356884791277 adäquat umgehen. Die Zahlen im Kreis markieren die Stelle im Code, wo ein Identifier im Callstack (dargestellt in Abbildung 3.2) hinzugefügt wird.



3.1.3. Hoisting

Funktionen, die mit der Deklaration-Syntax wie in Abschnitt 3.1.1 deklariert sind, werden hochgezogen. Stößt der JavaScript-Interpreter auf einer Funktion-Deklaration, wird diese Funktion noch vor ihren Aufruf verarbeitet. Wir betrachten das Programm 3.1. Die Funktion `isPrime` im Programm 3.1 testet, ob eine Ganzzahl eine Primzahl ist. Sie wird nach deren Aufruf in der Funktion `main` deklariert. Dieses Verhalten nennt man *Function-Hoisting*.

Function-
Hoisting

Fingerübung 3.1: Die harmonische Reihe

Deklarieren Sie eine Funktion `harmonic`, die ein Argument `n` erhält und die harmonische Reihe (Programm 2.6 Seite 50) bis zum `n`-te Glied berechnet.

Fingerübung 3.2: Großkreis

Schreiben Sie das Program `great-circle` aus der Übung 2, Übungsblatt 2 neu. Trennen Sie Ihr Programm in folgenden Funktionen:

- Die Funktion `toRadian(angle)` konvertiert einen Winkelmaß aus Grad in Radian.
- Die Funktion `toDegree(angle)` konvertiert einen Winkelmaß aus Radian in Grad.
- Die Funktion `greatCircle(x1, y1, x2, y2)` berechnet den *Großkreis*-Distanz d in Seemeile von zwei Koordinaten (x_1, y_1) und (x_2, y_2) .

Die Formel für die Berechnung von Großkreis-Distanz ist wie folgt gegeben:

$$d = 60 \cdot \arccos(\sin(x_1) \cdot \sin(x_2) + \cos(x_1) \cdot \cos(x_2) \cdot \cos(y_1 - y_2))$$

Alle Winkel in der Gleichung sind in Grad anzugeben.

Die mathematische Grundlage zum Thema „Großkreis“ kann man in [6], Kapitel 1 finden.

3.1.4. Callstack

Wird eine Funktion aufgerufen, speichert der JavaScript Interpreter den Kontext des Programmes (zum Beispiel Wert der Variablen, ect.) als neu Element (via Push-Operation) in einem Stack und übergibt er die Kontrolle an der Funktion. Nachdem die Funktion den Return-Wert zurückgibt, löscht der JavaScript Interpreter den Zustand vor dem Aufruf der Funktion (via Pop-Operation) aus dem Stack. Der Stack wo die Zustände des Programmes gespeichert sind, ist als Callstack bezeichnet.

Die Abbildung 3.2 illustriert den Callstack während des Ablaufs des Programmes 3.1. Der Global-Kontext wird als `window` illustriert, denn es gibt im Browser immer die Variable `window` in dem Global Scope. Andere JavaScript-Umgebung könnte andere Global Scope haben, zum Beispiel in einer Nodejs-Umgebung hat man statt `window` die Variable `global`. Die Variable `n` in der Funktion `window.main` und die Variable `n` in der Funktion `isPrime` sind unterschied, obwohl sie den gleichen Name, hier sogar den gleichen Wert, haben. Die Variable `argv` referenziert ein Objekt von Typ Array, also kein Primitiven Typ. Solche Datentypen sind zusammengesetzte Datentypen. Objekten von zusammengesetzten Datentypen werden nicht in Callstack angelegt, sondern in Heap. Sie werden genauer im Kapitel ?? diskutiert.

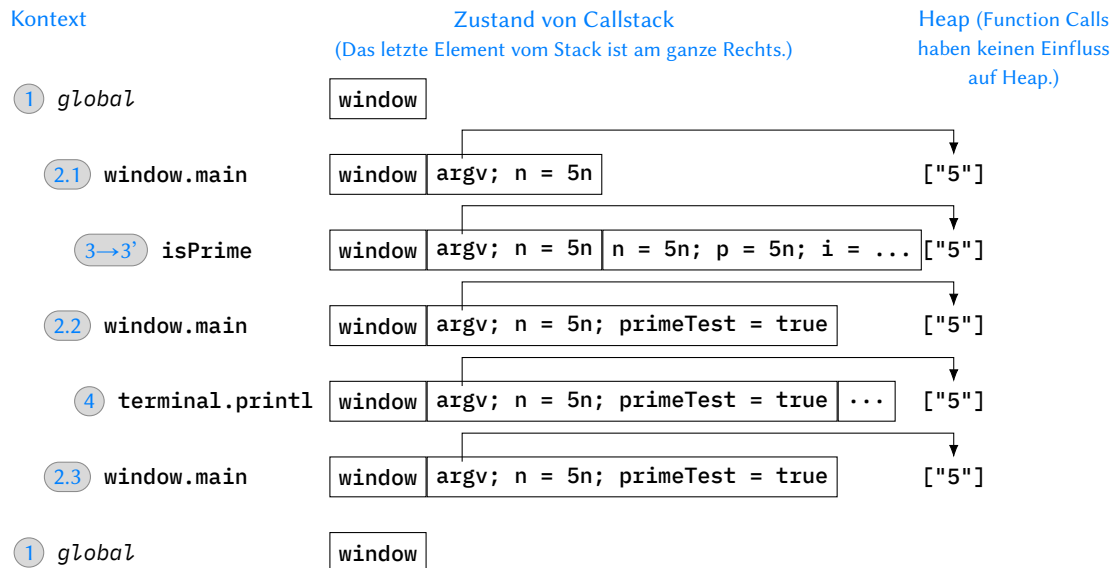


Abbildung 3.2.: Illustration von Callstack des Programmes 3.1

Die Speicherung der Kontext kostet Speicherplatz. Eine Kette von ungeschickte Funktion-Aufrufe in JavaScript kann zu einem `RangeError`-Fehler mit dem Fehlermeldung –je nach Browser– etwa „Maximum call stack size exceeded“ führen. Das Programm 3.2 provoziert absichtlich diesen Fehler. Die Funktionen `leftHand` und `rightHand` rufen sich gegenseitig auf, und füllen somit den Callstack voll.

Programm 3.2.: Drawing Hands (`drawing-hands`)

```
import {terminal} from "../dfhi.js";

window.main = function(...argv) {
  terminal.printl(leftHand() + " draws first!");
}

const leftHand = () => {
  return rightHand();
}

const rightHand = () => {
  return leftHand();
}
```

Der Callstack eines Programmes hat eine begrenzte Kapazität. Die Abbildung 3.5 ist eine künstlerische Illustration des Programmes.

3.1.5. Scope einer Variable in Funktion

Wir haben bereits im Abschnitt 2.4 den lexikalischen Scope einer Variable untersucht. Vor dem Standard ECMAScript 2015 kann man nur den Scope einer Variable durch Funktionen begrenzen. Variablen, die mit dem Schlüsselwort `var` deklariert sind, haben den Scope von der ganzen angehörigen Funktion. Die Funktion `isPrimeVarScope` wie auf der linken Seite ist eine Umformulierung der Funktion `isPrime` im Programm 3.1.

<pre>const isPrimeVarScope = (n) => { var p = BigInt(n); if (p < 2n) { return false; } for(var i = 2n; i <= p/i; ++i) { if (p % i === 0n) { return false; } } return true; }</pre>	<pre>const isPrimeVarScope = (n) => { var i = undefined; var p = BigInt(n); if (p < 2n) { return false; } for(i = 2n; i <= p/i; ++i) { if (p % i === 0n) { return false; } } return true; }</pre>
---	--

Die Variable `i` wird erst in der `for`-Schleife deklariert, jedoch hat sie den Scope in der ganzen Funktion, als ob die Variable `i` am Anfang der Funktion deklariert würde (rechten Seite). Man nennt dieses Verhalten *Variable Hoisting*. In einer JavaScript-Umgebung, der Standard ECMAScript 2015 und neuer unterstützt, soll man Variable mit `let` statt mit `var` deklarieren. So kann man den Scope einer Variable minimieren.

Variable
Hoisting

Fingerübung 3.3: Kapazität von Callstack

Ergänzen Sie das Programm 3.2 mit einer Global Variable `let stackCount = 0`, die inkrementiert und ausgegeben wird, jedes Mal wenn die Funktionen `leftHand` und `rightHand` aufgerufen werden.

Beobachten Sie die Variable `stackCount` in unterschiedlichen Browser.

3.1.6. Mathematische Funktionen

Eine *pure function* ist eine Funktion mit folgenden Eigenschaften:

pure func-
tion

1. Der Rückgabewert der Funktion hängt nur von den Argumenten der Funktion ab. Bei gleichen Argumenten gibt die Funktion das gleiche Ergebnis zurück.
2. Die Funktion hat keine Nebeneffekte. Das heißt, sie ändert keine nicht-lokale Variable, so wie die Argumenten und den Zustand der Umgebung.

Fingerübung 3.4

Warum sind diese Funktionen keine pure function?

1. Die Funktion `f1` wie folgt:

```
function f1(x) {
    return new Date();
}
```

2. Die Funktion `console.log`

3. Die Funktion `terminal.printl`

Wegen dieser Eigenschaften sind *pure functions* einfach zu benutzen und zu testen. Ein ideales Programm besteht nur aus *pure functions*. Man strebt deshalb an, Funktionen als *pure functions*, oder mindestens Funktionen ohne Nebeneffekt zu implementieren.

Die Implementierung einer mathematischer Funktion führt in der Regel zu einer (JavaScript) *pure function*. Denn mathematische Funktionen sind per-Definition *pure function*.

Newton-Verfahren zur Berechnung des Quadratwurzel

Das Bestimmen von x so dass $x = \sqrt{c}$ für gegebene $c \in \mathbb{R}^+$ ist die Lösung der Gleichung

$$x^2 - c = 0$$

Das bedeutet die Bestimmung von Nullstelle der Funktion $f(x) = x^2 - c$ im Bereich $x \in \mathbb{R}^+$. Wir entwickeln eine JavaScript Funktion `sqr(c)`, die den Wert von x annähernd berechnet. Die Funktion `sqr(c)` berechnet die Nullstelle von $f(x) = x^2 - c$. Nach Newton-Verfahren ist die Reihe

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)} \quad (\text{für } n = 0, 1, 2, \dots) \quad (3.1)$$

$$= x_n - \frac{x_n^2 - c}{2x_n} \quad (3.2)$$

konvergiert nach der Nullstelle der Funktion $f(x)$. Wir können die Reihe vereinfachen:

$$x_{n+1} = \frac{1}{2} \left(x_n + \frac{c}{x_n} \right) \quad (3.3)$$

Nun müssen wir die Abbruchbedingung der Berechnung von x_n für $n \in \mathbb{N}_0$ formulieren. Seien $g(x)$ eine Funktion und c in Wertbereich von $g(x)$. Gesucht ist einen Wert x_n , so dass $g(x_n) = c$. Die relative Fehlertoleranz $\varepsilon > 0$ von einem x_n , so dass $g(x_n) \approx c$ ist wie folgt definiert:

$$\frac{|g(x_n) - c|}{|c|} < \varepsilon \quad (3.4)$$

Sei $g(x) = x^2$ und sei ε eine festgelegte Konstant, ein Wert x_n gilt als eine approximierte Lösung der Gleichung

$$\begin{aligned} g(x) &= c \\ \Leftrightarrow x^2 &= c \end{aligned}$$

wenn die Bedingung

$$\frac{|x_n^2 - c|}{|c|} \leq \varepsilon \quad (3.5)$$

$$\Leftrightarrow |x_n^2 - c| \leq \varepsilon c \quad (3.6)$$

erfüllt.

Die Abbildung 3.3 stellt die geometrische Deutung des Newton-Verfahren dar. Die Funktion in Abbildung ist $f(x) = x^2 - 4.657$. Nach fünf Iterationen beginnend mit $x_0 = 4.657$ erreicht das Newton-Verfahren den Wert $x_4 = 2.1580088022069637$, welcher sehr nah an der Nullstelle von $f(x)$ liegt. Die in grauen gezeichneten Tangente $t_n(x) = f'(x_n)(x - x_n) + f(x_n)$ schneidet die Gerade $y = 0$ in x_{n+1} . Im günstigen Fall liegt x_{n+1} näher an der Nullstelle als x_n .

Aus der Gleichungen 3.3 und 3.6 können wir die Funktion `sqrt(c)` wie im Programm 3.3 implementieren. Man setzt in der Regel $x_0 = c$.

Fingerübung 3.5: Vergleich Newton-Verfahren

Erweitern Sie das Programm 3.3 mit der Funktion `window.main`, sodass Sie das Ergebnis von der Funktion `sqrt` mit dem Ergebnis von `Math.sqrt` vergleichen können.

Fingerübung 3.6: * Implementierung der Newton-Verfahren

Gegeben sei die Funktion $f(x) = x^3 - 3.2x + 1.23$. Schreiben Sie ein Programm, das die Nullstellen von $f(x)$ annähernd berechnen. Benutzen Sie das Newton-Verfahren in Ihrem Programm. Das ist, das Programm berechnet die Reihe 3.1 für $f(x)$.

Die Nullstellen von $f(x)$ sind $\{-1.956684533, 0.4051588507, 1.551525682\}$. Die erste Ableitung der Funktion $f(x)$ ist $f'(x) = 3x^2 - 3.2$.

1. Formulieren Sie zuerst die approximative Formel 3.1 für die Funktion $f(x) = x^3 - 3.2x + 1.23$.
2. Formulieren Sie die Abbruchbedingung der Berechnung von x_n , sodass der Wert $f(x_n)$ annähernd gleich 0 ist.

Approximation der natürlichen Exponentialfunktion

Die Funktion $f(x) = e^x$ für $x \in \mathbb{R}$, wobei $e \approx 2.71828$ die Euler-zahl ist, kann man mit der Taylor-Reihe annähernd berechnen:

$$e^x = \sum_{i=0}^{\infty} \frac{x^i}{i!} = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots$$

$$\approx \sum_{i=0}^n \frac{x^i}{i!} = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots + \frac{x^n}{n!}$$

Der erste Ansatz ist, die Nenner und Zähler von jedem Term $x^i/i!$ zu rechnen:

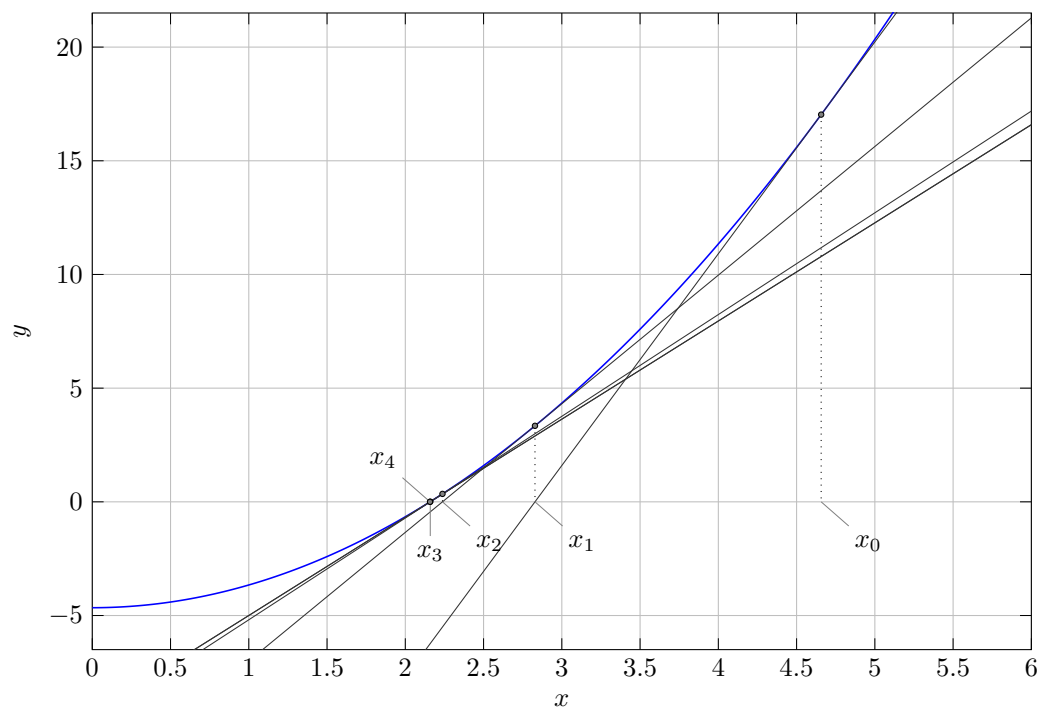


Abbildung 3.3.: Newton-Verfahren zur Berechnung von $c = \sqrt{4.657}$; $f(x) = x^2 - 4.657$

```

    Programm 3.3.: Funktion sqrt nach Newton-Verfahren (newton-method-sqrt)
const sqrt = (c) {
    const EPSILON = 1E-16;
    let xn = c;
    let q = c/xn; // auxiliary variable to avoid multiple division c/xn
    //  $\left| \frac{x_n^2 - c}{c} \right| > \varepsilon \Leftrightarrow \left| x_n - \frac{c}{x_n} \right| > \varepsilon \cdot \frac{c}{x_n}$ 
    while(Math.abs(xn - q) > EPSILON*q ) {
        xn = (q + xn) / 2;
        q = c/xn;
    }
    return xn;
}

```

Die Funktion verwendet die relative Toleranz als Abbruchbedingung. Statt die Division $\frac{c}{x_n}$ mehrfach auszuführen, wird eine Hilf-Variable q benutzt, um den Quotient temporär zu speichern.

```

let sum = 1.0;
const NUM_OF_TERM = 10;
for(let n = 1; n <= NUM_OF_TERM; ++n){
    let num = 1.0,
        den = 1.0;
    for (let i = 1; i <= n; ++i) {
        num *= x;
        den *= i;
    }
    sum += num / den;
}

```

Formen wir die Gleichung um, können wir die Nenner und Zähler zusammen in eine Schleife erfassen:

$$e^x = 1 + x + \left(\frac{x}{1} \cdot \frac{x}{2}\right) + \left(\frac{x^2}{2!} \cdot \frac{x}{3}\right) + \cdots + \frac{x^{n-1}}{(n-1)!} \cdot \frac{x}{n}$$

Der entsprechend Code ist

```

let sum = 1.0;
const NUM_OF_TERM = 10;
for(let n = 1; n <= NUM_OF_TERM; ++n){
    let term = 1.0;
    for (let i = 1; i <= n; ++i) {
        term *= x/i;
    }
    sum += term;
}

```

Hier haben wir die Anzahl der Iterationen (NUM_OF_TERM) willkürlich festgelegt. Die Terme mit höhere Werte von n haben dennoch weniger Einfluss auf das Endergebnis ($\lim_{n \rightarrow +\infty} \frac{x}{n} = 0$). Deshalb wenn der Wert von sum sich nicht mehr ändern, kann der äußere Schleife beendet. Wir können die Fortführen-Bedingung der Schleife umformulieren:

```

let sum = 0.0,
    term = 1.0;
for(let n = 1; sum !== sum + term; ++n) {
    sum += term;
    term = 1.0;
    for (let i = 1; i <= n; ++i) {
        term *= x/i;
    }
}

```

Dieser Code ist effizienter als der vorherige Code, dennoch werden die Termen neu berechnet, ohne das Ergebnis der Berechnung des vorherigen Terms wieder zu verwenden:

$$t_i = \frac{x^i}{i!} = \frac{x^{i-1}}{(i-1)!} \cdot \frac{x}{i} = t_{i-1} \cdot \frac{x}{i}$$

$$\Rightarrow e^x = 1 + \frac{x}{1} + t_1 \cdot \frac{x}{2} + t_2 \cdot \frac{x}{3} + \dots + t_{n-1} \cdot \frac{x}{n}$$

Die entsprechende Schleife lautet dann:

```
let sum = 0.0,
    term = 1.0;
for(let i = 1; sum !== sum+term; ++i) {
    sum += term;
    term *= x/i;
}
```

Fingerübung 3.7

Sie können die vier Berechnungsvarianten der Funktion $f(x) = e^x$ in vier Funktionen erfassen und die Genauigkeit jeweiligen Varianten gegenüber der Built-In-Funktion `Math.exp` überprüfen.

3.2. Bibliothek

Alle Programme, die wir bisher geschrieben haben, bestehen aus nur eine JavaScript-Datei. Diese Beschränkung wäre für große Programme sehr lästig und unnötig. Bei dem alten JavaScript Standard, also Version 1.5 und früher, kann man JavaScript Code in mehrere Dateien verteilen, und mit dem Tag `<script>` sie in einer HTML-Datei zusammen laden zu lassen. Dieser Ansatz exponiert dennoch alle Identifier aus dem Datei-Scope im Global-Scope. Die Abhängigkeit zwischen JavaScript-Dateien ist nicht explizit in JavaScript Code zu erkennen, sondern nur in HTML-Datei sichtbar. Er gilt deshalb als veraltet. Man sollte diesen Ansatz vermeiden.

Modernerer JavaScript Standard bietet die Möglichkeit, die Objekten in Module zu kapseln, um ausschließlich ausgewählte Objekten in andere Modulen sichtbar zu lassen. Man kann außerdem innerhalb einer JavaScript Datei die notwendigen Module importieren. Der neue Ansatz führt dazu, dass die Abhängigkeit von Modulen explizit direkt im JavaScript-Code erkannt wird. Es erhöht die Lesbarkeit und Wartbarkeit vom Code. Wir haben bereits diesen Ansatz benutzen. In Programme, in den das Objekt `terminal` aus dem Modul `'dfhi'` benötigt ist, wird dieses Modul mit dem Befehl

```
import {terminal} from './dfhi.js';
```

importiert. In diesem Abschnitt diskutieren wir über den Aufbau des eigenen Moduls, die Nutzung von Standard Bibliotheken und Bibliotheken aus Third-Party Anbieter, sowie den Konzept „Design by Contract“.

importiert

3.2.1. Aufbau eines JavaScript-Moduls

Werden Funktionen und Objekten häufig genutzt, werden sie in einem Modul gekapselt und exportiert. So kann man sie mehrfach in unterschiedlichen Stelle verwenden. Als Beispiel betrachten wir die ersten Codezeilen in der Datei `dfhi.js`. Sie sind wie folgt:

```
import {DomOutput} from "./DomIO.js";
// "private" variable, only for this file
const OUTPUT_ID = 'text-output';
const INPUT_ID = 'argv';
// Create Output Terminal
export const terminal = new DomOutput(OUTPUT_ID);
```

Die Datei `dfhi.js` verwendet das Objekt `DomOutput` aus dem Modul `./DomIO.js` in den gleichen Ordner, also importiert sie das Objekt `DomOutput`. Der `from`-Klausel des Befehls `import` erwartet einen String, von dem er zu einem URL auslösen kann. Der String kann ein absolute Path, absolute URL oder relative Path repräsentieren.

Die Datei `dfhi.js` stellt das Objekt `terminal` für andere (ihr unbekannte) Dateien zur Verfügung. Also exportiert sie das Objekt `terminal`, indem das Objekt `terminal` zusätzlich mit dem Schlüsselwort `export` deklariert wird. Die Konstante `OUTPUT_ID` ist nur relevant für die Funktionen in der Datei `dfhi.js`, und ist absichtlich nicht für andere JavaScript Modules verfügbar. Sie wird deshalb ohne Schlüsselwort `export` deklariert. Da die Konstante `OUTPUT_ID` nur in der Datei `dfhi.js` sichtbar ist, sagt man, dass sie einen Datei-Scope hat. Das Gleiche gilt auch für die Konstante `INPUT_ID`.

Durch den `import`-Befehl kann eine JavaScript-Datei explizit deklarieren, dass sie von der Datei `DomIO.js` abhängig ist. Durch den `export`-Befehl kann eine JavaScript-Datei als Modul gezielt deklarieren, welche Objekten in der Datei für andere Datei verfügbar sind, statt alle Objekten in der Global Scope zu exportiert.

Historische Hinweis: Vor Standard ECMAScript 2015 werden JavaScript-Dateien via `<script>`-Tag (ohne Attribute `type="module"`) in einem HTML gebunden:

```
<script src="js/xyz.js"></script>
<script src="abc.js"></script>
<!-- weitere Skripten -->
```

Alle Variablen und Funktionen in Datei-Scope von jeder Datei werden in Global Scope exportiert. Bei großen Projekten führt dieser Ansatz zum Namenskonflikt, denn ein Name kann in mehr als eine Datei verwendet werden. Die Datei, die später geladen wird, kann die Deklarationen aus der vorher geladenen Dateien überschreiben. Dieser Ansatz gilt als fehleranfällig und veraltet. Sie sollen nicht mehr verwendet werden.

Weitere syntaktische Varianten der `import`- und `export`-Befehlen kann man aus [9, 8] entgegennehmen. Wir benutzen in der Vorlesung die einfachste Form von `import`. Bei dem `export`-Befehl benutzen wir den „Exporting individual features“ Variante.

Fingerübung 3.8: Modul `study.js`

- Schreiben Sie eine Funktion `calculateHomeworkTime` im Modul `study.js`. Die Funktion soll zwei Argumenten `ects` und `lecturePerWeek` entgegennehmen, dann die Zeitstunden pro Woche für die Vor- und Nach-Arbeiten einer Vorlesung berechnen. Beachten Sie die Vorgabe des Zeitaufwands:
 - `ects` Anzahl der ECTS einer Vorlesung z.B 5 für JavaScript.
 - `lecturePerWeek` Anzahl der Doppelstunden (Abk. DS, jeweils 90 Minuten) pro Woche einer Vorlesung, z.B 2 DS für JavaScript.
 - Ein Semester hat in der Regel 15 Vorlesungswochen.
 - Ein ECTS entspricht den Aufwand von 30 Zeitstunden.

- Eine Zeitstunden entspricht 60 Minuten.
2. Erstellen Sie ein Programm `semester.js`, das die Funktion `calculateHomeworkTime` verwendet, um die gesamte Vor- und Nach-Arbeiten Zeit für alle im dritten Semester des Studiengang Informatik zu besuchenden Vorlesungen zu berechnen. Das Ergebnis soll in einer übersichtliche Tabelle dargestellt werden.
 3. Schreiben Sie eine Funktion `printHomeworkTime` im Modul `study.js`, um diese Tabellen zu erstellen. Sie können weitere Hilfsfunktionen schreiben. Überlegen Sie, welche Funktionen im Module `study.js` man exportieren kann.
Die Signatur einer `export`-ierter Funktion kann etwa so aussehen.

```
export calculateHomeworkTime = (ects, lecturePerWeek) => {
  // ....
}
```

3.2.2. Application Programming Interface

Bei der Nutzung einer Bibliothek vereinbaren die Bibliothek-Anwender mit dem Bibliothek-Entwickler die Schnittstelle der Bibliothek. Solche Schnittstelle nennt man Application Programming Interface, kurz API. Die Software, die eine Bibliothek verwendet, ist ein Client der Bibliothek. Die Schnittstelle kann man –als Metapher– wie einen Vertrag zwischen Bibliothek-Client und der Bibliothek vorstellen. Sie spezifiziert die Bibliothek. Spezifikationen einer Bibliothek sind unter anderem

- Welche Funktionalität stellt die Bibliothek zur Verfügung (z.B in JavaScript ist: Welche Funktionen und Objekten sind exportiert).
- Welche Bedingungen müssen von Client erfüllt werden, wenn er die Bibliotheken nutzt (z.B Datentypen und Wertbereich der Argumenten einer Funktion).
- Was der Client bei der Nutzung von der Bibliothek erwarten (z.B Nebeneffekt, Rückgabewert einer Funktion) kann.

Das Standard ECMAScript 2015 standardisiert eine umfangreiche API, die die Browser-Hersteller implementieren. Wir haben bereits zum Beispiel die API vom Objekt `Math` verwendet, und werden im Lauf des Kurses auch weitere API verwenden.

Fingerübung 3.9: Das Modul `chronos`

In dieser Übung erstellen Sie ein Modul `chronos` (Datei `chronos.js`) mit folgenden Funktionen:

- Die Funktion `isLeapYear(year)` testet, ob ein gregorianisches Jahr ein Schaltjahr ist (Übung 2.3).
- Die Funktion `countDaysOfMonth(month, year)` gibt die Anzahl der Tage in einem Monat eines Jahres zurück.
- Die Funktion `weekDayOfDate(day, month, year)` gibt den Wochentag in Zahl eines gregorianisches Datum zurück. Die Wochentage sind von 0 (für Sonntag) bis 6 (für Samstag) durchnummeriert. Notwendige Informationen finden Sie in der Übungsblatt 1.

Die Funktionen im Modul `chronos` sind naturgemäß reine mathematische Funktionen. Implementieren Sie sie so, dass sie *pure function* sind.

Erstellen Sie eine Datei `chronos-test.html` und eine Datei `chronos-test.js` um das Modul `chronos` automatisch zu testen. Alle Funktionen im Modul `chronos` sollen im Test mehrfach mit unterschiedlichen Argumenten aufgerufen werden. Die Datei `chronos-test.js` ist nur vom Modul `chronos` abhängig. Die Datei `chronos-test.html` lädt keine weitere Datei, außer nur die Datei `chronos-test.js`. Die Testergebnisse sollen mit Hilfe der Funktion `console.log` bzw. `console.error` in der Browser-Console ausgegeben werden.

3.3. DOM

Die DOM-API [17] wird von der W3C-Organisation spezifiziert. Die Browser-Hersteller implementieren diese API. Ein JavaScript-Entwickler benutzt diese API in ihrem Code. Wenn ein Browser deklariert, dass er die APIs implementiert, dennoch nicht an der Spezifikation hält, dann führt es zum Problem beim Code vom JavaScript-Entwickler. Man kann metaphorisch dieses Verhalten als „Vertrag nicht erfüllt“ bezeichnen.

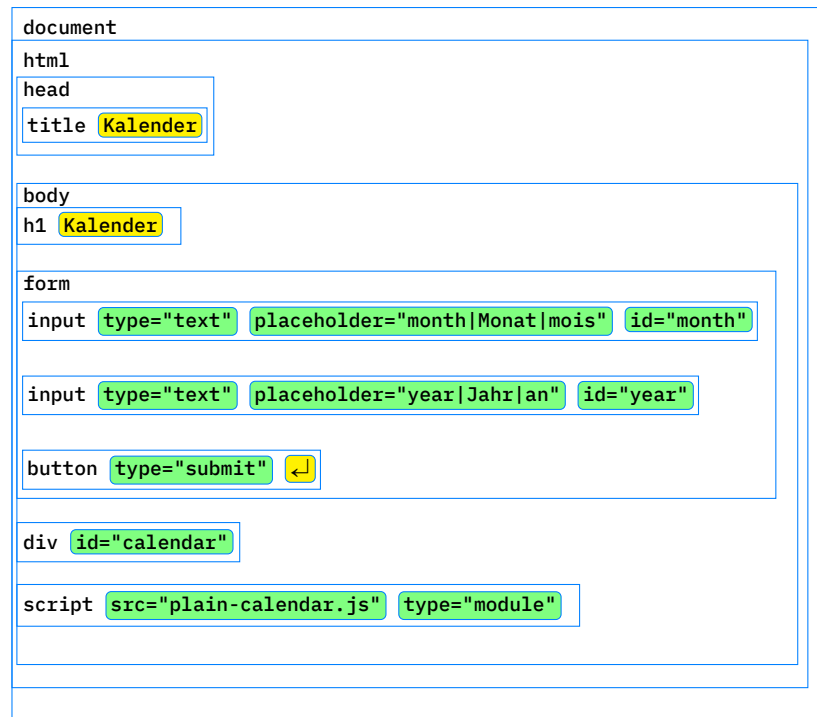
3.3.1. Struktur eines DOM

Ein HTML-Datei wird vom Browser zu einem Baum aufgespannt. Als Beispiel betrachten wir die Datei `plain-calendar.html` wie im Listing 3.4. Die Abbildung stellt schematisch den entsprechenden aufgespannten Baum dar.

Programm 3.4.: Die Datei (`plain-calendar.html`)

```
<html>
  <head>
    <title>Kalender</title>
  </head>
  <body>
    <h1>Kalender</h1>
    <form>
      <input type="text" placeholder="month|Monat|mois" id="month" />
      <input type="text" placeholder="year|Jahr|an" id="year" />
      <button type="submit"><|</button>
    </form>
    <div id="calendar"></div>
    <script src="plain-calendar.js" type="module"></script>
  </body>
</html>
```

Die HTML-Datei wird vom Browser als Baum-Datenstruktur aufgespannt. Die Abbildung 3.4 zeigt schematisch den Baum des obigen HTML-Code. Die Text-Knoten mit nur White-Spaces-Zeichen werden in der Abbildung nicht dargestellt.

Abbildung 3.4.: Baum-Darstellung der Datei `plain-calendar.html`.

Der DOM-Baum hat unterschiedlichen Typen von Knoten. Eine vollständige Liste der Typen von DOM-Knoten kann man unter [15] finden. Die Typen der Knoten im Baum der Datei `plain-calendar.html` sind wie folgt:

`Node.DOCUMENT_NODE` <https://developer.mozilla.org/en-US/docs/Web/API/Document>

`Node.ELEMENT_NODE` <https://developer.mozilla.org/en-US/docs/Web/API/Element>

`Node.ATTRIBUTE_NODE` <https://developer.mozilla.org/en-US/docs/Web/API/Attr>

`Node.TEXT_NODE` <https://developer.mozilla.org/en-US/docs/Web/API/Text>

Fingerübung 3.10: DOM Struktur

- Finden Sie die (ganzzahligen) Werten der Typen des Knoten! Z.B der Type `Node.ELEMENT_NODE` ist die Zahl 9.
- Was ist ein Knoten vom Typ `Node.DOCUMENT_TYPE_NODE`. Wie kann man die Datei `plain-calendar.html` anpassen, damit deren aufgespannte Baum einen Knoten vom Typ `Node.DOCUMENT_TYPE_NODE` besitzt.
- Ergänzen Sie den Tag `html` mit dem Attribute `lang="de"`. Wie dieser Knoten schematisch im Baum dargestellt?
- Kapseln Sie die Elementen `h1`, `form`, `div` in einem `div`-Element. Zeichnen Sie den Baum neu.

3.3.2. Operationen auf DOM

Ein DOM-Baum ist veränderbar. Die Struktur des DOM-Baum kann zur Laufzeit mit JavaScript manipuliert werden. Die DOM-API unterstützt die gängige Operationen auf einem Baum-Datenstruktur. Darüber hinaus lässt die API bestimmte Baum-Knoten auswählen, ohne den ganzen Baum explizit zu traversieren.

Knoten auswählen

Fingerübung 3.11

1. Untersuchen Sie die Methode `document.getElementById`. Überzeugen Sie sich, dass die Methode nur ein Element zurück gibt, wenn das DOM das Element mit gewünschtem `id`-Attribut enthält. Untersuchen Sie das Verhalten der Methode, wenn das DOM mehr als 1 Elementen mit einem gleichen Attribut `id` hat.
Überzeugen Sie Ihre Aussage, indem Sie Beispiel-Programme schreiben.
2. Die Methoden `document.querySelector` und `document.querySelectorAll` sind ab Standard ECMAScript 2015 verfügbar. Untersuchen Sie die beide Methoden. Mit welchem Argument kann man die Methode `document.querySelector` aufrufen, um ein Element mit gewünschtem `id`-Attribut auszuwählen?
3. Sie möchten aus dem DOM alle Elementen mit dem Tag-Namen `pre` auswählen, wie lautet das entsprechende Argument für den Aufruf der Methode `document.querySelectorAll`? Gibt es andere Alternativen?

Fingerübung 3.12

Erstellen Sie zehn gültigen Argumenten für den Aufruf der Methode `document.querySelectorAll`. Überzeugen Sie sich, dass die Argumenten nur gewünschte Elementen auswählen, indem Sie Beispielcode schreiben.

Eigenschaften von Knoten manipulieren

Fingerübung 3.13

Gegeben ist das Element `input` wie folgt:

```
<input type= "text" id="month" >
```

Wie kann man den eingegebene Wert des Elements ablesen und überschreiben?

Fingerübung 3.14

- Wie fügt man eine CSS-Klasse in einem Element hinzu?
- Wie entfernt man eine CSS-Klasse aus einem Element?

Knoten hinzufügen oder entfernen

Fingerübung 3.15

Was ist der Unterschied zwischen die Property `innerHTML` und `innerText` eines Elements? Überzeugen Sie sich anhand Beispielcode.

Fingerübung 3.16

Erklären Sie die Funktionalität des HTML-Tag `<template>` durch ein Beispiel.

Fingerübung 3.17

- Wie kann man ein Element mit gegebenen `id`-Attribut aus dem DOM entfernen?
- Wie kann man Elementen aus einer `NodeList` entfernen? Demonstrieren Sie zwei Fehlerquellen bei der Entfernung von Elementen aus einer `NodeList`.

DOM-Baum traversieren

(Thema im Tutorium)

3.4. Date

Fingerübung 3.18

Untersuchen Sie das Objekt `Date` aus der MDN-Webseite. Schreiben Sie das Programm `Calendar` aus der letzten Vorlesung um. Das Programm soll soweit wie möglich die Berechnung von Datum mit dem Object `Date` erledigen.

Welche Alternative API zum Objekt `Date` bieten moderne Browser an? Warum soll man in neue Projekte das Objekt `Date` nicht mehr benutzen?

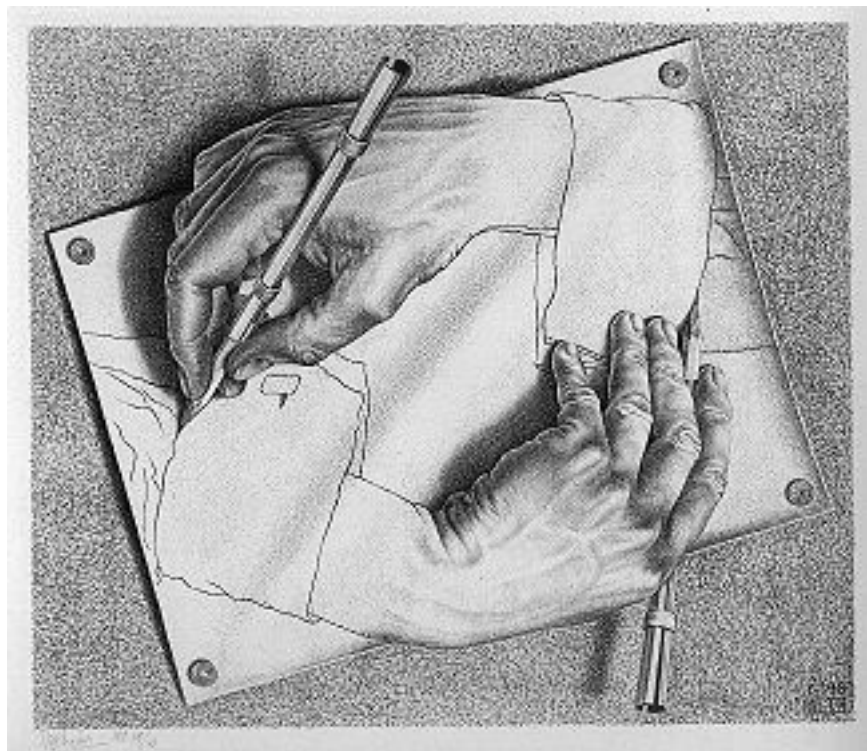


Abbildung 3.5.: Drawing Hands [3]

A. Ganzen Zahlen

Darstellung von ganzen Zahlen

Bestimmten Operationen –insbesonderes Bitweise Operationen– werden nur auf 32-Bit Integer durchgeführt. Wir untersuchen in diesem Abschnitt die Zweierkomplement Darstellung von 32 Bit ganzen Zahlen.

In Zweierkomplement werden die Zahlen in bestimmten Anzahl von binäre Stellen (Bits) dargestellt.

Ein n -Bits Zweierkomplement System kann insgesamt 2^n (n Stelle, jeweils 2 Werten) Zahlen darstellen. Sie sind:

- die Zahl 0;
- $2^{n/2} - 1$ positive Zahlen von 1 bis $2^{n/2} - 1$ und
- $2^{n/2}$ negativen Zahlen von -1 bis $-2^{n/2}$.

Die Zahl 0 hat alle binären Stelle den Wert 0. Die positiven Zahlen haben das erste Bit immer die Ziffer 0. Die weiter $n - 1$ Bits stellt den Betrag der Zahl dar. Die negativen Zahlen $-x$ für $x > 0$ werden als Komplement (Notation: $\sim$) von $x - 1$ berechnet.

Als Beispiel betrachten wir die Darstellung von ganzen Zahlen in 4 Bits. Die Anzahl der darstellbaren Zahlen ist $2^4 = 16$ (4 Stellen, jeweils 2 Werten, 0 und 1, annehmen kann). Die darstellbaren ganzen Zahlen sind die Zahlen von $-2^{4-1} = -8$ bis $2^{4-1} - 1 = 7$.

Die Zahlen von 0 bis 7 werden wie folgt dargestellt:

0 = 0B0000	4 = 0B0100
1 = 0B0001	5 = 0B0101
2 = 0B0010	6 = 0B0110
3 = 0B0011	7 = 0B0111

Die Zahlen von -8 bis -1 werden wie folgt dargestellt:

-8 = $\sim(8-1) = \sim(0B0111) = 0B1000$	-4 = $\sim(4-1) = \sim(0B0011) = 0B1100$
-7 = $\sim(7-1) = \sim(0B0110) = 0B1001$	-3 = $\sim(3-1) = \sim(0B0010) = 0B1101$
-6 = $\sim(6-1) = \sim(0B0101) = 0B1010$	-2 = $\sim(2-1) = \sim(0B0001) = 0B1110$
-5 = $\sim(5-1) = \sim(0B0100) = 0B1011$	-1 = $\sim(1-1) = \sim(0B0000) = 0B1111$

Ein 32-Bit Zweierkomplement System kann die ganzen Zahlen von -2^{31} bis $2^{31} - 1$ darstellen.

Bitweise Operatoren

Die Bitweisen Operatoren werden wie folgt definiert:

$x \ll y$ (left shift) verschiebt x um y Bits nach links, von rechts werden 0 hereingeschoben.

x >> y (right shift) verschiebt x um y Bits nach rechts, herausgeschobene Bits werden verworfen. Vorzeichen vom y wird beibehalten.

x >>> y (zero-fill right shift) verschiebt x um y Bits nach rechts, herausgeschobene Bits werden verworfen und 0s werden von links herein verschoben.

x & y (bitwise and) jeder Bit vom Rückgabewert ist 1 wenn beide Bits in x und y an dem entsprechenden Stelle 1 sind, sonst 0.

x | y (bitwise or) jeder Bit vom Rückgabewert ist 1 wenn die Bits von x oder von y oder beides an der entsprechenden Stelle 1 sind.

x ^ y (bitwise exclusive or) jeder Bit vom Rückgabewert ist 1 wenn die Bits von x und y an der entsprechenden unterschiedlich von einander sind.

~ x gibt das Komplement von x zurück.

Sind x oder y nicht 32-Bit ganzen Zahlen, werden sie zuerst in 32-Bit ganzen Zahlen konvertiert. Die Konvertierung kann man in [2, Abschnitt 7.1.5, 7.1.6] weiter lesen. Da die ganzen Zahlen in 32-Bit dargestellt werden, sind die Bit Shift Operationen nur sinnvoll bei $y < 32$. Ist y größer als 31, wird das Rest-Division von y durch 32 als zweite Operand verwendet.

Wir betrachten folgenden Beispielen:

Darstellung der Bit-Muster einer Zahl Zahlen in JavaScript kann man mit der Method

`(math expression).toString([radix])`

in String mit angegebenen Basis (das optionale Argument *radix*) konvertieren. Im folgenden Code zeigt die Bit Muster der Zahl 12 auf dem Browser:

```
let x = 12;
let bitPattern = x.toString(2);
terminal.printl(bitPattern);
// -> 1100
```

Die Method `toString` bewahrt das Vorzeichen von *math expression*. Deshalb wird negative Zahlen in „negative“ Vorzeichen deren Betrag konvertiert, was wir nicht immer haben wollen. Damit JavaScript die Zahl als *unsigned* 32-Bit ganze Zahl interpretiert, kann man folgendes Idiom benutzen:

unsigned
Integer

`(math expression >>> 0).toString([radix])`

In den nächsten Abschnitten wird das Idiom durch verschiedenen konkreten Code illustriert. Die folgenden Code zeigt die Bit Muster von -12 als 32-Bit ganze Zahl auf dem Browser:

```
let x = -12
let bitPattern = (x >>> 0).toString(2)
terminal.printl(bitPattern)
// -> 1111 1111 1111 1111 1111 1111 1111 0100 Das ist gleich wie ~(12-1) == ~11
// ~0B0000 0000 0000 0000 0000 0000 0000 1011 == ~(12-1)
```

Wir benutzen dieses Idiom ohne weitere Erklärung in den nächsten Abschnitt um das Ergebnis zu veranschaulichen. Das Leerzeichen zwischen Ziffer wird nicht von `toString()` erzeugt. Sie werden nachträglich hinzugefügt, damit das Kommentar leserlicher wird.

Left Shift

```

const x = 2**31 - 1; // 0111 1111 1111 1111 1111 1111 1111 1111 (2147483647)
let s = x << 1;      // 1111 1111 1111 1111 1111 1111 1111 1110 (-2)
                    //                                     Bit Overflow + to -
s = x << 31;         // 1000 0000 0000 0000 0000 0000 0000 0000 (-2147483648)
                    //                                     max. of shifted bits
s = x << 32          // 0111 1111 1111 1111 1111 1111 1111 1111 (as if << 0)

```

Right Shift

```

const x = (-2)**31; // 1000 0000 0000 0000 0000 0000 0000 0000 (-2147483648)
let s = x >> 1;     // 1100 0000 0000 0000 0000 0000 0000 0000 (-1073741824)
                    //                                     Preserve Sign
s = x >>> 1;        // 0100 0000 0000 0000 0000 0000 0000 0000 ( 1073741824)
                    //                                     Override Sign - to +
s = x >> 31;        // 1111 1111 1111 1111 1111 1111 1111 1111 (-1)
s = x >>> 31;       // 0000 0000 0000 0000 0000 0000 0000 0001 (1)

```

Bitweise logische Operationen

```

const x = 0xFF00;    // 1111 1111 0000 0000
const y = 0xAAAA;    // 1010 1010 1010 1010
let a = x & y;        // 1010 1010 0000 0000 (AA00)
let o = x | y;        // 1111 1111 1010 1010 (FFAA)
let xo = x ^ y;       // 0101 0101 1010 1010 (55AA)
let n = ~x;           // 1111 1111 1111 1111 0000 0000 1111 1111 (FFFF 00FF)

```

Für kleine ganzen Zahlen kann man die Exponenten von 2 mit Left Bit-Shift Operation (<<) berechnen: $a \ll 1$ ist gleich wie $a * 2$. Die Zuweisung $a = a \ll e$; kann auch als $a \ll= e$; gekürzt werden. Der Befehl `power *= 2`; kann somit als `power <<= 1`; umgeschrieben werden. Meistern Browser implementieren die Bit-Shift nur für ganzen Zahl bis zum 32 Bit groß.

Fingerübung A.1: Anwendung von Left-Bit-Shift Operation

- Schreiben Sie das Program 2.3 um, verwenden Sie dazu den Operator << bzw. den Operator <<=.
- Vergleichen Sie die Ausgaben von zwei Implementierungen. Erklären Sie den Unterschied.
- Welche Operation kann man noch in JavaScript benutzen, um Exponent zu rechnen?
- Was ist `Number.MAX_SAFE_INTEGER`? Kann das Ergebnis der Operation << diesen Wert erreichen?

B. Git Kompakt-Kurs

B.1. Installation

Installieren Sie folgende Software auf Ihrem Rechner:

- **Git**
- Ein Program um Dateien zu vergleichen. Standardmäßig wird **Vimdiff** verwendet, man kann aber andere Programme wie **KDiff3** verwendet. In diesem Tutorial wird **KDiff3** verwendet.

B.1.1. Für Linux

Für Debian basierte Systeme sind die Installation denkbar einfache. Man kann die Programme als Software Paket via **apt** installieren:

```
sudo apt install git kdiff3
```

B.1.2. Für MacOS

(Selbstversorgt)

B.1.3. Für Windows

Git In Windows kann man Git und einem minimalem Shell über das Software-Paket „Git for Windows“ installieren. Der Link zum „Git for Windows“ ist <https://gitforwindows.org/>. Die Installation ist auch denkbar einfach.

KDiff3 Es gibt einen Installer von **KDiff3** für Windows. Man kann die Installer-Datei vom Link <http://kdiff3.sourceforge.net/> herunterladen.

Nach der Installation müssen Sie die Umgebungsvariable **Path** im Windows erweitern, sodass die Variable auch den Pfad zu dem **KDiff3** enthalten.

HTW https Zertifikat Die Konfiguration-Datei für **git** unter Windows ist die Datei

`C:\Program Files\Git\mingw64\etc\gitconfig`

Ändern Sie den originalen Konfiguration-Parameter **sslBackend** in der Konfiguration-Datei von **sslBackend = openssl** zu **sslBackend = schannel**, wenn **git clone** Fehler schlägt.

B.2. Grundbegriffe

[Repository](#) *Platz für Ihr Notizen*

[Commit und Push](#) *Platz für Ihr Notizen*

[Tag](#) *Platz für Ihr Notizen*

[Branch und Merge](#) *Platz für Ihr Notizen*

B.3. Benutzung von Git

[Klone ein Repository – Clone](#) *Platz für Ihr Notizen*

[Die Rolle des Git Index beim Vorbereiten von Revisionen – Add](#) *Platz für Ihr Notizen*

[Projekte und Dateien in das Git Repository übernehmen – Commit](#) *Platz für Ihr Notizen*

[Änderungshistorie untersuchen – Log](#) *Platz für Ihr Notizen*

Erstellung von Entwicklungszweigen (Branches) und deren Reintegration (Merge) Platz für Ihr Notizen

Lösen von Konflikten Platz für Ihr Notizen

Erzeugen von Versionen – Tags Platz für Ihr Notizen

Bonus: Erstellung eines Repositories (Init) Platz für Ihr Notizen

B.4. Die Kommunikation zwischen Repositories

Klonen eines bestehenden Repositorys (Clone) *Platz für Ihr Notizen*

Übernehmen von Änderungen (Fetch und Pull) *Platz für Ihr Notizen*

Änderungen in ein Repository schieben (Push) *Platz für Ihr Notizen*

Bonus: Umsetzung eines zentralen Repositorys (Bare Repository) *Platz für Ihr Notizen*

B.5. Bonus: vimdiff

dp	diffput: (p => put) puts changes under the cursor into the other file making them identical (thus removing the diff).
do	diffget: (o => obtain). The change under the cursor is replaced by the content of the other file making them identical.
]c	Jump to the next diff
[c	Jump to the previous diff
:diffg RE	get from REMOTE
:diffg BA	get from BASE
:diffg LO	get from LOCAL
:x	as same as :wq

C. Node und NPM

C.1. NPM Package Management

Bisher werden nur Funktionen aus der API vom Standard ECMAScript 2015 verwendet. Bibliotheken von Third-Party werden noch nicht in unserem Code eingebunden. In diesem Abschnitt werden wir das Software-Paket **NPM** durch Schritt-für-Schritt Übungen kennenlernen, um Third-Party Bibliotheken in unserem Projekt zu verwenden. **NPM** ist ein Werkzeug um Abhängigkeit-Verwaltung systematisch durchzuführen. Hierzu muss man zwischen Konzept und Werkzeug auseinander halten. Man benutzt ein möglichst zeitgemäßes Werkzeug um die Abhängigkeit-Verwaltung zu automatisieren. So ist die Zustand des Software-Projektes zur jeder Zeit nachvollziehbar.

C.1.1. Installation der notwendige Software

Der Installationsprozess in diesem Abschnitt wird einmalig durchgeführt, um das Paket **NPM** samt seiner Infrastruktur als Abhängigkeit-Verwaltung-Werkzeug zu nutzen

Fingerübung C.1: NPM Installation

Installieren Sie auf Ihrem Rechner das Programm **Node**. Die Installation ist erfolgreich, wenn Sie die Befehle **node** und **npm** aus einem Terminal z.B. **cmd** aufrufen könne (Abbildungen C.1 und C.2).

- Für ein Linux-Betriebssystem kann man die Seite <https://nodejs.org/en/download/package-manager/> konsultieren, um die passende Installation für bestimmte Linux-Distribution zu erzielen. In Linux haben Sie in der Regel ein Shell-Programm wie „Konsole“ oder „Terminal“, je nach Ihre Distribution.
- Für Windows und macOS kann man die geeignete Installation-Dateien aus der Seite <https://nodejs.org/de/download/> herunterladen und sie wie üblichen Software Pakten installieren. In Windows benutzen Sie das **cmd** oder das **PowerShell** als Terminal. In macOS benutzen Sie das Programm **Terminal**.

```
C:\Windows\system32>node --version  
v14.15.1
```

```
C:\Windows\system32>npm --version  
6.14.8
```

```
C:\Windows\system32>
```

Abbildung C.1.: Erfolgreiche Installation von NPM, Beispielausgabe im Windows

```
C:\Windows\system32>node --version
Der Befehl "node" ist entweder falsch geschrieben oder
konnte nicht gefunden werden.

C:\Windows\system32>npm --version
Der Befehl "npm" ist entweder falsch geschrieben oder
konnte nicht gefunden werden.

C:\Windows\system32>
```

Abbildung C.2.: Erfolgreiche Installation von NPM, Beispielausgabe im Windows

Fingerübung C.2: Installation von node-gyp und windows-build-tools (optional)

Sie brauchen die NPM-Paketen `node-gyp` ggf. `windows-build-tool`, wenn Ihr Projekt C/C++ Code enthält und Sie in Windows entwickeln. Das Paket `node-gyp` ist optional in anderen Betriebssystemen. Benutzen Sie kein Windows, können Sie diese Übung überspringen.

1. Öffnen Sie ein Terminal, in Linux ist das Terminal Ihrer Distribution –etwa **Konsole** in eine KDE-Umgebung–, in Windows ist das `cmd`- oder `PowerShell`-Terminal. Beim Windows müssen Sie das Terminal mit dem Administrator-Recht öffnen.
2. Installieren Sie das Paket `node-gyp` wie folgt:
 - a) In Linux/macOS: `sudo npm install --global node-gyp`
 - b) In Windows: `npm install --global node-gyp windows-build-tools`
3. Testen Sie ob die Installation erfolgreich ausgeführt wird, indem Sie den Befehl `node-gyp` eingibt. Die Abbildung C.3 zeigt die Ausgabe des Befehls.

Studieren Sie die Dokumentation der beide Paketen. Sie kann man aus <https://www.npmjs.com/> finden.

```
C:\Windows\system32>node-gyp

Usage: node-gyp <command> [options]

where <command> is one of:
- build - Invokes `msbuild` and builds the module
- clean - Removes any generated build files and the "out" dir
- configure - Generates MSVC project files for the current module
- rebuild - Runs "clean", "configure" and "build" all at once
- install - Install node development files for the specified node version.
- list - Prints a listing of the currently installed node development files
- remove - Removes the node development files for the specified version

node-gyp@7.1.2 C:\Users\latex-user\AppData\Roaming\npm\node_modules\node-gyp
node@14.15.1

C:\Windows\system32>
```

Abbildung C.3.: Die Installation von `node-gyp` ist erfolgreich abgeschlossen

C.1.2. Beispiel: Aufbau eines NPM-Projekt

Das Beispiel-Projekt **plot-experiment** ist ein JavaScript für Frontend. Es ist für Browser konzipiert. Der JavaScript-Code läuft in einer Browser-Umgebung. Das eigene Codebase des Projektes sowie Laufzeit-Code seines Abhängigkeiten bestehen aus nur JavaScript. Die Source Code benötigt daher keine weitere Kompilierung. Eine optionale indirekte Abhängigkeit ist allerdings in C geschrieben und benötigt einen C-Compiler.

Fingerübung C.3: Projekt **plot-experiment**

Studieren Sie das Projekt **plot-experiment**!

1. Aktualisieren Sie Ihr Lokale Repository **example-projects-2026** via `git pull origin master`.
2. Öffnen Sie das Projekt **plot-experiment** in WebStorm.
3. Überprüfen Sie ob Sie einen Terminal-Subwindow (Links Unten auf dem Status-Menü) in WebStorm öffnen können.
4. Lesen Sie die Datei **README.md** sowie die Datei **package.json** im Projekt und beantworten Sie folgenden Fragen:
 - a) Welche Rolle spielt die Datei **package.json**?
 - b) In welchem Ordner befinden sich die eigene Sourcecode Dateien des Projektes?
 - c) Welche 3rd-Party Bibliotheken (Abhängigkeiten) werden im Projekt für welche Zweck verwendet?
 - d) Wie man die 3rd-Party Bibliotheken installieren?

Fingerübung C.4: Workflow mit **parcel-bundler**

Entwickeln Sie das Projekt **plot-experiment** weiter!

1. Wie startet man einen HTTP-Server mit Hilfe des Paket **parcel-bundler** für die Entwicklung? Zum Beispiel: Wie man die Datei **jsxgraph-experiment.html** über einen HTTP-Server im Browser öffnet?
2. Wie werden Unit-Testen organisiert? Wie startet man die Unit-Testen?
3. Konzeptionelle Frage: warum brauchen wir ein produktives Build? Wie können wir sicherstellen, dass alle Abhängigkeiten eines Projektes auch zusammengepackt und zur Laufzeit des Projektes auch vom Rechner der Ende-Benutzer geladen wird?
4. Wie erstellt man ein produktives Build des Projektes (den Ordner **dist**) automatisch? Was fällt Ihnen aus beim Studieren von ***.html**-Dateien des Projekt?
5. Studieren Sie die Dokumentation von das NPM-Paket **jsxgraph**, sowie die Dateien **src/jsxgraph-experiment.*** um:
 - die Punkten mit dem Koordinate $P_1(1, 4)$; $P_2(3, 7)$; $P_3(-1, 1)$;
 - die Gerade $y = 3 \cdot x + 1$

| auf dem Browser zu zeichnen.

Literatur

- [1] Douglas Crockford. *JavaScript: The Good Parts*. 1. Aufl. O'Reilly and Associates, 1. Mai 2008. ISBN: 978-0-596-55487-3.
- [2] ECMA International, Hrsg. *ECMAScript® 2019 Language Specification*. Juli 2019. URL: <https://www.ecma-international.org/ecma-262/10.0/index.html#Title>. Nur als Referenz geeignet, nicht als Lehrbuch geeignet.
- [3] Maurits Cornelis Escher. *Drawing Hands*. Lithograph. Jan. 1948. URL: <https://mcescher.com/gallery/back-in-holland/>.
- [4] Maurits Cornelis Escher. *Symmetry 131*. Arbeiten auf Papier, Ink. 1967. URL: <https://todo.fillit/gallery/???>.
- [5] David Flanagan. *JavaScript: Das umfassende Referenzwerk*. Übers. von Lars Schulten und Ulrich Speidel. 3. Aufl. Deutsche Ausgabe der 5. Auflage. Köln: O'Reilly Germany, 2007. ISBN: 978-3-897-21491-0. Als Nachschlagewerk oder Lehrbuch für Fortgeschrittene sehr gut geeignet.
- [6] Kurt Meyberg und Peter Vachenauer. *Höhere Mathematik 1: Differential- und Integralrechnung Vektor- und Matrizenrechnung*. Springer, Aug. 2003. ISBN: 978-3-540-41850-4.
- [7] MDN Web Docs, Hrsg. *BigInt*. Mozilla. 17. Sep. 2019 18:41. URL: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/BigInt.
- [8] MDN Web Docs, Hrsg. *export*. Mozilla. 28. Nov. 2019 10:15. URL: <https://developer.mozilla.org/en/docs/Web/JavaScript/Reference/Statements/export>.
- [9] MDN Web Docs, Hrsg. *import*. Mozilla. 28. Nov. 2019 10:15. URL: <https://developer.mozilla.org/en/docs/Web/JavaScript/Reference/Statements/import>.
- [10] MDN Web Docs, Hrsg. *Math*. Mozilla. 29. Juli 2019 14:50. URL: https://developer.mozilla.org/en/docs/Web/JavaScript/Reference/Global_Objects/Math.
- [11] MDN Web Docs, Hrsg. *Number*. Mozilla. 29. Juli 2019 14:50. URL: https://developer.mozilla.org/en/docs/Web/JavaScript/Reference/Global_Objects/Number.
- [12] MDN Web Docs, Hrsg. *Number.MIN_VALUE*. Mozilla. 29. Juli 2019 14:50. URL: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Number/MIN_VALUE.
- [13] MDN Web Docs, Hrsg. *String*. Mozilla. 6. Aug. 2019 04:30. URL: https://developer.mozilla.org/en/docs/Web/JavaScript/Reference/Global_Objects/String.
- [14] MDN Web Docs, Hrsg. *JavaScript reference*. Mozilla. URL: <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference>.
- [15] MDN Web Docs, Hrsg. *Node.nodeType*. Mozilla. URL: <https://developer.mozilla.org/en-US/docs/Web/API/Node/nodeType>.
- [16] Robert Sedgewick und Kenwin Wayne. *Computer Science: An Interdisciplinary Approach*. Addison-Wesley Professional, Juli 2016. ISBN: 978-0-134-07642-3.

- [17] The World Wide Web Consortium, Hrsg. *Document Object Model (DOM) Technical Reports*. 7. Apr. 2004. URL: <https://www.w3.org/DOM/DOMTR>.
- [18] Wikipedia, Hrsg. *Schaltjahr*. 5. Sep. 2019 11:40. URL: <https://de.wikipedia.org/wiki/Schaltjahr>.